Optimización Heurística de Algoritmos de Aprendizaje Automático Mediante Enjambre de Partículas

Álvaro Pleguezuelos Escobar

Tutor: Rafael del Vado Vírseda

Trabajo de Fin de Grado Curso 2024/2025



Facultad de Ciencias Matemáticas

Grado en matemáticas

20 de febrero de 2025

Agradecimientos

Este trabajo pone fin a una de las etapas más bonitas de mi vida y quiero expresar mi agradecimiento a las personas que me han acompañado durante el camino.

Empiezo por mi pilar fundamental, mi apoyo incondicional, María, la primera que va a leer mi trabajo de principio a fin sin tan si quiera sugerírselo, la que me ha levantado en cada suspenso y ha estado en los malos momentos, empezando por el de este curso. Sin ti este camino no habría sido ni la mitad de bonito ni especial. Gracias y mil gracias.

A mi madre, por confiar ciegamente en mí sin importar el momento o el lugar, por facilitarme la vida, aguantarme y darme toques de atención en los momentos necesarios. Gracias por esos 'Álvaro, no te veo estudiando', por tomarme la lección cuando era un enano, por mil cosas más que me llevarían otras cien páginas. Gracias mamá, te quiero.

A mi padre, realmente la verdadera razón por la que he hecho estos estudios. Desde pequeño despertaste en mi la curiosidad por el razonamiento y entender el por qué de las matemáticas, el ir más allá. Por ser el primero en responder a mis por qués cuando en el colegio no sabían responderme. De alguna forma esto también es tuyo. Gracias papá.

A mi alma gemela, hermana de sangre y de alma, Malena. Aunque seas la pequeña siempre has sido y serás un referente en mi vida por tu disciplina, valentía y dedicación a lo que te importa. Mil gracias por ser quien eres.

A mi abuela Marga, por esos paseos por Princesa hasta Plaza de España y comilonas después de Álgebra. Gracias abuela, te quiero.

A toda la tropa Matasuegras, Juan, Pablo, Miguel, Pelayo, Mini, Nacho y Casado, compañeros de vida, con subidas y bajadas, pero siempre ahí cuando hace falta. Estoy convencido de que vendrán años buenos, pero dejadme dudar de si lo van a ser tanto como los últimos seis, han sido de libro. Gracias chavales.

A mis pocos pero verdaderos amigos de la facultad, Ramón, Dani y Adri, permitidme decir que sin pandemia habríamos tardado más en sacarnos la carrera por lo bien que nos lo pasábamos fuera de la facultad. Compartir los suspensos con vosotros hacía que se llevara mejor. Gracias por ser mis compañeros de fatigas y por los buenos momentos juntos.

A mi tutor Rafa, por el profesor que eres, pero sobre todo, por encontrar ratos para mi en este momento de tu vida. He apreciado cada correo contestado, de verdad. Muchas gracias por acompañarme en este TFG.

Por último, al gran Rober Tomé, por hacer entendible el comienzo del grado, por tus ganas de enseñar y por tu cercanía. Eres toda una institución en la facultad sin pertenecer oficialmente a ella. Gracias.

Resumen

El Aprendizaje Automático es una herramienta clave para resolver problemas de diversa índole. Sin embargo, su precisión depende del ajuste adecuado de los hiperparámetros, cuya optimización puede ser costosa computacionalmente, especialmente en espacios de gran dimensión. En este contexto, las técnicas heurísticas se posicionan como una alternativa viable frente a métodos exactos como *Grid Search*. Este trabajo estudia el algoritmo metaheurístico *Particle Swarm Optimization* tanto teórica como experimentalmente, aplicándolo a la clasificación por ránking de jugadores de tenis del circuito ATP. Los resultados muestran que el PSO es una opción eficiente aunque requiere un ajuste adecuado de parámetros para su correcto funcionamiento.

Palabras Clave: *Aprendizaje Automático, PSO, Enjambre de Partículas, Optimización, Heurística, Tenis.*

Abstract

Machine Learning is a key tool for solving diverse problems. However, its accuracy depends on properly tuning hyperparameters, whose optimization can be computationally expensive, especially in high-dimensional spaces. In this context, heuristic techniques emerge as a viable alternative to exact methods such as *Grid Search*. This work studies the metaheuristic algorithm *Particle Swarm Optimization* both theoretically and experimentally, applying it to the ranking classification of ATP circuit tennis players. Results show that PSO is an efficient option, although proper parameter tuning is necessary for its proper performance.

Keywords: *Machine Learning, PSO, Particle Swarm Optimization, Optimization, Heuristics, Tennis.*

Índice general

1. Introducción **1**

1.1. Motivación y objetivos 1

1.2. Contexto y antecedentes del trabajo 2

1.2.1. Aprendizaje Automático 2

1.2.2. Optimización matemática 4

1.2.3. Optimización heurística 6

1.2.4. Estructura del trabajo 6

2. Algoritmos de Aprendizaje Automático **7**

2.1. Aprendizaje supervisado 7

2.1.1. k-NN 7

2.1.2. Árboles de Decisión 8

2.1.3. Bagging 9

2.1.4. Random Forest 9

2.1.5. Support Vector Machines (SVM) 10

2.1.6. Logistic Regression 11

2.2. Aprendizaje no supervisado 11

3. Optimización heurística mediante enjambre de partículas **13**

3.1. Fundamentos físicos 14

3.2. Funciones y operadores 17

3.2.1. Initialize() 18

3.2.2. Dynamic() 19

3.2.3. Best() 22

3.2.4. Stop() 23

3.3. Mutaciones 23

4. Optimización del Aprendizaje Automático mediante enjambre de partículas 25

4.1. Introducción 25

4.2. Hiperparámetros 25

4.2.1. Logistic Regression: 26

4.2.2. k-NN 26

4.2.3. Support Vector Machines (SVM) 27

4.2.4. Random Forest 27

4.3. Ajuste de hiperparámetros 29

4.4. Caso de estudio 30

4.5. Preprocesamiento del dataset 30

4.5.1. Cálculo de Métricas Agregadas 30

4.5.2. Clasificación por Grupo de Ranking 31

4.5.3. Variables del Dataset Final 31

4.5.4. Resumen del Procesamiento 32

4.5.5. Resultados 34

4.5.6. Análisis estadístico 41

5. Conclusiones y trabajo futuro **53**

5.1. Conclusiones 53

5.2. Trabajo futuro 54

6. Anexo **55**

Capítulo 1

Introducción

En un mundo tan cambiante, vanguardista y tecnológico como el actual, la Inteligencia Artifical se posiciona como uno de los grandes puntos de inflexión en la historia de la humanidad. Aunque todavía esté en sus comienzos, se vislumbran indicios de lo que puede ser una gran revolución. Sin embargo, los centros de datos masivos, los nuevos semiconductores y el desarrollo de nuevos GPU's no parecen ser suficientes para dar los siguientes pasos. La IA parece tener solamente dos alternativas si quiere seguir creciendo al ritmo que lo ha estado haciendo hasta el momento. La primera y más largoplacista es desarrollar una nueva tecnología a nivel hardware que sea capaz de alimentar todo lo que la IA necesita para evolucionar, por ejemplo a través de la computación cuántica. La segunda opción es desarrollar y mejorar la tecnología software creando nuevos algoritmos, métodos de resolución y optimización que no sean tan masivamente costosos conservando la precisión y eficacia. Aquí es donde las heurísticas como el PSO pueden jugar un papel fundamental, dado que, aunque sean modelos que no garanticen una mejor solución, pueden dar resultados igual de buenos o inlcuso mejores, como se ha demostrado en este trabajo.

1.1. Motivación y objetivos

Cursando las asignaturas de Ciencias Computacionales en la Facultad de Matemáticas de la UCM, me empecé a interesar por la Inteligencia Artificial y sus aplicaciones. Este trabajo también es fruto del conocimiento adquirido durante mis estudios en la Universidad de Gante, donde tuve la oportunidad de cursar la asignatura *Machine Learning*. Las aspiraciones con este trabajo son:

- Adquirir soltura en el lenguaje de programación *Python* y en herramientas estadísticas como JASP.
- Poner a prueba mis conocimientos de *Machine Learning*, así como ampliarlos con un estudio más detallado de sus fundamentos.

- Profundizar en métodos de optimización heurísticos para ver su aplicabilidad real comparándolos con otros métodos de optimización más comunes.
- Desarrollar en un proyecto el conocimiento adquirido en la asignatura *Modelos Estadísticos*.
- Introducirme en esta nueva tecnología con vistas a un futuro laboral.

1.2. Contexto y antecedentes del trabajo

Según [1], la IA o Inteligencia Artificial es una subdisciplina de las Ciencias Computacionales que se encarga de la construcción de sistemas y algoritmos que se puedan aproximar a la deducción y al razonamiento humanos. Entre otras tareas que aborda la IA se encuentran el reconocimiento de voz y de imagen, la comprensión del lenguaje natural, la toma de decisiones o la resolución de problemas de diferentes tipos. La IA tiene un fuerte fundamento matemático, combinando disciplinas como la lógica, la estadística, la teoría de la computación, el álgebra e incluso la topología, entre muchas otras.

Dentro de la IA, existen diferentes enfoques desde los sistemas de IA débil, que están diseñados para realizar tareas específicas, más simples y tangibles, hasta la IA fuerte, que intenta simular la inteligencia humana de manera más completa. Esta nueva herramienta puede ser implementada en casi todos los campos, la medicina, la industria, el transporte, las finanzas, e incluso el entretenimiento, y también cambia completamente la forma en que interactuamos con la tecnología mientras aumenta la efectividad de innumerables procesos. Hay expertos que se atreven a decir que la IA será más importante que la famosa revolución industrial, y que cualquier negocio que no se adapte a esta nueva tecnología se quedará atrás en el mercado.

1.2.1. Aprendizaje Automático

El Aprendizaje Automático o *Machine Learning* es una forma de Inteligencia Artificial que realiza predicciones en base a un conjunto de datos proporcionado. A través de algoritmos y modelos matemáticos se identifican patrones y se realizan predicciones a partir de ellos. Existen gran cantidad de algoritmos. No obstante, en este documento se tendrán en cuenta las dos clases principales: **algoritmos supervisados** y **algoritmos no supervisados**.

El auge del *Machine Learning* se debe a varios factores clave. Entre ellos se encuentra el **acceso a datos masivos** generados continuamente por dispositivos o redes sociales, entre muchos otros. Además, destaca la capacidad computacional que está adquiriendo el hardware hoy en día, con los conocidos **High Performance Computers** (HPC) cada vez más potentes, los revolucionarios GPU's de compañías como NVIDIA o el desarrollo de plataformas en la nube, **Cloud Computing** en inglés, como AWS de Amazon o Azure

de Windows, que dan acceso, precisamente, al usuario de a pie a esta nueva y revolucionaria tecnología.

Un proyecto de *Machine Learning* se compone de varios pasos fundamentales. A continuación se hace una breve explicación de los mismos:

- **Paso 1:** Trata de recolectar datos que sean informativos, de calidad y cuantiosos para el problema que se quiera resolver.

- **Paso 2:** Seguidamente, se hace una limpieza de datos en la que se suelen eliminar posibles datos no informativos denominados *outliers*, pero también se completan valores faltantes y se transforman variables para que sean interpretables o facilitar su comprensión.

- **Paso 3:** A continuación, se dividen los datos en dos partes: una para entrenamiento del modelo, que suele constar de un 80 % de los datos aproximadamente, y la otra de prueba, que suele constar del 20 % restante, para evaluar el funcionamiento del modelo. Es importante esta división, dado que se necesita la independencia entre los datos de entrenamiento y los de prueba para evaluar correctamente el modelo. Si hacemos un test sobre un dato que ha sido utilizado para entrenar el modelo, la evaluación resultante estará sesgada. Lo que se pretende es crear un modelo que generalice, no uno que funcione bien con los datos usados para entrenarlo.

- **Paso 4:** A continuación se selecciona el modelo, o los modelos, que se va a usar para resolver el problema. La elección de uno u otro va a ser más o menos conveniente en función del problema que se quiera resolver, siendo tarea del usuario el interpretar correctamente cuál es el modelo que el problema requiere para una resolución efectiva. Los tipos de modelos se dividen en tres grandes categorías: **Aprendizaje Supervisado**, **Aprendizaje no Supervisado** y **Aprendizaje por Refuerzo**.

- **Paso 5:** Tras haber elegido el modelo llega el momento de elegir una métrica sobre la que evaluarlo. Si es un problema de clasificación se pueden utilizar medidas como la *Accuracy* (Exactitud), *Recall* (Precisión) o *F1-Score*, y si es de regresión, medidas como *Mean Squared Error (MSE)*, *Mean Absolute Error (MAE)* y *Mean Squared Root Error (MSRE)*, entre muchas otras. Es competencia del usuario elegir la métrica adecuada en función del problema que se quiera resolver. Por ejemplo, si lo que se quiere es no computar falsos negativos en un problema de clasificación, como puede ser determinar si un tumor es maligno o benigno en función de una imagen, el *Recall* se perfila como una buena métrica, mientras que si el problema trata de diferenciar entre clases que tienen la misma importancia, dada una imagen, como determinar si es gato o perro, *Accuracy* es más adecuada. A la métrica elegida para evaluar el desempeño del modelo se la conoce más comunmente como función objetivo.

- **Paso 6:** Para validar el modelo se pueden utilizar diversas técnicas [2]. La más básica, llamada *Hold-Out Method*, entrena el modelo uniformemente sobre el conjunto de entrenamiento y lo valida en el conjunto de prueba como se muestra en la Figura 6.1a.

  De la misma forma, se puede usar *Hold-Out* para hacer una selección de hiperparámetros o incluso de modelo si el usuario no está seguro de cuál utilizar para su problema, creando un conjunto de validación dentro del conjunto de entrenamiento, como se muestra en la Figura 6.1b.

  Existen otros métodos más fiables y que ayudan al modelo a entrenarse mejor, asegurando que no se ajusta a un determinado conjunto de datos. Estos métodos utilizan validación cruzada y están basados en un proceso iterativo de *Hold-Out* pero escogiendo diferentes conjuntos de datos en cada iteración. Entre ellos se encuentran *k-Fold Cross-Validation* en el que se divide el conjunto de entrenamiento del dataset en k subconjuntos de igual tamaño, k-1 conjuntos dedicados al entrenamiento y uno a la validación. Este proceso es repetido k veces eligiendo uno de los k subconjuntos diferentes en cada iteración para la validación y se promedian los resultados obtenidos.

  Existe un caso específico de *Cross-Validation* en el que el k elegido es precisamente el número total de datos de entrenamiento, dejando un solo dato para validar y $k-1$ para entrenar el modelo. A este método se le conoce como *Leave One Out Cross-Validation (LOOCV)*, caracterizado por su alto coste computacional.

  Por último se destaca el *Nested Cross-Validation*, utilizado especialmente cuando se está haciendo selección entre varios modelos y adicionalmente se ajustan hiperparámetros en los mismos. Esto sirve fundamentalmente para hacer un análisis exhaustivo y posterior selección entre los modelos que el usuario considera potenciales soluciones para resolver el problema. Según [3], el uso de *Cross-Validation* puede llevar al usuario a un error de generalización como consecuencia de estudiar la selección de modelos y ajuste hiperparamétrico en los mismos subconjuntos.

  El número de subconjuntos k_1 y k_2 en los bucles interior y exterior los elige el usuario. En la Figura 6.1c se puede observar la elección de $k_1 = 3$ en el bucle exterior y $k_2 = 2$ en el bucle interior. Al igual que el $LOOCV$, este método es de alto coste computacional, teniendo en cuenta que el total de modelos a evaluar se incrementa sustancialmente al haber un bucle encajado dentro de otro.

- **Paso 7:** Una vez desplegado el modelo en producción, el aprendizaje puede continuar. De hecho, el aprendizaje continuo es fundamental hoy en día en herramientas de IA como $Chat-GPT$ que aprenden a base de ingestar nuevos datos masivos cada día proporcionados por los usuarios que las utilizan y monitorizadas continuamente para ir mejorando su rendimiento.

1.2.2. Optimización matemática

La optimización matemática es una rama de las matemáticas aplicadas que tiene como objeto encontrar los mejores elementos (mínimos o máximos) de un conjunto de soluciones

posibles, conocido como el espacio de búsqueda. A continuación, se presentan algunos conceptos clave que la componen [4]:

- **Función Objetivo:** Es la función que se desea minimizar o maximizar. En problemas de optimización, esta función describe el objetivo principal, como minimizar costos, maximizar beneficios, o mejorar el rendimiento.
- **Variables de Decisión:** Son las variables independientes que se ajustan para encontrar la mejor solución. Éstas pueden ser continuas (como cantidades numéricas) o discretas (como opciones binarias).
- **Restricciones:** Las restricciones definen los límites dentro de los cuales deben moverse las variables de decisión. Pueden ser igualdades (ecuaciones) o desigualdades (inecuaciones) que reflejan limitaciones prácticas del problema.
- **Puntos Factibles:** Son aquellos puntos que cumplen con todas las restricciones impuestas. El conjunto de todos los puntos factibles conforma el espacio de búsqueda.
- **Solución Óptima:** Es el punto factible que minimiza o maximiza la función objetivo. Dependiendo del tipo de problema, puede haber una única solución óptima o múltiples soluciones equivalentes.

Dentro de la optimización matemática, se pueden encontrar problemas de distinta índole, lo cual ha generado la necesidad de crear distintos tipos de optimización, como la de tipo lineal, no lineal, convexa, combinatoria, etc.

Uno de los factores más importantes de la optimización matemática es que garantiza la o las soluciones óptimas del espacio de búsqueda, si el problema es factible, bien sea por métodos exactos como puede ser el *Simplex* o bien por métodos de aproximación como el *Método de Newton*. Empiezan en un punto inicial de las variables independientes en el espacio de búsqueda y van generando una secuencia en la que se va mejorando la solución. La secuencia generada por este proceso iterativo recibe el nombre de 'iteraciones'.

Independientemente del algoritmo que se escoja para el problema de optimización a resolver, este debe poseer las siguientes tres propiedades:

1. Robustez: El algoritmo tiene que ser capaz de tener un buen comportamiento en una gran variedad de problemas de su clase.
2. Eficacia: Tiene que ser computacionalmente accesible tanto a nivel de almacenamiento como a nivel temporal.
3. Precisón: El algoritmo debe ser capaz de garantizar una convergencia a la solución óptima.

1.2.3. Optimización heurística

¿Qué sucede cuando nos enfrentamos a problemas complejos en los que los métodos exactos carecen de eficiencia o son computacionalmente muy costosos? Esta pregunta es la motivación principal de la creación de la Optimización Heurística, una clase de algoritmos que buscan soluciones aproximadas sin garantizar que la convergencia sea el óptimo global del espacio de búsqueda. Las heurísticas se diferencian de los métodos exactos en que estos encuentran soluciones razonables en menos tiempo y con una memoria computacional de menor coste. Estos problemas ganan importancia en problemas donde el espacio de búsqueda es muy grande, o simplemente el nivel de procesamiento del usuario no es lo suficientemente bueno como para poder abordar el problema de manera exacta.

Según [5] las heurísticas se clasifican en dos clases:

1. Las **heurísticas de evaluación** buscan la exploración del espacio de búsqueda a través de funciones que asignan valores a soluciones parciales. Estas heurísticas utilizan una función de evaluación para estimar la calidad de las soluciones priorizando las que parecen más prometedoras sin generar nuevas soluciones.
2. Las **heurísticas de generación**, a diferencia de las de evaluación, buscan crear nuevas soluciones a partir de las existentes creando nuevos estados o soluciones, en lugar de sólamente hacer la evaluación. Se utilizan principlamente para ampliar la exploración del espacio de búsqueda y evitar el estancamiento en óptimos locales.

A continuación se detallan algunos ejemplos de heurísticas comunes tanto evolutivas como generativas:

Método Heurístico	**Clase**	**Tipo de Algoritmo**
A*	Evaluación	Algoritmo de búsqueda informada
Best-First Search	Evaluación	Algoritmo de búsqueda informada
Recocido Simulado (SA)	Generación	Algoritmo metaheurístico
Algoritmos Genéticos	Generación	Metaheurística basada en evolución
PSO (Particle Swarm Optimization)	Generación	Metaheurística basada en inteligencia de enjambres

Cuadro 1.1: Clasificación de Métodos Heurísticos según Evaluación o Generación

1.2.4. Estructura del trabajo

La memoria ha sido estructurada en cinco capítulos. El capítulo actual contiene una breve introducción dirigida a motivar el estudio llevado a cabo en este trabajo. En el Capítulo 2 se explican brevemente los métodos de Aprendizaje Automático utilizados en la parte experimental. En el Capítulo 3 se abordan los fundamentos del PSO a nivel teórico y sus posibles configuraciones. En el Capítulo 4 se realiza un estudio del PSO de un caso concreto y se explican estadísticamente los resultados obtenidos. Se finaliza el trabajo con el Capítulo 5, en el que se detallan las conclusiones y posibles líneas de investigación que continúen con el estudio de las heurísticas.

Capítulo 2

Algoritmos de Aprendizaje Automático

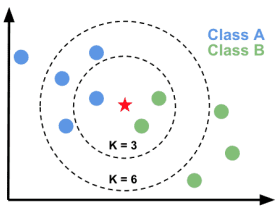
En este capítulo se introducen las dos clases de Aprendizaje Automático que se han utilizado en capítulos posteriores, así como aplicaciones de los mismos. Se han estudiado los dos principales tipos de aprendizaje, aprendizaje supervisado y aprendizaje no supervisado. Dependiendo del caso que se esté estudiando, o incluso de la variable respuesta que se quiera obtener, será más conveniente usar o uno u otro. Los modelos de aprendizaje por refuerzo no se han tratado debido a su irrelevancia en este trabajo. Las principales fuentes de información han sido [6] y [7].

2.1. Aprendizaje supervisado

El aprendizaje supervisado es el encargado de entrenar un modelo dados unos datos previamente etiquetados, es decir, el *input* y *output* son conocidos. De esta forma, el algoritmo incrementa su exactitud a medida que va entrenando el modelo. Se van a analizar los siguientes modelos: *Support Vector Machines* (SVM), *k-Nearest Neighbour* (k-NN), *Bagging*, *Decision Tree*, *Random Forest* y *Logistic Regression*.

2.1.1. k-NN

Este modelo es uno de los más sencillos e intuitivos del conjunto de supervisados. Está basado principalmente en asignar la etiqueta predominante asociada a sus k vecinos más cercanos. El parámetro k es a elección del usuario.



Se muestra un ejemplo en la Figura 2.1.1 donde se puede observar cómo cambiando el parámetro k puede variar la clasificación de una nueva instancia entre una clase y otra.

2.1.2. Árboles de Decisión

Los *Decision Trees* o Árboles de Decisión en español, son, al igual que k-NN, de fácil comprensión y adecuados para problemas en los que haya que procesar grandes cantidades de datos. Su principal desventaja es el riesgo de *overfitting* haciendo árboles demasiado profundos que se ajusten demasiado bien al conjunto de entrenamiento y penalizando una buena generalización. Esto ocurre especialmente cuando hay varianzas elevadas en variables del *dataset*.

Los árboles se componen de tres diferentes tipos de nodos:

1. **Nodo Raíz:** La entrada de este nodo es el *dataset* en su totalidad. Se subdivide en dos o más sub-nodos llamados nodos de decisión, resultados de una función sobre uno o más atributos. Estas subdivisiones pueden resultar en rangos o categorías que diferencien, y en consecuencia separen, el dataset.

2. **Nodo de Decisión:** Este tipo de nodos tiene tanto ramas de entrada como de salida. Son los nodos que ramifican el dataset o, análogamente, construyen las ramas del árbol.

3. **Hojas:** Este nodo tiene ramas de entrada pero no de salida. Representan los resultados finales de la rama que se ha generado desde el nodo raíz, pasando por los nodos de decisión hasta llegar a él.

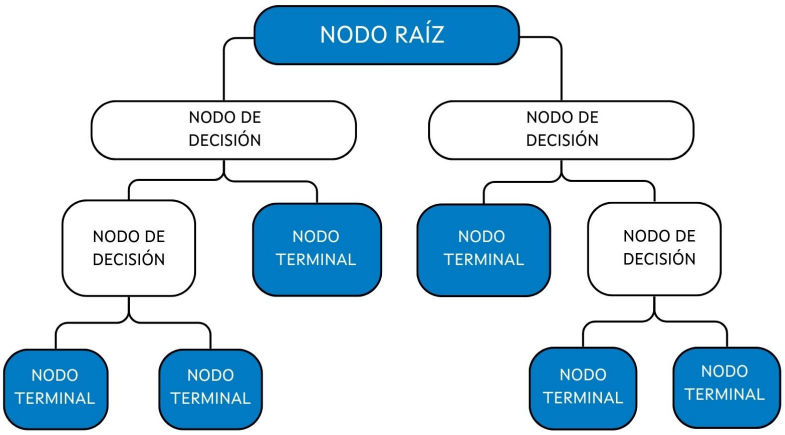


Figura 2.1: Estructura de Árbol de Decisión [8]

El entrenamiento del árbol no es más que un ajuste de los atributos que conforman los nodos para subdividir el dataset lo mejor posible. La elección de los atributos se escoge en función de lo informativo que sea ese atributo, es decir, de la ganancia de información, tal y como se explica en la sección 4.2.4 en mayor profundidad. Existen dos tipos de árboles de decisión según el problema que se quiera abordar: si es de variable discreta se suele utilizar el *Classification Tree*, mientras que si se pretende abordar un problema de variable continua, se suele utilizar *Regression Tree*.

2.1.3. Bagging

El *Bagging*, ilustrado en la Figura 2.2, es una técnica de generación de subconjuntos conformados por una selección de instancias de manera aleatoria con reemplazamiento, es decir, una instancia puede estar contenida en dos subconjuntos simultáneamente. Una vez generados los subconjuntos, se ajusta un modelo para cada uno. Finalmente, a la hora de hacer una predicción, se escoge el valor más repetido en caso de ser un problema de clasificación, y el valor medio devuelto por todos los modelos en el caso de regresión . El bagging ayuda a reducir el ruido causado por la varianza al utilizar varios subconjuntos para el entrenamiento de los diferentes modelos, y de esta forma, tener en cuenta todas las clasificaciones o regresiones realizadas.

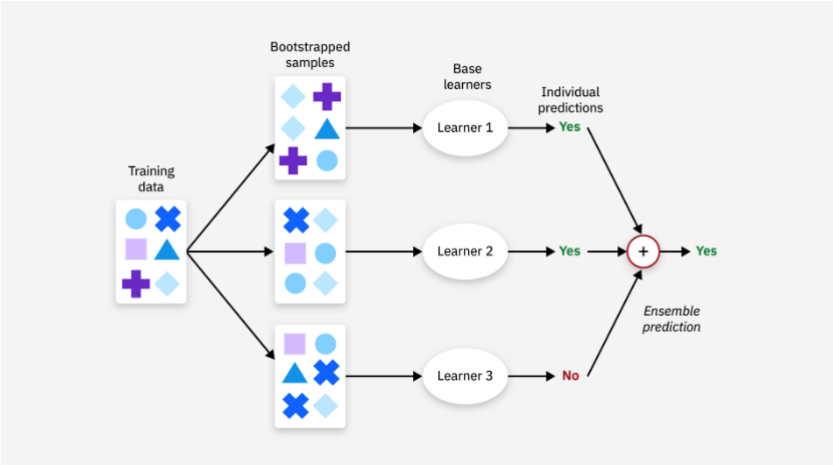


Figura 2.2: Ejemplo de Bagging [9]

2.1.4. Random Forest

Los *Random Forests* o Bosques Aleatorios en español no son más que la realización de un *Bagging* de Árboles de Decisión. Posee todas las ventajas de los Árboles de Decisión y se mitiga considerablemente el riesgo de *overfitting* y sensibilidad a varianzas elevadas gracias al efecto del *Bagging*. El principal incoveniente de este modelo de aprendizaje es, sin lugar a dudas, el coste computacional, al tener que generar árboles de decisión en masa, lo que suele conllevar un tiempo de ejecución elevado. Se muestra un ejemplo en la siguiente Figura 2.3:

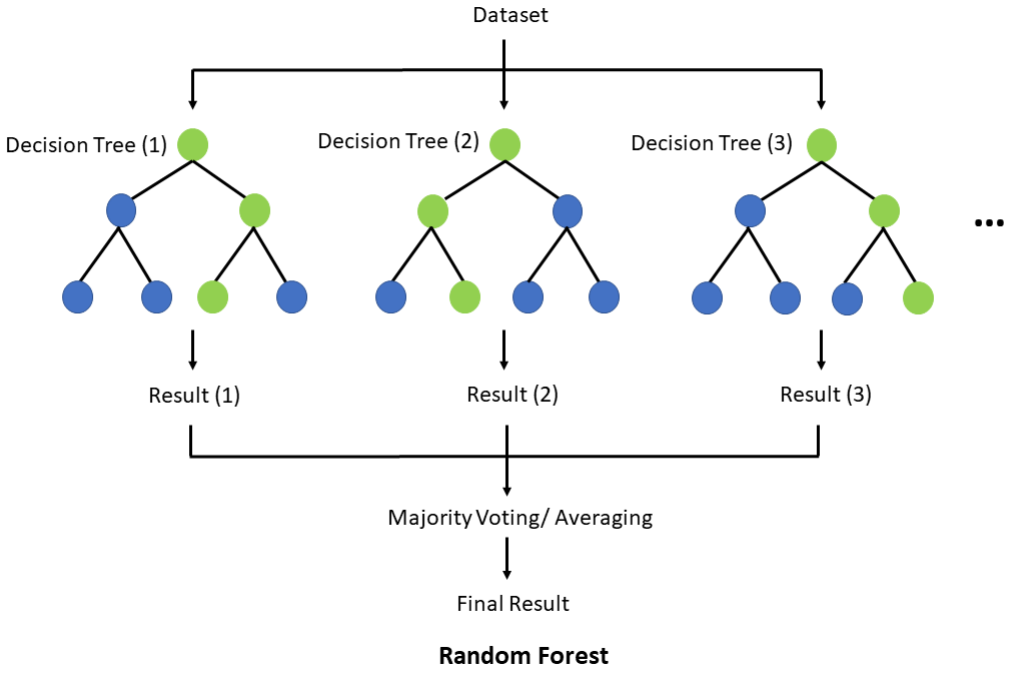


Figura 2.3: Ejemplo de Bosque Aleatorio [10]

2.1.5. Support Vector Machines (SVM)

Este método es utilizado normalmente en problemas de clasificación, aunque también se han desarrollado adaptaciones para problemas de regresión. El objetivo del SVM es encontrar hiperplanos que separen (y asocien) regiones del espacio a las diferentes clases de la variable. Estos hiperplanos se calculan maximizando las distancias de los puntos de diferente clase. Muchas veces se encuentran puntos atípicos que impiden una separación perfecta. Esto se solventa utilizando *soft-margin*, flexibilizando el criterio de separación como se puede ver en la imagen derecha de la Figura 2.4.

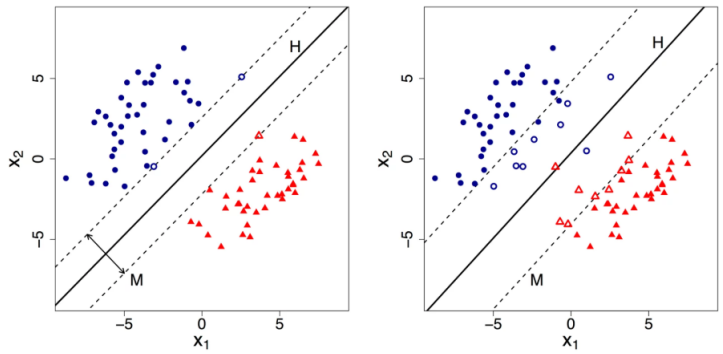


Figura 2.4: Ejemplo de SVM [10]

2.1.6. Logistic Regression

Comúnmente conocido en español por Regresión Logística, es un método de aprendizaje supervisado probabilístico de clasificación, normalmente binaria. A diferencia de los anteriores, este asigna probabilidad de pertenencia a las diferentes categorías y luego devuelve la clase que más probabilidad tenga asignada. Para ello, hace uso de la función *Sigmoid* o logística como se muestra a continuación en la Figura 2.5:

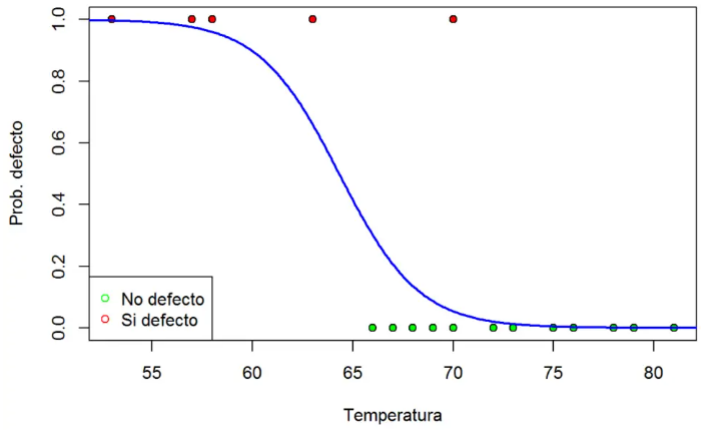


Figura 2.5: Ejemplo de Regresión Logística [11]

2.2. Aprendizaje no supervisado

El aprendizaje no supervisado se ocupa de clasificar y detectar patrones en los datos. Es especialmente valioso cuando no se conocen las características de los datos o no se logra comprender completamente su comportamiento. A diferencia del aprendizaje supervisado, carece de etiquetas preestablecidas, por lo que busca hallar estructuras ocultas dentro de ellos.

Entre los modelos más populares en el aprendizaje no supervisado de agrupamiento se encuentran:

- **K-Means:** Segmenta los datos en k grupos o clusters fundamentándose en la reducción de la distancia entre los puntos que se encuentran en cada grupo.
- **DBSCAN (Agrupamiento Espacial Basado en Densidad de Aplicaciones con Ruido):** Agrupa puntos que están densamente interconectados y puede detectar valores anómalos (outliers).

Entre los modelos de reducción de dimensionalidad cabe destacar:

- **PCA (Análisis de Componentes Principales):** Proyecta los datos en nuevas dimensiones ortogonales llamadas componentes principales, maximizando la variabilidad preservada.
- **t-SNE (t-Distributed Stochastic Neighbor Embedding):** Método no lineal que conserva la estructura de cercanía en un espacio de menor dimensión.

También se encuentran modelos de distribución de datos, como los **Gaussian Mixture Models** (GMM), en los que se representa la distribución de los datos bajo la suposición de que se originan en varias distribuciones normales, y de aprendizaje, basado en representación con algoritmos fundamentados en redes neuronales profundas que facilitan la identificación de características relevantes en datos no etiquetados, como pueden ser los **Autoencoders**.

Tras aportar suficiente contexto, se pasa al núcleo de este trabajo: la explicación teórica del PSO seguida de su aplicación experimental.

Capítulo 3

Optimización heurística mediante enjambre de partículas

En 1995, Russel Eberhart (ingeniero) y James Kennedy (psicólogo) [12] desarrollaron el primer algoritmo metaheurístico evolutivo basado en el comportamiento de conjuntos de animales tales como los bancos de peces, los enjambres de abejas o las bandadas de pájaros. Fueron, precisamente, los bancos de peces los que les llevaron a desarrollar este algoritmo, al observar que los peces del banco se beneficiaban del comportamiento del mismo, desplazándose de forma sinérgica y a la vez aportando conocimiento de su entorno al resto del banco.

El PSO sigue la siguiente lógica: "El conjunto de soluciones al problema de optimización es definido como un enjambre de partículas que puede desplazarse a través del espacio de parámetros trazando trayectorias conducidas por el mejor rendimiento tanto de sí mismos como de sus vecinos" [13].

El algoritmo PSO no es más que una nube de partículas que se mueve por un espacio determinado en busca de una posición que reduce o maximiza los beneficios de su objetivo. En el plano físico, siguiendo con el ejemplo de los bancos de peces, este objetivo podría ser el punto donde se encuentra la comida que, traducido a Aprendizaje Automático, correspondería con la función objetivo. La función de la que hablamos puede ser tanto una función de ganancia, en la que la nube de partículas buscará una posición que intente maximizarla, o de pérdida, en la que intente minimizarla. Se habla de intención a la hora de maximizar o minimizar la función objetivo porque este algoritmo es un optimizador metaheurístico, en el que la convergencia no garantiza el máximo o mínimo global de la función en el espacio de búsqueda, aunque sí garantiza un óptimo local. En consecuencia, a la hora de aplicar este algoritmo, se debe tener precaución con los resultados que se obtienen, no perdiendo de vista su condición heurística.

3.1. Fundamentos físicos

Antes de profundizar en el algoritmo, se explica la naturaleza biológica de sus componentes. A continuación, se muestra una tabla que clarifica la equivalencia matemática de los fundamentos biológicos que se tienen en cuenta en el algoritmo.

Fundamentos biológicos	**Traducción matemática**
Posición de un integrante del colectivo de individuos	Una solución potencial del problema
Las posiciones del colectivo de individuos	Un conjunto de soluciones del problema
Un factor externo que genere la interacción del colectivo (alimento, depredador)	Valor de las funciones de mérito o penalización de cada solución

Cada partícula de la nube se denota como x_i. En consecuencia, una potencial solución del problema va a venir dada por la posición de una partícula i, donde $D \in \mathbb{N}$, es el número de dimensiones o variables de decisión que puede tomar la partícula:

$$x_i = (x_{i1}, x_{i2}, ..., x_{iD}) \tag{3.1}$$

que es parte del conjunto de todas las partículas del enjambre:

$$x = (x_1, ..., x_N) \tag{3.2}$$

Imaginemos un banco de peces buscando comida. Cada uno de los peces busca por sí solo una dirección específica a la que dirigirse. Más tarde, antes del siguiente movimiento, comunican al resto del banco si han encontrado comida o no, y en caso negativo, la mejor posición posible para seguir buscando. Una vez determinado el mejor pez, o lo que es lo mismo, la mejor posición a la que dirigirse en el próximo movimiento, el resto de peces se mueven acorde a su propio criterio, como antes, pero influídos por la posible mejor posición encontrada anteriormente. De esta forma, el banco actúa como una malla en la que cada nodo x_i intenta descubrir la posición óptima, influido por la información que va proporcionando el enjambre x.

Se supone a partir de ahora que la función objetivo f es una función de **pérdida**, y en consecuencia, el propósito de la nube es encontrar valores que la **minimicen**.

Antes de desarrollar el sistema físico, hay que incidir en que la resolución de la ecuación diferencial de segundo orden asociada al movimiento de las partículas tiene en cuenta dos aproximaciones cuyo fundamento radica en discretizar el sistema considerando que dado un instante de tiempo inicial t_0, el sistema es invariable en $(t_0, t_0 + 1)$, por lo que el n-ésimo movimiento de una partícula queda determinado en el instante de tiempo $t = t_0 + n$. Considerando, sin pérdida de generalidad, $t_0 = 0$, y se establece que la variable temporal

t sobre la que evoluciona el sistema es discreta, concretamente, $t \in \mathbb{N}$. Estas aproximaciones son resultado de aplicar el Método de Diferencias Finitas con $h = 1$. De esta forma, resulta un sistema de ecuaciones en diferencias.

La primera de estas aproximaciones considera que la **velocidad** en el siguiente movimiento, es decir, en el instante de tiempo $t + h$, corresponde con la suma de la velocidad y la aceleración inmediatamente anteriores, en el instante de tiempo t.

$$\ddot{x_i}(t) \approx \frac{\dot{x_i}(t+h) - \dot{x_i}(t)}{h} \overset{h=1}{=} \dot{x_i}(t+1) - \dot{x_i}(t) \tag{3.3}$$

Análogamente, se considera que la **posición** en el instante de tiempo $t + h$, corresponde con la suma de la posición y la velocidad en el instante de tiempo t, resultado de aplicar el mismo método que en (3.3).

$$\dot{x_i}(t) \approx \frac{x_i(t+h) - x_i(t)}{h} \overset{h=1}{=} x_i(t+1) - x_i(t) \implies x_i(t+1) \approx x_i(t) + \dot{x_i}(t) \tag{3.4}$$

Adicionalmente, antes de desarrollar matemáticamente las ecuaciones dinámicas que determinan el movimiento de cada una de las partículas del enjambre, se detallan a continuación los tres factores clave que se han tenido en cuenta a la hora de plantear la primera ecuación del sistema 3.9:

1. Desplazamientos previos de las partículas: Como se ha adelantado anteriormente, la próxima posición de una partícula está influida por la velocidad anterior $\dot{x_i}(t)$. El parámetro $b \in (0, 1)$ se encarga de modular esta influencia:

$$-b\dot{x_i}(t) \tag{3.5}$$

2. Búsqueda del mejor valor a nivel **local** por una partícula: Cada partícula guarda la mejor posición que ha encontrado a lo largo de toda su trayectoria. A esta variable la denotamos $p_i(t)$. Se actualiza únicamente si el valor que ha encontrado en su actual movimiento, $f(x_i(t))$, es mejor que todos los anteriores, $f(p_i(t))$. El parámetro $c_l \in (0, 1)$ es el encargado de modular la fuerza atractora que va a ejercer $p_i(t)$ sobre el próximo movimiento de la partícula:

$$c_l(p_i(t) - x_i(t)) \text{ donde } p_i(t+1) = f^{-1}(\text{mín}\{f(x_i(t)), f(p_i(t))\}) \tag{3.6}$$

3. Búsqueda del mejor valor a nivel **global** por el resto de partículas: Del mismo modo, el enjambre en su totalidad guarda la mejor posición que ha encontrado entre todas las trayectorias de todas las partículas. A este parámetro se le denota como $g(t)$ y se actualiza únicamente cuando alguna de las partículas, por ejemplo $x_{i_0}, i_0 \in \{1, ..., N\}$, del enjambre han encontrado un valor $f(x_{i_0})$ mejor que todos

los anteriores $f(g(t))$. El parámetro $c_g \in (0,1)$ es el encargado de modular la fuerza atractora que va a ejercer $g(t)$ sobre el próximo movimiento de la partícula:

$$\begin{aligned} &c_g(g(t) - x_i(t)) \text{ siendo} \\ &g(t+1) = f^{-1}(\min_{i\in\{1,\dots,N\}}\{f(x_i(t)), g(t))\}) \overset{(3.6)}{=} f^{-1}(\min_{i\in\{1,\dots,N\}}\{f(p_i(t)), g(t))\}) \end{aligned} \tag{3.7}$$

El sistema mecánico asociado a cada partícula $x_i, i = 1, ..., N$ viene dado por este sistema de ecuaciones en diferencias. La primera ecuación viene dada por los factores nombrados anteriormente, mientras que la segunda ecuación es consecuencia de (3.4). Más adelante se concretarán las condiciones iniciales:

$$\begin{cases} m\ddot{x}_i(t) = -b\dot{x}_i(t) + c_l(p_i(t) - x(t)) + c_g(g(t) - x_i(t)) \\ x_i(t+1) = \dot{x}_i(t) + x_i(t) \end{cases} \tag{3.8}$$

En el sistema (3.8), la masa m es atraída por el mejor local y el mejor global, y es frenada por su velocidad, que actúa como fuerza de rozamiento a través de b, como se puede ver en la primera ecuación.

A continuación se supone, sin pérdida de generalidad, que todas las partículas de la nube tienen la misma importancia asignándoles la masa $m = 1$. Como resultado, el sistema dinámico asociado a cada partícula se traduce en:

$$\begin{cases} \ddot{x}_i(t) = -b\dot{x}_i(t) + c_l(p_i(t) - x_i(t)) + c_g(g(t) - x_i(t)) \\ x_i(t+1) = x_i(t) + \dot{x}_i(t) \end{cases} \tag{3.9}$$

Finalmente, resolviendo el sistema sustituyendo por la primera aproximación dada (3.3) en la primera ecuación del sistema:

$$\begin{aligned} &\ddot{x}_i(t) = \dot{x}_i(t+1) - \dot{x}_i(t) = -b\dot{x}_i(t) + c_l(p_i(t) - x_i(t)) + c_g(g(t) - x_i(t)) \implies \\ &\implies \dot{x}_i(t+1) = (1-b)\dot{x}_i(t) + c_l(p_i(t) - x_i(t)) + c_g(g(t) - x(t)) \end{aligned} \tag{3.10}$$

En consecuencia, cada iteración $t \in \mathbb{N}$ del enjambre queda definido de la siguiente forma, donde se ha considerando que la derivada posicional es la velocidad de la partícula y $\omega = 1 - b$:

$$\begin{cases} v(t+1) = \omega v(t) + c_l(\dot{x_l}(t) - x(t)) + c_g(\dot{x_g}(t) - x(t)) \\ x(t+1) = v(t) + x(t) \end{cases} \tag{3.11}$$

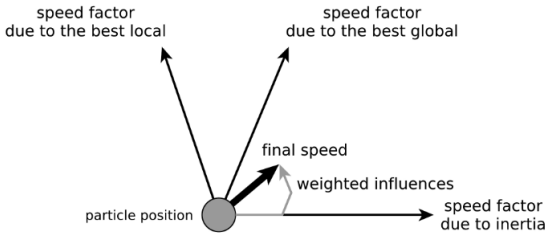


Figura 3.1: Representación de una partícula del enjambre [14]

En la Figura 3.1 se muestra una representación gráfica de cómo se compone el vector velocidad. Abajo, se muestra el pseudocódigo de un algoritmo PSO básico 1.

Algorithm 1 PSO básico; $g = PSO(N, w, c_l, c_g, bounds)$

$[p, g] \leftarrow$ **Best**(x)

repeat

  $[x, v] \leftarrow$ **Dynamic**(x, v, p, g)

  $[p, g] \leftarrow$ **Best**(x, p, g)

until Stop(g)

return g

Primeramente, el algoritmo inicializa las posiciones y velocidades de cada elemento de la nube de partículas con la función $Initialize()$, seguidamente determina los mejores puntos locales p y el mejor punto a nivel global g gracias a la función $Best()$, la cual utiliza la función de pérdida o ganancia f que es optimizada para devolver los puntos nombrados anteriormente. Después se entra en un bucle que termina con un criterio de parada establecido por la función $Stop()$, en el que se actualizan las posiciones de la nube de partículas, y se recalculan los valores g y p gracias a la función $Dynamic()$.

El algoritmo tiene un funcionamiento sencillo a nivel genérico, aunque en el siguiente apartado se verá en profundidad el desempeño de estas cuatro funciones que se implementan dentro del *PSO básico*, que son ciertamente más laboriosas.

3.2. Funciones y operadores

En la sección anterior se ha representado el pseudocódigo de un PSO básico en el que se han utilizado cuatro funciones que se van a detallar y estudiar en profundidad a continuación. Dichas funciones son **Initialize**(), **Best**(), **Dynamic**() y **Stop**().

3.2.1. Initialize()

La función **Initialize**() es la que se encarga de generar valores de posición y velocidad de la nube de partículas. La generación de dichos valores va a depender estrechamente del conocimiento previo del usuario sobre una posible estimación de la solución óptima, reduciendo así el espacio de búsqueda y, consecuentemente, abaratando el coste computacional del algoritmo.

- Si existe conocimiento previo del usuario: Se puede proceder siguiendo una distribución Gaussiana en torno a los valores que el usuario determine como posibles soluciones o puntos próximos a ellas. Otra manera de proceder es hallar estas potenciales soluciones con otros algoritmos (normalmente de un coste computacional menor) e incluir puntos aleatorios sobre el espacio de búsqueda siguiendo una distribución uniforme.

- Si no existe conocimiento previo del usuario: La forma más conveniente es establecer puntos de manera aleatoria que hace asignar equiprobabilidad a todos los puntos del espacio, conforme a una distribución uniforme sobre el rango de los valores posibles de dichos parámetros.

Aunque cualquier forma de inicialización es válida, se debe tener en cuenta que la población inicial tiene que ser lo suficientemente diversa para que el algoritmo dé margen a la nube de partículas para moverse libremente por el espacio de búsqueda, y no dé como resultado un solo valor, que podría resultar ser un falso óptimo. En cuanto a la inicialización de los valores de las velocidades, tenemos los dos mismos supuestos que en el de las posiciones iniciales:

- Si existe conocimiento previo del usuario: Se inicializan las velocidades en dirección a la que el usuario plantea como solución óptima.

- Si no existe conocimiento previo del usuario: Se inicializan todos los valores como nulos para que las velocidades en las distintas direcciones sean equiprobables.

A pesar de que la inicialización de las velocidades de las partículas es importante para su dinámica en el inicio, cabe destacar que a medida que se va iterando el algoritmo, dejan de serlo.

Algorithm 2 Initialize; $[x, v] =$ **Initialize**$(x, v, bounds)$

for $i = 1 : N$ **do**
  $v(i) \leftarrow 0$
  for $j = 1 : D$ **do**
    $x(i, j) \leftarrow U(lb(j), ub(j))$
  end for
end for
return $[x, v]$

Se muestra en 2 el pseudocódigo del algoritmo usado en el Capítulo 4, en el que se ha utilizado una inicialización de vectores de velocidad nulos y vectores de posición aleatorios por una distribución uniforme entre los límites dimensionales.

3.2.2. Dynamic()

Algorithm 4 Dinámica básica; $[x, v] =$ **DynamicBasic**(x, v, p, g)

for $i = 1 : N$ **do**
  $u_g \leftarrow rand(0, 1)$
  $u_l \leftarrow rand(0, 1)$
  $v(i) \leftarrow w \cdot v(i) + c_l \cdot u_l(p(i, k) - x(i, k)) + c_g \cdot u_g(g - x(i))$
  $x(i) \leftarrow x(i) + v(i)$
end for
return x, v

La función **Dynamic**() es la encargada de actualizar las posiciones y velocidades a través del sistema (3.11). El algoritmo *DynamicBasic()* consta del bucle sobre el número de partículas que conforman la nube N. Para cada partícula se genera un vector de peso $u = (u_l, u_g)$ que asigna un peso a las constantes cognitiva y cooperativa c_l y c_g. Este vector da variabilidad al modelo dado que el vector es generado aleatoriamente sobre una distribución uniforme $U(0, 1)$.

La diferencia entre las funciones *DynamicBasic()* y *DynamicAdvanced()*, cuyo pseudocódigo se detalla a continuación, es la inclusión de restricciones en las posiciones y velocidades de las partículas con las funciones $LimitPos(x, v)$ y $LimitSpeed(x, v)$. Dichas funciones se van a encargar de acotar las posiciones al espacio de búsqueda y de controlar la velocidad de las partículas para evitar convergencias tempranas que omitan puntos de interés del espacio de búsqueda.

Algorithm 5 Dinámica avanzada; $[x, v] =$ **DynamicAdvanced**(x, v, p, g)

for $i = 1 : N$ **do**

$u_l \leftarrow rand(0, 1)$

$u_g \leftarrow rand(0, 1)$

$v(i) \leftarrow w \cdot v(i) + c_l \cdot u_l \cdot (p(i) - x(i)) + c_g \cdot u_g \cdot (g - x(i))$

$x(i) \leftarrow x(i) + v(i)$

end for

$[x, v] \leftarrow LimitPos(x, v)$

$[v] \leftarrow LimitSpeed(x, v)$

return x, v

Para ver cómo funcionan estas dos últimas funciones, $LimitsPos()$ y $LimitSpeed()$, el usuario ha de establecer previamente bajo propio criterio el espacio donde quiere que el PSO realice la búsqueda. Estos límites corresponden, como se verá más adelante, con los límites de cada hiperparámetro del algoritmo de Aprendizaje Automático que se quiera optimizar heurísticamente. Dichos límites van a ser denotados como lb_j y ub_j haciendo referencia a *lower bound* y *upper bound*, respectivamente, donde $j \in 1, ..., D$ y que conforman la variable *bounds* de la función.
En $LimitPos()$, para corregir la posición de una partícula, se observa si la última actualización sobrepasa los límites en cada una de las D dimensiones. En caso positivo, hay varias formas de proceder;

1. $LimitsPos1()$ trata de establecer al valor x_{ij} el valor máximo ub_j en caso de que exceda el límite superior, o el valor mínimo lb_j en caso de que exceda el límite inferior. Además de la actualización de la posición, es conveniente modificar las velocidades de las dimensiones que hayan sido corregidas que, a su vez, se pueden resolver de dos formas.

   - Si lo que se desea es que en su próximo desplazamiento se tenga en cuenta sólamente la posición de los mejores, se procede anulando v_{ij}, $v_{ij} \leftarrow 0$.

   - Si se quiere penalizar que la partícula haya excedido la frontera, se negativiza la velocidad con la que la partícula ha salido del j-ésimo límite, $v_{ij} \leftarrow -v_{ij}$, como si su velocidad hubiese sido reflejada en la pared de dicho límite.

2. $LimitPos2()$ además de reflejar la velocidad como en el último caso, adicionalmente refleja la posición simulando un choque elástico con la frontera que había excedido $x_{ij} \leftarrow lb_j - v_{ik}$ (inferiormente) o $x_{ik} \leftarrow ub_j - v_{ik}$ (superiormente), penalizando así la dirección con la que la partícula se desplazaba.

Algorithm 6 Posición límite; $[x, v]$ = **LimitPos1(x,v)** $= (x, v)$

for $i = 1 : N$ **do**
  for $j = 1 : D$ **do**
    if $x(i, j) < lb(j)$ **then**
      $x(i, j) \leftarrow lb(j)$
      $v(i, j) \leftarrow$ **Options** $\{0, -v(i, j)\}$
    else if $x(i, j) > ub(j)$ **then**
      $x(i, j) \leftarrow ub(j)$
      $v(i, j) \leftarrow$ **Options** $\{0, -v(i, j)\}$
    end if
  end for
end for
return x, v

Se muestra en 6 y 7 los pseudocódigos de la limitación de posición por *LimitPos1* y *LimitPos2* respectivamente.

Algorithm 7 Posición choque elástico; $[x, v]$ = **LimitPos2(x,v)** $= (x, v)$

for $i = 1 : N$ **do**
  for $j = 1 : D$ **do**
    if $x(i, j) < lb(j)$ **then**
      $x(i, j) \leftarrow lb(j) - v(i, j)$
      $v(i, j) \leftarrow -v(i, j)$
    else if $x(i, k) > ub(j)$ **then**
      $x(i, j) \leftarrow ub(j) - v(i, j)$
      $v(i, j) \leftarrow -v(i, j)$
    end if
  end for
end for
return x, v

La función *LimitSpeed*() tiene como propósito acotar la velocidad que las partículas pueden alcanzar. Según [15], la velocidad es excesiva cuando las partículas se mueven demasiado rápido en el espacio de búsqueda, lo que puede llevar a problemas como una convergencia prematura o un movimiento descontrolado que haga que las partículas no puedan concentrarse en un área prometedora del espacio de búsqueda.

$$\begin{aligned} v_{ij}^{max} &= \kappa(ub_j - lb_j), \kappa \in (0, 1) \\ v_{ij}^{min} &= -\kappa(ub_j - lb_j), \kappa \in (0, 1) \end{aligned} \tag{3.12}$$

Según [16], v_{ij} es excesiva cuando alcanza valores que sobrepasan entre el 10 % y el 20 % del rango del espacio de búsqueda en una dimensión, mientras que otros [15] sugieren que velocidades menores al 50 % del rango podrían ser consideradas como válidas. Para corregir las velocidades, se tiene la función *LimitSpeed*() la cual aplica (3.12) como criterio de consideración de velocidad excesiva y procede de dos formas:

1. Frenando completamente la velocidad $v_{ij} \leftarrow 0$

2. Asignando la máxima velocidad permitida en caso de excederla superiormente o asignando la mínima velocidad permitida si la excede inferiormente.

Algorithm 8 Limitación velocidad; $[v] =$ **LimitSpeed(x,v)** $= (x, v)$

for $i = 1 : N$ **do**
  for $j = 1 : D$ **do**
    if $v(i,j) < -\kappa(ub(j) - lb(j))$ **then**
      $v(i,j) \leftarrow$ **Options**$\{0, -\kappa(ub(j) - lb(j))$
    else if $v(i,j) > \kappa(ub(j) - lb(j))$ **then**
      $v(i,j) \leftarrow$ **Options**$\{0, -\kappa(ub(j) - lb(j))$
    end if
  end for
end for
return x, v

3.2.3. Best()

La función *Best*() tiene como objetivo determinar las mejores posiciones de las partículas a nivel local p_i y la mejor a nivel global g. El lector habrá podido comprobar que la función de pérdida f puede no ser inyectiva. De hecho, normalmente no lo es. Como consecuencia, a la hora de aplicar (3.6) y (3.7), puede que exista un x_{i_0} de tal forma que $f(x_{i_0}) = f(g)$. En tal caso, existen varias formas de abordar el problema, como puede ser dividir el enjambre en sub-poblaciones que atiendan a los respectivos mejores globales con *Clustering* o *Niching*. También se utilizan técnicas basadas en agrupar partículas cercanas entre sí que atiendan al mismo mejor global como el *Crowding Distance*.

Aunque hay numerosas técnicas, aparte de las ya nombradas, en las que en determinados problemas, como pueden ser los multimodales monoobjetivo, pueden ser de especial interés, en este documento solo se va a trabajar con la actualización ya presentada en (3.6) y (3.7).

Algorithm 9 Selección de mejor local y global; $[p, g] = \textbf{Best(p,g)} = (p, g)$

for $i = 1 : N$ **do**
  if $f(x(i)) < f(p(i))$ **then**
    $p(i) \leftarrow x(i)$
    if $f(p(i)) < f(g)$ **then**
      $g \leftarrow p(i)$
    end if
  end if
end for
return p, g

3.2.4. Stop()

Por último, falta detallar un criterio de parada. ¿Hasta cuándo va a ser iterado el algoritmo? Entre los criterios más comunes se encuentran:

1. Una vez el algoritmo alcanza un número indicado de iteraciones $t = t_0$.
2. Establecer un umbral de error ϵ a partir del cual se considera válida la solución $|f(g)| < \epsilon$.
3. Dado un $\delta > 0$ que $f(g(t-1)) - f(g(t)) < \delta$ para un cierto número de iteraciones $n_o \in \mathbb{N}$, indicando así un estancamiento en el enjambre.

Existen múltiples criterios de parada a cada cual más complejo, pero no se va a hacer incidencia en ellos dado que carecen de importancia en este trabajo.

3.3. Mutaciones

Como el movimiento de las partículas en PSO se basa en su velocidad y en los atractores generados por las mejores posiciones locales y globales, en situaciones en las que las partículas no encuentran nuevas posiciones superiores durante muchas iteraciones, puede darse el caso de que todas ellas converjan a una mejor posición local. Esto puede causar que el algoritmo no continúe generando mejoras en generaciones posteriores, dando lugar a una falta de exploración del espacio de búsqueda que tendrá, como probable consecuencia, el retorno de un falso óptimo al final del algoritmo.

Para este problema, algunas versiones del PSO implementan una etapa de mutación: algunos de los valores de decisión de determinadas partículas son alterados aleatoriamente. Las funciones de mutación pueden ser análogas a las empleadas en algoritmos genéticos

(GA) con cadenas de variables reales.

Además, algunos algoritmos han dispuesto diversas mutaciones para la combinación. Por ejemplo, en OMOPSO, una variante multiobjetivo de PSO, un tercio de las partículas de la población se somete a mutación uniforme, otro tercio a mutación no uniforme, y el tercio restante no se ve afectado por ninguna función de mutación.

En el proceso de mutación aplicado en la parte experimental del Capítulo 4, se ha creado una función denominada *Diversity*, que calcula la desviación estándar de las posiciones de cada dimensión y a continuación hace una media de todas ellas. Esta función no es más que una forma de cuantificar la dispersión de las partículas en el espacio de búsqueda. Para que una iteración entre en el proceso de mutación se han establecido las siguientes restricciones:

1. La función *Diversity* tiene que estar por debajo de 50. Puede parecer elevado, pero próximamente se verá que hay variables del modelo que se entrena que pueden tomar valores ciertamente grandes. Esto último se suma al alto riesgo de estancamiento en un mínimo local.
2. Como las partículas se pueden seguir moviendo alrededor de un mínimo local aún estando estancadas, se ha considerado que un estancamiento es todo conjunto de 5 iteraciones cuyo rango de la función *Diversity* es menor a 5. De esta forma se da cierta laxitud a lo que se considera estancamiento.
3. Las dos últimas imposicones se tienen que dar a partir de la quinta iteración del enjambre. Al principio se quiere que el enjambre busque sin mutaciones.

Por último se ha establecido un ratio de mutación del 30 %, es decir, si la iteración entra en el proceso mutativo porque cumple todas las condiciones, un 30 % de las partículas del enjambre mutan. El cómo mutan es muy sencillo: se reposicionan las partículas en el espacio de búsqueda aleatoriamente como en la inicialización del enjambre. Finalmente, cuando ha terminado el proceso mutativo, se reestablece de nuevo el contador del estancamiento a 0. De esta forma, permitimos más mutaciones a lo largo de la optimización.

Capítulo 4

Optimización del Aprendizaje Automático mediante enjambre de partículas

4.1. Introducción

El éxito de los algoritmos de Aprendizaje Automático depende en gran medida de la configuración adecuada de sus parámetros, conocidos como hiperparámetros. Esto plantea la siguiente pregunta: ¿es posible optimizar los algoritmos de Aprendizaje Automático utilizando técnicas inspiradas en el comportamiento de las nubes de partículas? La respuesta a esta cuestión da paso a un amplio campo de investigación, ya que los métodos bioinspirados, como el Particle Swarm Optimization (PSO), han demostrado ser de gran utilidad para explorar espacios de búsqueda de alta dimensionalidad, reduciendo significativamente el coste computacional del entrenamiento de los modelos al ofrecer soluciones cercanas al mejor ajuste hiperparamétrico posible.

El enfoque biológico-evolutivo permite abordar problemas complejos de optimización de forma rentable, tanto en términos de tiempo de cómputo como de recursos como puede ser la memoria. Sin embargo, surgen preguntas importantes como: ¿cuál es la rentabilidad real de emplear PSO en comparación con otros métodos de optimización? ¿Qué nivel de precisión se puede alcanzar? En este capítulo se abordarán estas cuestiones.

4.2. Hiperparámetros

En el Capítulo 2 (2), se han presentado algunos algoritmos más en profundidad que otros. Los modelos *Logistic Regression*, *k-NN*, *Random Forest Classifier* y *SVM* van a

ser utilizados en el problema a resolver, que se adelanta que va a ser de clasificación, de manera preliminar, antes de pasar al estudio del PSO. A continuación se detallan los hiperparámetros que se han estudiado en este capítulo de cada uno de los modelos. La fuente principal de esta información ha sido [6]:

4.2.1. Logistic Regression:

Según [17] los 3 hiperparámetros más importantes de este algoritmo son:

1. **Solver:** Es el algoritmo usado para el problema de optimización. Las opciones son *lbfgs*, *sag*, *saga*, *bilinear* y *newton-cg*.

2. **Penalty:** Se trata de la regularización que recibe el modelo para intentar reducir el error de generalización. Los más conocidos y usados son las regularizaciones de *Lasso* (L1) 4.1 y *Ridge* (L2) 4.2. El primera penaliza los coeficientes del modelo de manera lineal, mientras que el segundo lo hace de manera cuadrática, penalizando más aún los coeficientes por encima de 1 en valor absoluto y favoreciendo aquellos por debajo de 1 también en valor absoluto.

$$\min_{\beta} \sum_{i=1}^{n} (y_i - X_i\beta)^2 + \lambda \sum_{j=1}^{p} |\beta_j| \tag{4.1}$$

$$\min_{\beta} \sum_{i=1}^{n} (y_i - X_i\beta)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \tag{4.2}$$

   Como se puede observar, ambos métodos dependen de un parámetro λ, el encargado de controlar la cantidad de regularización que se aplica a la optimización. Se tiene que tener cierto cuidado puesto que si se elige uno demasiado alto, el modelo no será capaz de aprender del todo bien causando *underfitting*.

3. **C:** Es el inverso de la fuerza de regularización, es decir, $C = \frac{1}{\lambda}$. Valores pequeños implican alta regularización y valores grandes poca regularización.

4.2.2. k-NN

Como se ha visto anteriormente, k-NN asigna la clase mayoritaria de los k vecinos más cercanos. Los hiperparámetros a estudiar son los siguientes:

- **n_estimators:** Como es natural, el hiperparámetro más importante de este modelo es establecer el número k apropiado de vecinos más cercanos. Un k demasiado pequeño puede ajustar demasiado a nivel local desatendiendo la posición global de la instancia a clasificar, *overfitting,* y con un k demasiado grande, puede ocurrir justo el efecto contrario.

- **Weights:** A través de la clase '*distance*' se pueden asginar pesos a los vecinos en función de la distancia a la que estén de la instancia a clasificar. De esta forma cuanto más lejos está el vecino, menos importancia se le da y viceversa. También se puede usar '*uniform*' para mantener los pesos iguales en todos los vecinos. De hecho, es el valor por defecto.
- **Indexing Algorithms:** El algoritmo necesita computar las distancias del resto de instancias al punto a clasificar. Dado que computarlas todas puede resultar costoso, especialmente si el dataset es grande, se pueden hacer uso de algoritmos como *kd-tree* o *ball tree* para evitar hacer tantas operaciones.

4.2.3. Support Vector Machines (SVM)

- **Kernel:** Como se ha adelantado en el Capítulo 2 2, uno de los grandes problemas a los que se puede enfrentar el SVM es a la imposibilidad de separar mediante un hiperplano de manera lineal dos clases diferentes. Sin embargo, existen funciones llamadas *kernels* ϕ que mapean el espacio facilitando la separación de dichas clases. Uno de los más conocidos para abordar la clasificación por coronas o circunferencias es el *RBF (Radial Basis Function)*. Por supuesto, existen *kernels* polinomiales, de *Sigmoid* o Gaussianos entre muchos otros.

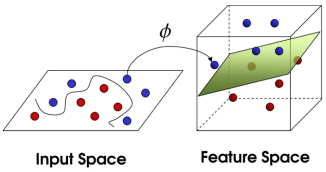


Figura 4.1: Ejemplo SVM [18]

- **C:** Es el parámetro encargado de la regularización de la que se ha hablado en el Capítulo 2 respecto del '*margin width*'. Frecuentemente hay que dar flexibilidad al modelo para omitir ciertas instancias que se escapan de la región en la que se deberían encontrar para que el modelo pueda ajustar correctamente. Al igual que en *Logistic Regression*, un C demasiado pequeño puede conducir a que los márgenes sean excesivamente holgados y por lo tanto haya problemas de *underfitting*.
- **Grado:** Este hiperparámetro solo aplica en kernels polinomiales. Como se puede intuir, es el grado máximo que el kernel puede adoptar para mapear el espacio. Cuanto mayor el grado más riesgo de *overfitting* se tiene, y cuanto menor, más riesgo de *underfitting*.

4.2.4. Random Forest

- **n_estimators:** Este es el total de árboles de decisión que se entrenan en el proceso de *bagging* del modelo. A más árboles, más informativo es, pero pagando un coste computacional alto.

- **max_depth:** Establece un límite en el que cual los árboles de decisión dejan de ramificar, previniendo así el *overfitting*, aunque un valor demasiado bajo puede causar el efecto contrario.

- **criterion:** Es la función que mide la calidad de una subdivisión en un nodo marcada por un atributo. Se ven a continuación los diferentes tipos:

  1. Entropy: O más conocida como la entropía de Shannon, es una medida fundamental en la teoría de información. En un árbol de decisión mide la aleatoriedad de una subdivisión, representada matemáticamente en 4.3.

  $$H = -\sum_i p_{i,k} \log_2 p_{i,k}) \tag{4.3}$$

  donde $p_{i,k}$ representa el ratio de la clase k en la subdivisión del i-ésimo nodo. Si la entropía de la subdivisión es igual a 0, significa que solo existe una clase en el conjunto de entrada con ese atributo, conduciendo a una subdivisón perfecta. En cambio, si la entropía es 1, significa que existe el mismo número de instancias de cada clase haciendo la subdivisión con ese atributo, implicando incertidumbre y pérdida de información.

  2. Gini Impurity: Al igual que la entropía, calcula el grado de impureza de una subdivisión. La ecuación matemática es la siguiente:

  $$G = 1 - \sum_i p_{i,k}^2 \tag{4.4}$$

  A diferencia de la entropía que varía entre los valores 0 y 1, la Impureza de Gini tiene un rango de $[0, 0{,}5]$. De la misma forma, una subdivisión pura o perfecta se da cuando el valor es igual a 0.

  3. Log_loss: Es una medida similar a la entropía pero con una función de perdida logarítimica:

  $$\text{Log Loss} = -\frac{1}{N}\sum_{i=1}^{N} [y_i \log p(y_i) + (1 - y_i) \log(1 - p(y_i))] \tag{4.5}$$

  donde $p(y_i) = p_{i,k}$ e y_i representa la clase verdadera de la muestra i con un 1 si pertenece a la clase positiva y con un 0 si pertenece a la clase negativa.

- **min_samples_split:** Indica el número mínimo de muestras necesarias para que un nodo pueda subdividirse. Si no las alcanza se convierte en un nodo tipo hoja resultando en el final de una rama.

- **min_samples_leaf:** Este parámetro establece el mínimo de muestras que tiene que haber en un nodo tipo hoja. Suele ayudar a prevenir el *overfitting*, podando el árbol antes de que ajuste demasiado.

- **max_features:** Este parámetro contiene la función que limita el número de características a considerar en una subdivisión. Por ejemplo, si toma el valor '*sqrt*', el número máximo de características a considerar en un nodo es $max_features = sqrt(n_features)$

4.3. Ajuste de hiperparámetros

La modificación de hiperparámetros es un procedimiento esencial en el Aprendizaje Automático, ya que puede determinar la distinción entre un modelo de buen desempeño y uno que no logra generalizar adecuadamente. Existen diferentes técnicas de búsqueda de ellos. A continuación se detallan las más populares y la que abarca este trabajo, así como algunas ventajas y desventajas que tienen.

Grid Search es un procedimiento que examina cada una de las posibles combinaciones de los valores de hiperparámetros dentro del espacio de búsqueda. Este procedimiento se lleva a cabo mediante $Cross - Validation$.

- **Ventajas:** Se asegura hallar la combinación más adecuada dentro del conjunto establecido. Además es sencillo ponerlo en marcha y es fácilmente paralelizable.
- **Desventajas:** Tiene un costo computacional elevado, particularmente si existen numerosos hiperparámetros.

El **Random Search** elige mezclas aleatorias de hiperparámetros en vez de analizar todas las posibles.

- **Ventajas:** Es computacionalmente menos costoso que la búsqueda en cuadrícula (Grid Search). Se puede utilizar para tener una idea intuitiva de las regiones del espacio hiperparamétrico más prometedoras.
- **Desventajas:** No asegura hallar la combinación más adecuada dado que se dejan numerosas combinaciones sin analizar. Si el espacio de búsqueda es considerablemente extenso, podría requerir numerosas iteraciones para hallar una configuración razonablemente buena.

La **Optimización Bayesiana** representa el desempeño del modelo como una función probabilística y emplea técnicas como los Procesos Gaussianos para prever qué combinaciones de hiperparámetros pueden proporcionar resultados superiores antes de examinarlas de manera directa.

- **Ventajas:** Suele ser más eficaz que Grid Search dado que disminuye la cantidad de evaluaciones requeridas, y que Random Search, dado que aunque este algoritmo no evalúe todas y cada una de las combinaciones, utiliza modelos probabilísticos que guían la búsqueda a las regiones con más densidad de probabilidad utilizando conocimientos de iteraciones anteriores.
- **Desventajas:** Aunque teóricamente parezca ser una solución que se acerca a la perfección en cuanto a balance entre coste computacional y un buen ajuste, es bastante más complicado de poner en práctica que los dos anteriores.

El PSO, como ya se ha explicado, es un método inspirado en procesos naturales que generan nuevas soluciones en cada iteración a medida que exploran el espacio.

- **Ventajas:** Es capaz de encontrar buenas soluciones sin necesidad de evaluar todas las combinaciones posibles y funciona especialmente bien en espacios de búsqueda complejos y multidimensionales.
- **Desventajas:** Es susceptible de quedar atrapado en máximos o mínimos locales, impidiendo la correcta exploración del espacio de búsqueda en su totalidad. Además requiere del ajuste de parámetros como la velocidad de aprendizaje de las partículas, la inercia o el número de partículas a utilizar.

4.4. Caso de estudio

En esta sección se presenta el estudio de un problema concreto, en el que se ha estudiado el comportamiento del PSO y, posteriormente, se ha hecho un análisis estadístico de los parámetros que conforman el algoritmo.

El enlace que da acceso al código en Python, el dataset original, los ficheros csv resultantes, así como las imágenes utilizadas a lo largo del trabajo se encuentra aquí.

El problema elegido para este estudio es la clasificación de los 10, 100 y 250 mejores tenistas a lo largo de la temporada completa del circuito de tenis ATP (Asociación de Tenis Profesional).

4.5. Preprocesamiento del dataset

Se ha utilizado [19] como fuente de información, la cual contiene reportes de todos los partidos de cada jugador en cada temporada, en concreto 42 variables que han sido reducidas a 8. Se han escogido las temporadas desde el 2010 hasta el 2023. Para hacer tangible el problema y poder ser analizado, se ha transformado este conjunto de datos en un resumen de estadísticas por jugador y temporada, a partir del cual se extraen diversas métricas que describen el rendimiento. A continuación se detallan las fórmulas utilizadas 4.5.1 y se presenta una tabla resumen con la descripción de cada variable 4.1.

4.5.1. Cálculo de Métricas Agregadas

Para cada jugador p en una temporada s, se han calculado las siguientes métricas agregadas:

$$\text{matches_played}_{p,s} = \sum_i \mathbb{I}(\text{player}_i = p, \text{season}_i = s) \tag{4.6}$$

$$\text{wins}_{p,s} = \sum_i \mathbb{I}(\text{player}_i = p, \text{season}_i = s, \text{winner}_i = p) \tag{4.7}$$

$$\text{losses}_{p,s} = \text{matches_played}_{p,s} - \text{wins}_{p,s} \tag{4.8}$$

$$\text{win_percentage}_{p,s} = \frac{\text{wins}_{p,s}}{\text{matches_played}_{p,s}} \times 100 \tag{4.9}$$

$$\text{straight_sets_wins}_{p,s} = \sum_i \mathbb{I}(\text{winner}_i = p, \text{sets_lost}_i = 0) \tag{4.10}$$

$$\text{avg_opponent_rank_loss}_{p,s} = \frac{\sum_i \text{opponent_rank}_i \cdot \mathbb{I}(\text{winner}_i \neq p)}{\text{losses}_{p,s}} \tag{4.11}$$

$$\text{tournaments_played}_{p,s} = |\{\text{tournament}_i \mid \text{player}_i = p, \text{season}_i = s\}| \tag{4.12}$$

$$\text{matches_won_high_1st_serves}_{p,s} = \sum_i \mathbb{I}(\text{winner}_i = p, \text{1st_serve_percentage}_i \geq 70) \tag{4.13}$$

4.5.2. Clasificación por Grupo de Ranking

Para categorizar a los jugadores según su posición en el ranking al final de cada temporada, se ha utilizado la siguiente asignación:

$$\text{ranking_group}_{p,s} = \begin{cases} \text{Top 10}, & \text{si ranking}_{p,s} \leq 10 \\ \text{Top 100}, & \text{si } 10 < \text{ranking}_{p,s} \leq 100 \\ \text{Top 250}, & \text{si ranking}_{p,s} > 100 \end{cases} \tag{4.14}$$

4.5.3. Variables del Dataset Final

Las métricas calculadas, junto con la variable objetivo *ranking_group* que agrupa a los jugadores según su ranking final de temporada ('Top 10', 'Top 100', 'Top 250'), se resumen en la siguiente tabla:

Variable	Descripción
matches_played	Total de partidos jugados en la temporada.
wins	Número de partidos ganados en la temporada.
losses	Número de partidos perdidos en la temporada.
win_percentage	Porcentaje de victorias respecto al total de partidos jugados.
tournaments_played	Número de torneos en los que participó el jugador durante la temporada.
straight_sets_wins	Número de partidos ganados en sets corridos (sin perder set).
avg_opponent_rank_loss	Ranking promedio de los oponentes contra los que se ha perdido.
matches_won_high_1st_serves	Partidos ganados con un alto porcentaje de primeros servicios ($\geq 70\,\%$).
ranking_group	Variable objetivo: Agrupación según el ranking final de temporada.

Cuadro 4.1: Descripción del dataset de rendimiento en tenis (2010-2023)

4.5.4. Resumen del Procesamiento

Observando la distribución 4.2 de la variable respuesta respecto de las variables independientes, se ha podido observar que el preprocesamiento y la selección de la variable respuesta son adecuadas al estudio, dado que los jugadores que están en cada de uno de los subgrupos del ranking presentan diferencias visualmente significativas en cada una de las distribuciones.

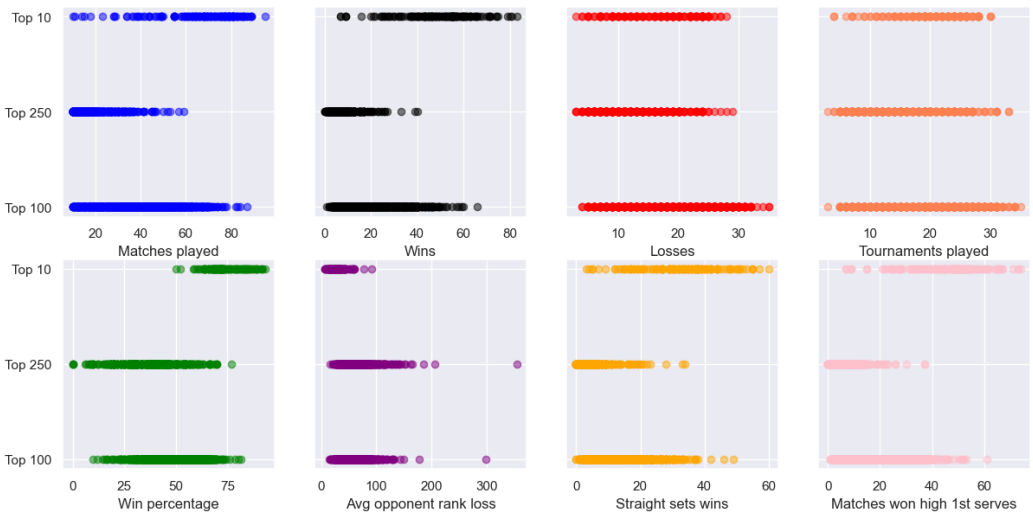


Figura 4.2: Distribución de variables por *ranking_group*

A continuación se muestra la matriz de correlaciones 4.3 asociada:

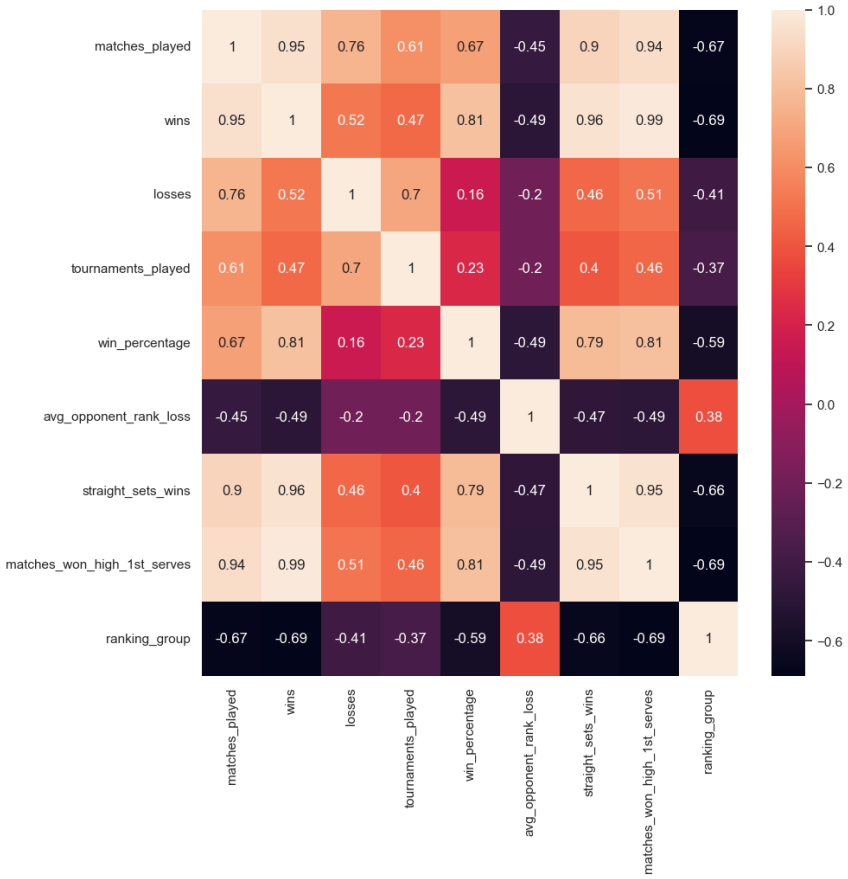


Figura 4.3: Matriz de correlaciones

Se puede observar que existen correlaciones muy altas entre las variables independientes. Esto podría llegar a motivar la eliminación de algunas de ellas, en particular *straight_sets_wins* y *matches_won_high_1st_serves*, pero se explica a continuación por qué son necesarias.

Al ser un problema de clasificación, uno de los factores más importantes es que los datos diferencien bien los rangos o límites en los que se mueven cada una de las clases de la variable respuesta *ranking_group* respecto de las variables independientes. Por ello, los fallos en la clasificación, una vez se entrene el modelo y se haga el test con datos reales para observar cómo se comporta, no se van a encontrar en los valores más obvios como pueden ser Novak Djokovic o Rafael Nadal que están en las primeras posiciones del ránking y son fácilmente diferenciables. En contraposición, las predicciones erróneas se van a encontrar en las fronteras de los grupos que se han detallado. Por ejemplo, entre un jugador de posición 9 en el ránking, uno de posición 13 o uno en la posición 102.

Es por esto que *straight_sets_wins* y *matches_won_high_first_serves* son cruciales. Los jugadores del 'Top 10' se diferencian por ganar partidos de forma holgada contra jugadores lejos de las primeras posiciones del ránking, es decir, ganar los partidos en sets corridos o con un muy alto porcentaje de primeros servicios. Del mismo modo, los jugadores 'Top 100' no se diferencian generalmente de los 'Top 250' por el número de partidos que juegan, sino por tener alguna victoria en sets corridos y jugar algunos partidos con porcentajes

de primeros servicios altos. Por la misma razón, variables con baja correlación que aparentemente pueden ser identificadas como no informativas como *avg_opponent_rank_loss*, podrían tener un efecto en el modelo sobre la decisión de clasificación de jugadores cercanos a las fronteras, dado que los jugadores más altos del ranking suelen perder contra jugadores que están en las máximas posiciones y viceversa.

Como el lector habrá podido intuir, las clases están desbalanceadas, dado que hay muchos menos jugadores 'Top 10' que 'Top 100'. Aparentemente debería haber más jugadores 'Top 250' que 'Top 100', pero esto no sucede en el dataset preprocesado, debido a que hay una gran cantidad de jugadores de bajo ránking que normalmente juegan muy pocos partidos y de muy bajo nivel, dando lugar a los indeseados outliers. Dichos jugadores han sido eliminados para que el modelo generalice mejor. En consecuencia del desbalance de clases, se aplica *SMOTE* 4.4, una técnica de sobremuestreo que genera instancias artificiales para un mejor entrenamiento del modelo y evitar un posible sesgo producido por la falta de instancias 'Top 10' o 'Top 250'. Esta técnica solo se aplica en los datos de entrenamiento ya que generar instancias artificiales en un conjunto de prueba es incorrecto en Aprendizaje Automático.

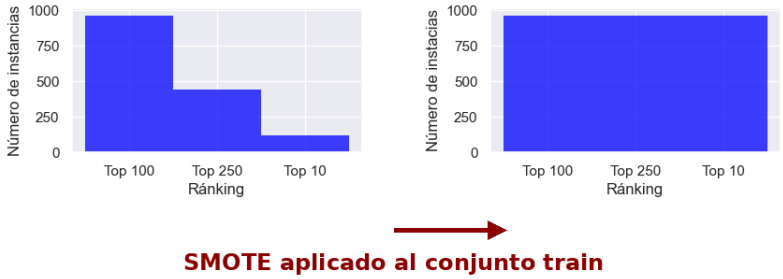


Figura 4.4: Aplicación de SMOTE al conjunto de entrenamiento

4.5.5. Resultados

Para la realización de este trabajo se ha contado con un ordenador portátil con las siguientes características técnicas:

Marca y modelo de la CPU: Intel Core i5.
Memoria RAM: 8 GB DDR4.
Almacenamiento: SSD 512 GB.
S.O. y distribución: Windows 11.

Para de realizar una evaluación preliminar de cómo se comportan los modelos de Aprendizaje Automático supervisado de clasificación sobre este dataset, se ha llevado a cabo una búsqueda por *3-5-Nested Cross-Validation* en la que se han estudiado algunos hiperparámetros de los modelos *SVM*, *Logistic Regression*, *k*-NN y *Random Forest Classifier*. Los valores de los hiperparámetros a estudiar han sido los siguientes:

Modelo	**Hiperparámetros**
4*Random Forest	Número de árboles: {10, 100, 500} Criterio: {gini, entropy, log_loss} Profundidad máxima: {5, 10, 20} Características: {sqrt, log2, None}
3*SVM	C: {0.1, 1, 10} Kernel: {linear, rbf, sigmoid} Grado (para kernel polinómico): {2, 3, 4}
3*Regresión Logística	Solver: {newton-cg, lbfgs, liblinear, sag, saga} Penalización: {l1, l2} C: {0.01, 0.1, 1}
3*k-NN	Vecinos: {3, 5, 10} Pesos: {uniform, distance} Algoritmo: {auto, ball_tree, kd_tree}

Cuadro 4.2: Hiperparámetros estudiados para cada modelo

En el cuadro inferior, se muestran los resutlados $F1 - score$ promediados sobre todos los modelos del *Nested Cross-Validation*, dando lugar a la métrica *F1-Macro*. Se ha utilizado el *F1-Score*, ya que es una métrica intermedia entre el *Accuracy* y el *Recall*, indicada para problemas de clase desbalanceada.

Modelo	**F1-Macro promedio (Nested CV)**
Random Forest	0.8825 ± 0.0164
SVM	0.8344 ± 0.0207
Logistic Regression	0.8227 ± 0.0170
k-NN	0.8746 ± 0.0165

Cuadro 4.3: Comparación de modelos basada en F1-Macro con Nested Cross-Validation

Puesto que se necesita que el espacio de búsqueda sea lo suficientemente grande como para conseguir que el PSO sea un algoritmo de búsqueda que pueda aportar resultados ventajosos, se ha elegido como modelo a estudiar el *Random Forest Classifier*. Es conveniente dado el extenso repertorio de hiperparámetros que se pueden evaluar. Además, es ideal para un problema de clasificación como el que se ha presentado y ha dado el mejor resultado en la prueba realizada de selección de modelo.

El trabajo nace con la idea de demostrar que el ajuste hiperparamétrico de algoritmos de Machine Learning puede ser llevado a cabo por modelos metaheurísticos que se acerquen a ese mejor ajuste. Las pruebas llevadas a cabo han demostrado una alta fiabilidad del PSO, llegando incluso a **mejorar** los resultados.

Lo primero que se ha hecho ha sido el ajuste Grid Search de los hiperparámetros detallados

a continuación, así como todos los valores candidatos de dichos hiperparámetros. El ajuste se ha realizado con *5-Cross-Validation*, previa separación 80 % entrenamiento y 20 % test.

Hiperparámetros	**Valores**
n_estimators	10, 50, 100, 150, 200, 500
criterion	gini, entropy, log_loss
max_depth	5, 10, 20
min_samples_split	2, 5, 10
min_samples_leaf	1, 2, 3, 4, 5
min_weight_fraction_leaf	0.0, 0.1, 0.2, 0.3, 0.4, 0.5
max_features	sqrt, log2, None

Cuadro 4.4: Hiperparámetros utilizados en el modelo Random Forest

Como era de esperar, el tiempo de ejecución de Grid Search ha sido bastante elevado, a consecuencia de las 14580 combinaciones con 5 carpetas de validación cruzada por modelo, con un total de 14580 x 5 = 72900 ajustes.

Modelo	Random Forest Classifier
Mejores Hiperparámetros	
criterion	entropy
max_depth	20
max_features	None
min_samples_leaf	1
min_samples_split	5
min_weight_fraction_leaf	0.0
n_estimators	100
Mejor Score	0.8831512276115955
Tiempo de Ejecución	533m 15.1s

Para poder comparar la métrica de valoración *F1-Macro* del PSO con las del Grid Search, se hace *5-Cross-Validation* en cada uno de los modelos. Al intentar aplicar PSO aL *Random Forest Classifer* surgen dos problemas:

1. El primero y más importante viene ligado a las bases de PSO. Se recuerda, que el PSO tiene como fundamento las ecuaciones en diferencias detalladas en 3.11, y éstas no atienden a problemas espacios de búsqueda discretos si no continuos. Por este motivo, surge la necesidad de redondear los valores que el PSO haya ido encontrando por el espacio de búsqueda delimitado por los límites inferiores y superiores de cada uno de los hiperparámetros objeto de estudio como se muestra en 4.5.

```
def funObj(x):
    # 2) Definimos el modelo con esos hiperparámetros
    model = RandomForestClassifier(n_estimators = round(x[0]), max_depth= round(x[1]),
            criterion = criterion_map_rf[round(x[2])], min_samples_split = round(x[3]),
            min_samples_leaf = round(x[4]), min_weight_fraction_leaf = round(x[5], 2),
            max_features = max_features_map_rf[round(x[6])])
```

Figura 4.5: Implementación en Python de los redondeos

2. Además, se tienen dos hiperparámetros en concreto que son categóricos, *criterion* y *max_features*. Esto se ha resuelto mapeando las clases a números de la siguiente forma:

$$\begin{aligned} \text{criterion_map_rf}(\texttt{gini}) &= 0, \\ \text{criterion_map_rf}(\texttt{entropy}) &= 1, \\ \text{criterion_map_rf}(\texttt{log_loss}) &= 2. \end{aligned}$$

$$\begin{aligned} \text{max_features_map_rf}(\texttt{sqrt}) &= 0, \\ \text{max_features_map_rf}(\texttt{log2}) &= 1, \\ \text{max_features_map_rf}(\texttt{None}) &= 2. \end{aligned}$$

Para hacer un estudio apropiado del PSO se ha ido modificando el comportamiento y la población del enjambre. Se han hecho evaluaciones del PSO con número de partículas impares del 3 al 15 inclusive, un total de 7 evaluaciones por cada configuración del algoritmo. Todas ellas han sido hechas con dinámica avanzada dado que no se quería exceder ni superior ni inferiormente los límites de los rangos de cada hiperparámetro. En la incialización del enjambre se ha elegido la opción de asignación aleatoria uniforme de las posiciones entre su límites inferior y superior y con velocidades nulas, como se ha detallado en el pseudocódigo 2. Se ha escogido la función *LimitPos1* 6 con la opción de penalización de la velocidad, negativizándola. Además, en cuatro configuraciones, se ha aplicado la función de limitación de velocidad con anulación, correspondiente con la primera opción del pseudocódigo 8. El criterio de parada ha sido llegar a 30 iteraciones. Se detallan, a continuación, las combinaciones de los modelos evaluados:

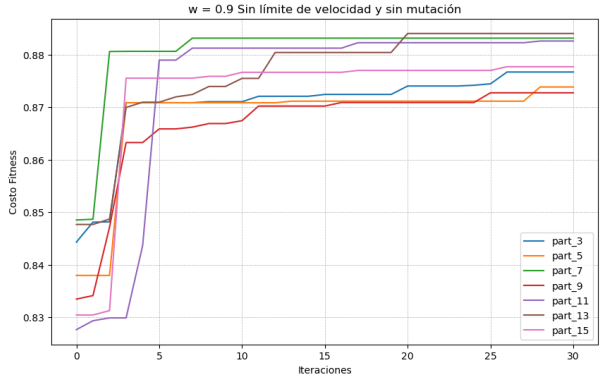
1. PSO con inercia fija, sin mutaciones y sin límite de velocidad 4.6a.
2. PSO con inercia fija, sin mutaciones y con límite de velocidad 4.6c.
3. PSO con incercia fija, con mutaciones y sin límite de velocidad 4.6e.
4. PSO con incercia fija, con mutaciones y con límite de velocidad 4.6g.
5. PSO con inercia evolutiva, sin mutaciones y sin límite de velocidad 4.7a.
6. PSO con inercia evolutiva, sin mutaciones y con límite de velocidad 4.7c.
7. PSO con inercia evolutiva, con mutaciones y sin límite de velocidad 4.7e.
8. PSO con inercia evolutiva, con mutaciones y con límite de velocidad 4.7g.

Las inercias fijas han sido establecidas en w=0.9 dado que se considera el valor estándar que mejor funciona, como se ha estudiado en [20] o [21]. Los coeficientes sociales y cognitivos que dan peso a los vectores velocidad que apuntan al mejor local y al mejor global han sido fijados en sus valores estándar c_1=2 y c_2=2. Esto es consecuencia de que la gran mayoría de la literatura y los estudios, [22] entre otros, establecen que un rango óptimo para estas constantes está comprendido entre 1.5 y 2.5, dejando claro que establecer el

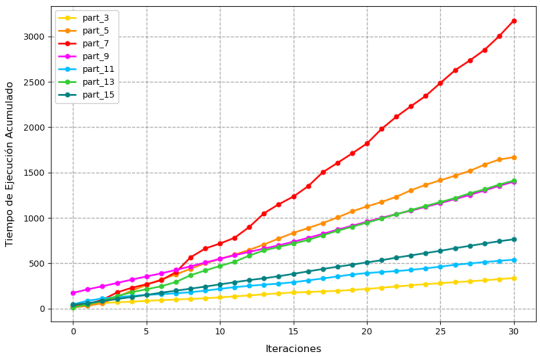
valor 2 a ambas constantes es un muy buen punto de partida que mantiene el equilibrio entre la atracción social y la atracción particular.

Es cierto que hay estudios en los que estas constantes son modificadas evaluando el comportamiento del modelo. Como se ha estudiado en [23], la modificación de estas constantes no tiene una clara aportación significativa hacia mejorar los resultados, es más, retocarlas podría llevar a un peor comportamiento del enjambre. A diferencia de las constantes anteriores, sí que se ha comprobado en [24], que la modificación de la inercia, el número de partículas que componen el enjambre y el establecimiento o no de límites en las velocidades de las partículas, sí supone una diferencia sustancial en la convergencia, comportamiento, tiempos de ejecución y mejora del modelo.

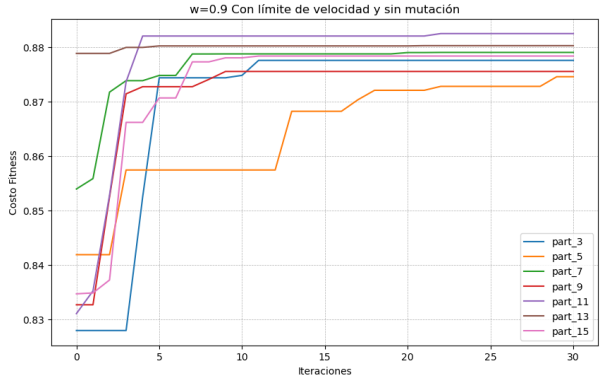
El lector se habrá podido preguntar el por qué de incluir un w dinámico en el trabajo si la gran mayoría de estudios sugieren uno fijo entre 0.4 y 0.9. El estudio de un w dinámico viene motivado por el problema mencionado anteriormente, en el que se explica que el PSO es un algoritmo que está hecho para trabajar en espacios continuos. Como la mayoría de las variables son discretas, es fácil que haya partículas que quieran salir de un determinado punto pero el redondeo que se hace, siga aproximando al punto en el que se encuentra, estancando la partícula e impidiendo la correcta exploración. Esto sucede especialmente en las últimas iteraciones donde los vectores velocidad generados ya no tienen un módulo lo suficientemente amplio para vencer este redondeo debido a la convergencia del enjambre. Se puede ver especialmente bien en la Figura (f) 4.7c, donde la función objetivo sigue mejorando en las últimas iteraciones, a diferencia de los demás modelos en los que no se han incluido mutaciones.



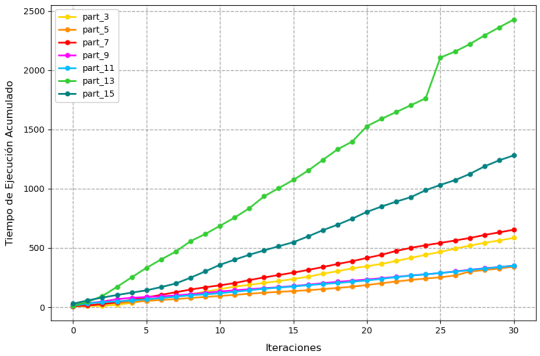
(a) Fitness - Modelo 1



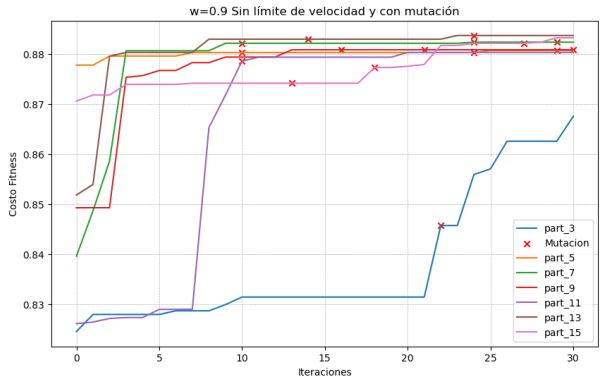
(b) Tiempo de ejecución - Modelo 1



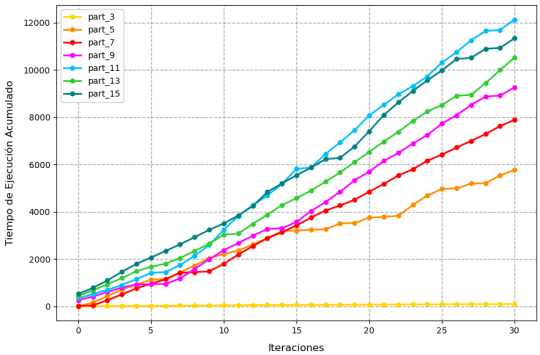
(c) Fitness - Modelo 2



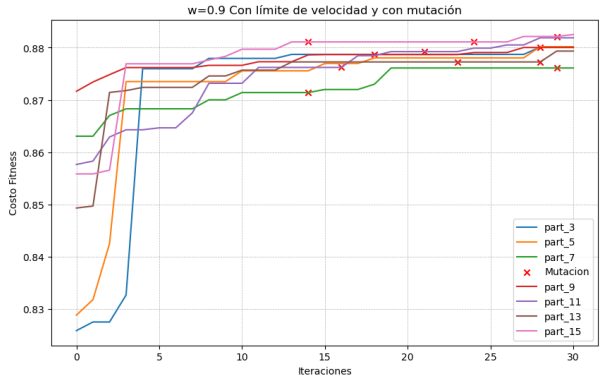
(d) Tiempo de ejecución - Modelo 2



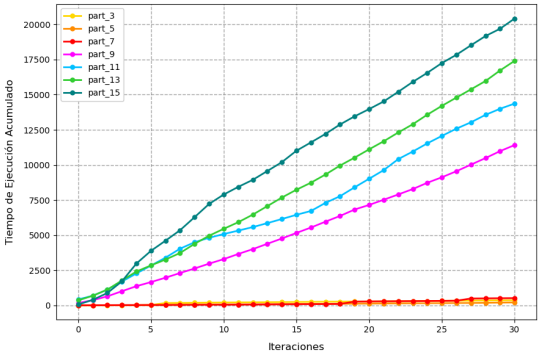
(e) Fitness - Modelo 3



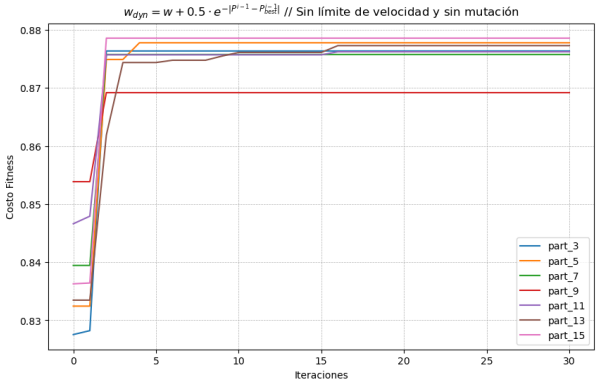
(f) Tiempo de ejecución - Modelo 3



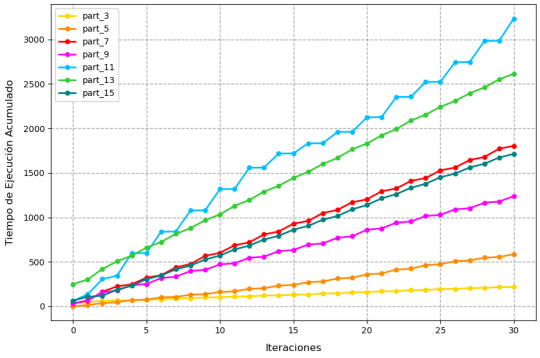
(g) Fitness - Modelo 4



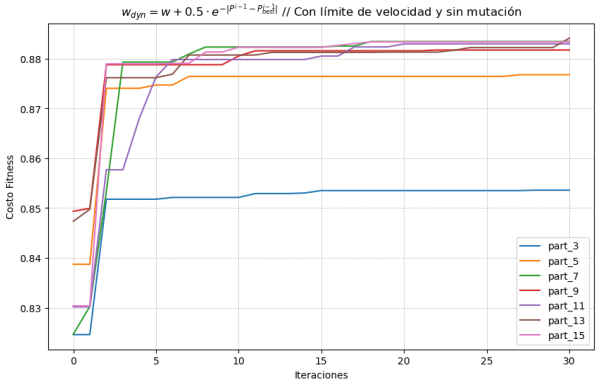
(h) Tiempo de ejecución - Modelo 4



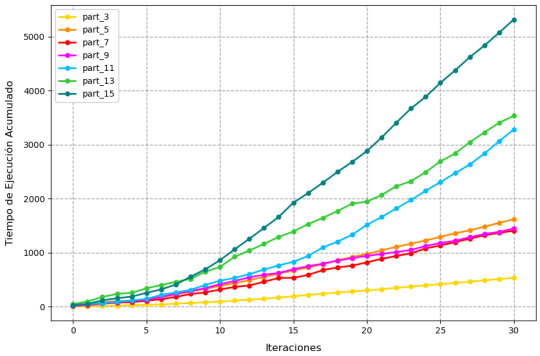
(a) Fitness - Modelo 5



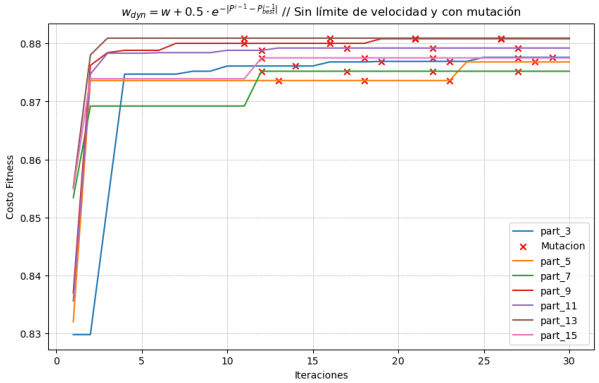
(b) Tiempo de ejecución - Modelo 5



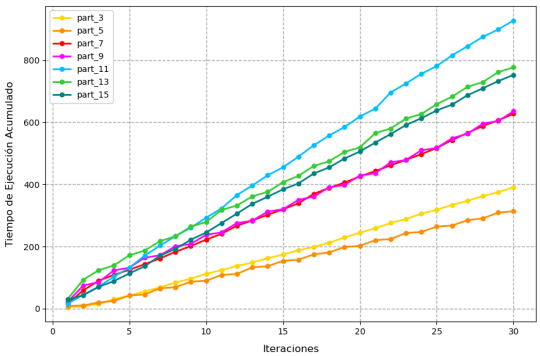
(c) Fitness - Modelo 6



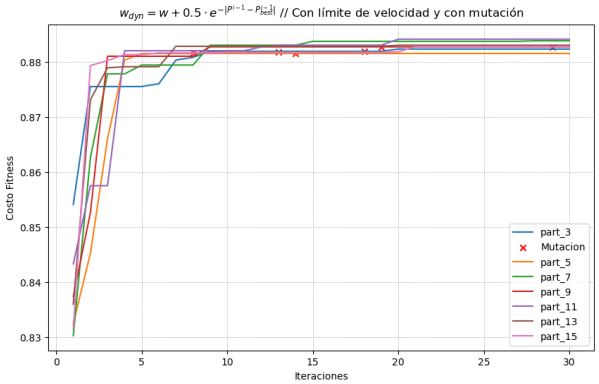
(d) Tiempo de ejecución - Modelo 6



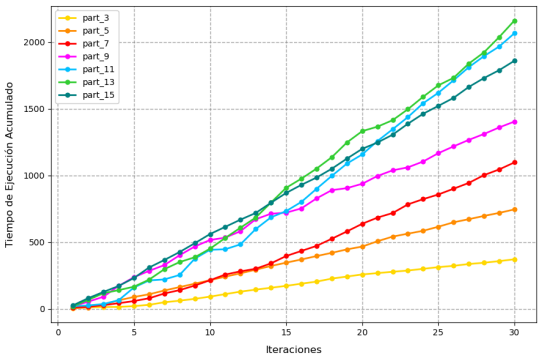
(e) Fitness - Modelo 7



(f) Tiempo de ejecución - Modelo 7



(g) Fitness - Modelo 8



(h) Tiempo de ejecución - Modelo 8

Figura 4.7: Comparación de fitness y tiempos de ejecución para los diferentes modelos

El w evolutivo utilizado ha sido tomado de [25]:

$$w_{dyn} = w + 0{,}5 \cdot e^{-|(P^{i-1} - P^{i-1}_{best})|} \tag{4.15}$$

Es importante destacar el mal e irregular comportamiento del modelo, a nivel general, sin límite de velocidad. En las Figuras 4.6a, 4.6c, 4.6e y 4.6g se pueden observar dos claras diferencias con las que tienen límite de velocidad:

1. Una convergencia prematura debido a la débil y escasa mejora del fit a partir de la décima iteración.
2. La dispersión de los fits en las 30-ésimas iteraciones es mucho más elevada. Se aprecian mucho más las diferencias al configurar el modelo con más o con menos partículas. Esto quiere decir que los límites de velocidad ayudan especialmente a los PSO con un número de partículas inferior.

4.5.6. Análisis estadístico

Para llevar a cabo los resultados estadísticos, se ha utilizado la aplicación JASP.
A continuación, se procede a hacer el análisis estadístico de las cuatro variables que conforman el modelo y que hacen variar el comportamiento del enjambre en el espacio de búsqueda:

1. Run: Número de partículas en la población (Ordinal)
2. Mutación: Si hay o no mutación de partículas (Factorial binaria)
3. Speed: Si hay o no límite de velocidad del enjambre (Factorial binaria)
4. w_type: Define si se ha utilizado un w evolutivo o fijo (Factorial binaria)

Lo primero que se ha hecho ha sido marginar en un dataset las últimas iteraciones de todos los modelos que se han estudiado anteriormente, en total 56 instancias.

```
Pob,runtype,CostoFitness,TiempoRun,n_estimators,max_depth,criterion,min_samples_split,min_samples_leaf,max_features,mutation,speed,w_type
3,1,0.8763623948,217.003844,195,2,log_loss,3,0,sqrt,NO,NO,evol
5,1,0.8777751384,586.616797,500,20,gini,10,1,sqrt,NO,NO,evol
7,1,0.8757581002,1803.126053,402,9,log_loss,7,1,None,NO,NO,evol
9,1,0.8691918268,1238.848742,500,12,entropy,2,5,sqrt,NO,NO,evol
11,1,0.8762252274,3234.268661,500,20,log_loss,2,5,None,NO,NO,evol
13,1,0.8772823957,2614.4586,10,20,log_loss,2,1,sqrt,NO,NO,evol
15,1,0.8785711336,1713.25478,36,19,gini,7,2,sqrt,NO,NO,evol
3,2,0.853591264,532.036322,500,5,gini,2,1,None,NO,YES,evol
5,2,0.87677073,1619.634756,500,20,log_loss,2,5,None,NO,YES,evol
7,2,0.8834446756,1405.165082,500,12,log_loss,2,1,log2,NO,YES,evol
9,2,0.8817397453,1447.943525,233,19,entropy,2,1,log2,NO,YES,evol
11,2,0.882972462,3280.420464,500,20,log_loss,2,1,None,NO,YES,evol
```

A continuación se ha realizado un estudio de los posibles outliers que se puedan tener en el dataset objeto de estudio. Para ello se hace un boxplot de la media y la varianza de las cuatro variables respecto de la variable objetivo *CostoFitness*:

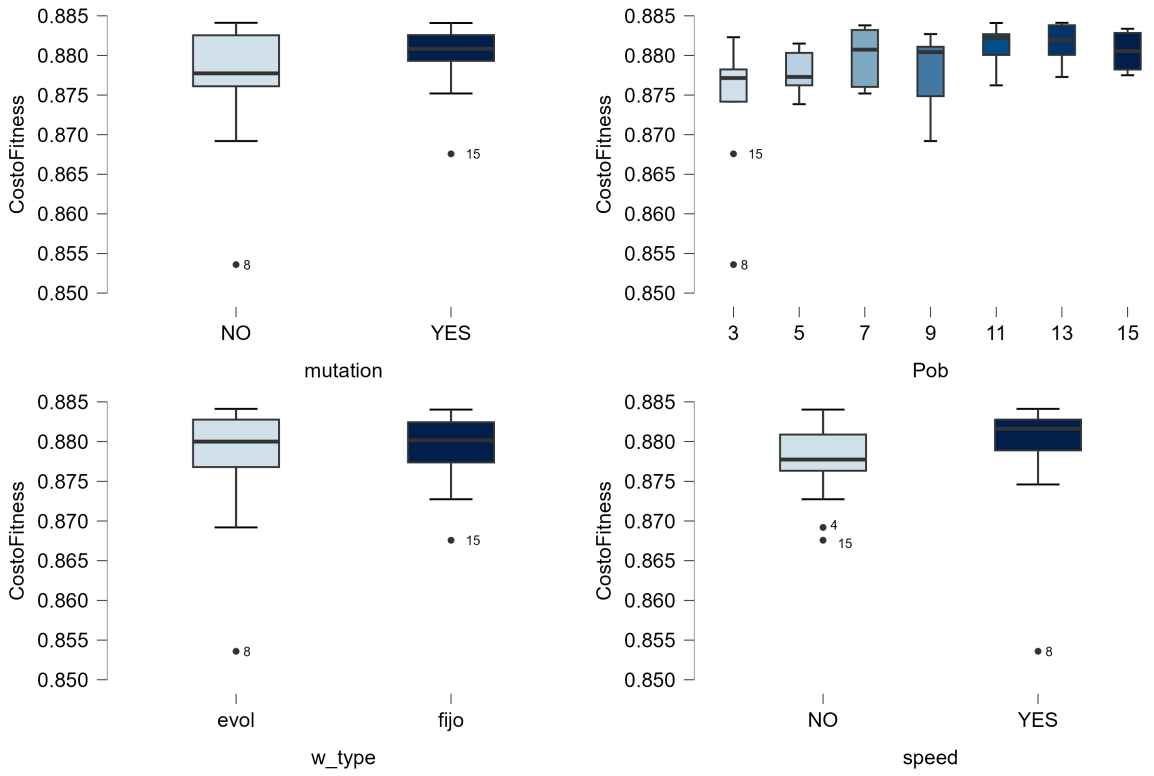


Figura 4.8: Boxplot

Se procede a hacer la limpieza de outliers, un total de 2 instancias, no solo por la condición de outlier por la varianza, sino también por los escasos datos que se tienen en este dataset. Es de suma importancia que los datos que se tengan sean fiables. Después de la eliminación de estos dos outliers se ha hecho el QQ-plot de cada variable resultando que, aparentemente, no hay ninguna instancia fuera de la normalidad de los residuos 6.3.

Antes de ver qué variables son significativas respecto de $CostoFitness$, se establecen las variables *speed, mutation* y *w_type* como factores y *Pob* como covariable pues no se quiere analizar el comportamiento de la variable respuesta respecto de los grupos de la variable *Pob*, sino el análisis por el efecto continuo del número de partículas del enjambre. Además, como se puede ver en la Figura 4.8, la media de la variable objetivo aumenta a medida que el número de partículas se hace más grande, y como consecuencia, se estima un efecto gradual y medible en el resultado. Antes de generar el modelo, es necesario estudiar si la covariable es independiente de los factores para asegurar la no multicolinealidad. Esta prueba se ha realizado mediante un ANOVA 4.5 en donde se comprueba, al estar todos los p-valores muy alejados del nivel de significancia $\alpha = 0{,}05$.

Cuadro 4.5: ANOVA - Pob

Cases	Sum of Squares	df	Mean Square	F	p
speed	0,000	1	0,000	0,000	1,000
w_type	0,000	1	0,000	0,000	1,000
mutation	0,000	1	0,000	0,000	1,000
speed * w_type	3,360	1	3,360	0,190	0,665
speed * mutation	3,360	1	3,360	0,190	0,665
w_type * mutation	3,360	1	3,360	0,190	0,665
speed * w_type * mutation	0,000	1	0,000	0,000	1,000
Residuals	812,000	46	17,652		

Dado que se ha cumplido el principio básico de no multicolinealidad entre factores y covariable, se procede a estudiar el nivel de significancia de estas variables. Para ello, se decide hacer hacer un modelo ANCOVA sin interacciones entre factores 4.16:

$$Y \sim Pob + mutation + speed + w_type \tag{4.16}$$

En los resultados del ANCOVA detallados en la Figura 4.6, se ha podido comprobar que *speed, mutation* y *Pob* son significativas dado que sus p-valores $0{,}025, 0{,}017$ y $0{,}005$ respectivamente, son menores que $\alpha = 0{,}05$. Por otro lado, *w_type* no se puede considerar significativa dado que su p-valor 0.605 no es significativo en ningún nivel de significancia al uso.

Cuadro 4.6: ANCOVA - CostoFitness

Cases	Sum of Squares	df	Mean Square	F	p
mutation	$3{,}850 \times 10^{-5}$	1	$3{,}850 \times 10^{-5}$	5,374	0,025
speed	$4{,}399 \times 10^{-5}$	1	$4{,}399 \times 10^{-5}$	6,140	0,017
Pob	$6{,}224 \times 10^{-5}$	1	$6{,}224 \times 10^{-5}$	8,689	0,005
w_type	$1{,}937 \times 10^{-6}$	1	$1{,}937 \times 10^{-6}$	0,270	0,605
Residuals	$3{,}438 \times 10^{-4}$	48	$7{,}164 \times 10^{-6}$		

Para comprobar la normalidad de los residuos se muestra la visualización de los cuantiles de los residuos del modelo en la Figura 4.9, donde se puede ver cómo la distribución está en línea recta indicando normalidad.

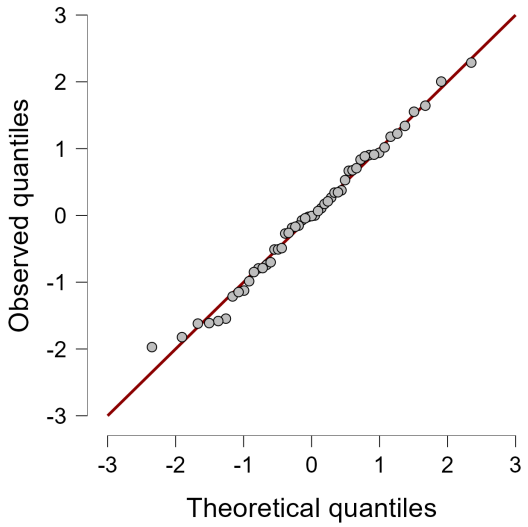


Figura 4.9: Gráfico Q-Q de los residuos.

Finalmente, para corroborar la normalidad de los residuos del modelo de manera inferencial, se realiza el siguiente contraste de hipótesis de *Shapiro-Wilk* mostrado en la Figura 4.10.

$$\begin{cases} H_0: \ \varepsilon_i \sim N(0, \sigma^2 I) \\ H_1: \ \varepsilon_i \not\sim N(0, \sigma^2 I) \end{cases} \tag{4.17}$$

Test de Normalidad	
Prueba	Shapiro-Wilk
Datos	Residuos
Estadístico	0.9841
***p*-valor**	0.6898

Figura 4.10: Resultados del test de Shapiro-Wilk.

Como se puede observar, el p-valor asociado al test es 0.689, al ser mayor que $\alpha = 0{,}05$, la hipótesis nula no puede ser rechazada, sugiriendo la normalidad de los residuos del modelo.

Para evaluar la homocedasticidad (igualdad de varianzas), se utiliza un gráfico de residuos vs. valores ajustados. Este gráfico ayuda a detectar si la varianza de los residuos es constante a lo largo de las predicciones realizadas por el modelo. Como se puede ver en la Figura 4.11, los residuos están dispersos de forma aleatoria a lo largo de la línea horizontal $y = 0$ indicando homocedasticidad.

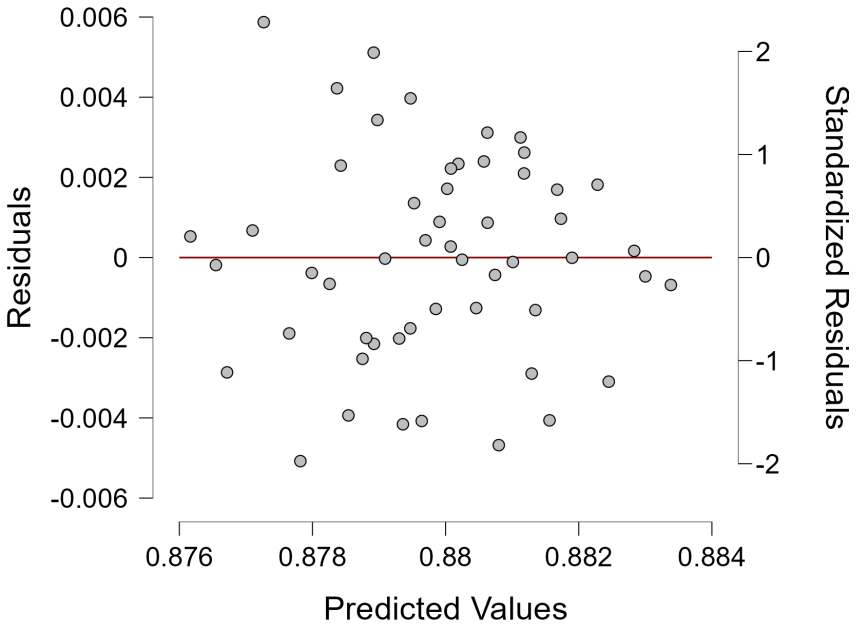


Figura 4.11: Homocedasticidad de los residuos

Al igual que con la normalidad de los residuos, la homocedasticidad se va a comprobar también de manera inferencial utilizando el test de *Levene*. Como se puede ver en la Figura 4.12, el p-valor 0.102 es mayor que el nivel de significancia $\alpha = 0{,}05$, y por lo tanto no se puede aceptar la hipótesis alternativa confirmando así la homocedasticidad:

$$\begin{cases} H_0: \ \sigma_1^2 = \sigma_2^2 = \cdots = \sigma_k^2 \\ H_1: \ \exists \ i, j \quad \text{tales que} \quad \sigma_i^2 \neq \sigma_j^2 \end{cases} \tag{4.18}$$

Test de Igualdad de Varianzas	
Prueba	Levene
Datos	Residuos
Estadístico	1.847
***p*-valor**	0.102

Figura 4.12: Resultados del test de Levene.

La última condición que queda para dar el modelo por válido es la independencia de los residuos. Para ello se hace el test de $Durbin - Watson$. Esta prueba está conformada por el estadístico de $Durbin - Watson = DW = \frac{\sum_{t=2}^{n}(e_t - e_{t-1})^2}{\sum_{t=1}^{n} e_t^2}$ que varía entre 0 y 4. Si el estadístico es claramente menor que 2, los residuos tendrán autocorrelación positiva manifestada en alguna forma de tendencia. Si el estadístico es mayor que 2, tendrán muy poca autocorrelación usualmente manifestada con una alternancia entre altos y bajos en cada observación de los residuos. Si el estadístico es aproximadamente 2, digamos que se

encuentra a una distancia d_α de 2, es el nivel significativo α el que marca si es suficiente poca distancia o no para ser considerado cercano. En el caso de estudio, se ha utilizado un $\alpha = 0{,}05$ y ha resultado un p-valor de 0.459 como se puede ver en la Figura 4.7, por lo que la hipótesis alternativa no puede ser aceptada, y por lo tanto, se acepta la hipótesis nula de independencia de los residuos de la muestra estudiada.

$$\begin{cases} H_0 : \rho = 0 \\ H_1 : \rho \neq 0 \end{cases} \tag{4.19}$$

Test de Independencia	
Prueba	Durbin-Watson
Estadístico	1.921
p**-valor**	0.459

Cuadro 4.7: Resultados del test de Durbin-Watson.

Ahora que ya se han probado los tres supuestos, se puede dar por válido el modelo, confirmando así que dos factores y la covariable son significativas en el modelo. A continuación se muestran en las Figuras 4.13 y 4.15 los gráficos de las medias marginales, así como un rango predictivo para ver cómo interactúan los factores y la covariable tomando sus distintos valores a un nivel de confianza del 95 %.

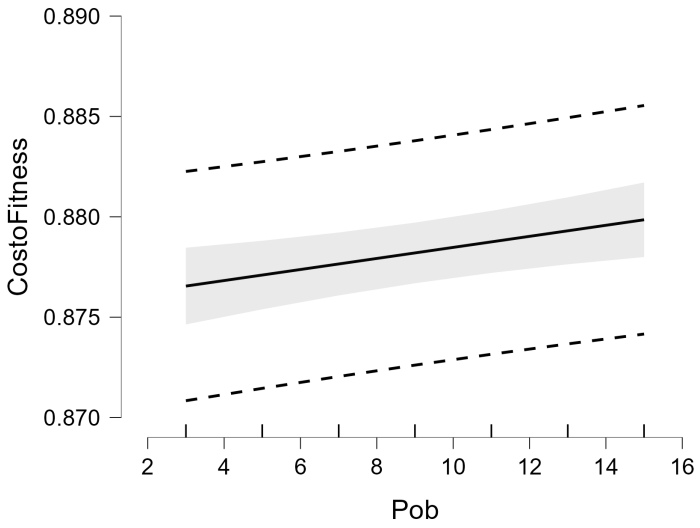


Figura 4.13: Medias marginales *Pob*

Se puede observar en la Figura 4.13 una relación lineal positiva entre el número de partículas *Pob* y la variable respuesta *CostoFitness*. Esta relación se podía intuir desde el principio del trabajo: a más partículas, más espacio de búsqueda se recorre y por lo tanto más posibilidades de encontrar un óptimo global se tienen. También se podía intuir desde el principio que a más partículas más modelos a evaluar se tienen y por lo tanto más coste computacional de la ejecución como se puede observar en la gráfica 4.14. Lo

que no se podía prever, es lo bien que ha funcionado el enjambre con pocas partículas. Se recuerda que el objetivo del PSO no es encontrar el mejor global, sino una aproximación al mejor global cuando el espacio de búsqueda es razonablemente extenso. Las ejecuciones con pocas partículas (3,5,7) han demostrado ser altamente fiables a la hora de encontrar una aproximación sin resultar con ejecuciones tan costosas como con enjambres más poblados, especialmente con los límites de velocidad y mutaciones activadas.

Cuadro 4.8: Estadística descriptiva de CostoFitness por número de partículas

	CostoFitness						
	3	5	7	9	11	13	15
Valid	8	8	8	8	8	8	8
Missing	0	0	0	0	0	0	0
Mean	0,874	0,878	0,880	0,878	0,881	0,882	0,881
Std. Deviation	0,009	0,003	0,004	0,005	0,003	0,003	0,003
Minimum	0,854	0,874	0,875	0,869	0,876	0,877	0,877
Maximum	0,882	0,881	0,884	0,883	0,884	0,884	0,883

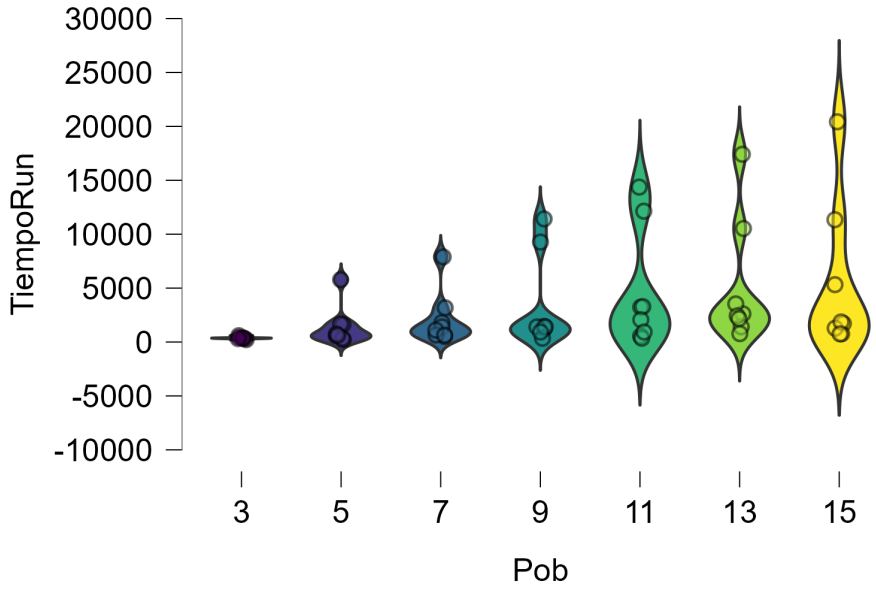


Figura 4.14: Distribución en modo violín de los tiempos de ejecución por número de partículas del enjambre

Adicionalmente, de las 8 ejecuciones con 3 partículas, solamente 2 han dado una posición cuyo $CostoFitness < 0{,}87$. Es decir, un 75 % del total de ejecuciones han dado un resultado de casi el 88 % de F1-Score con una media de 354 segundos, aproximadamente equivalente a 6 minutos. Como se puede ver en la Figura 4.14, se pueden ver ejecuciones de hasta 20.000 segundos equivalentes a 5,5 horas con unos resultados que varían de media un 0.7 %, un porcentaje despreciable si se tiene en cuenta que un enjambre con 5, 7, 9, 11, 13, 15 partículas tarda de media respectivamente un 400 %, 607 %, 969 %, 1302 %, 1442 %, 1534 % más.

Cuadro 4.9: Estadística descriptiva de los tiempos de ejecución por número de partículas

	TiempoRun						
	3	5	7	9	11	13	15
Valid	8	8	8	8	8	8	8
Missing	0	0	0	0	0	0	0
Mean	354.716	1449.235	2149.202	3432.210	4612.889	5108.486	5432.888
Std. Deviation	155.165	1833.572	2479.155	4319.826	5480.616	5827.945	7043.357
Minimum	104.926	217.875	525.058	349.590	350.053	776.912	752.639
Maximum	584.986	5785.062	7896.043	11418.355	14369.002	17412.568	20424.974

Para terminar el estudio estadístico de la significancia de los factores del modelo se realizaría una prueba Post-HOC, pero dado que los factores significativos son binarios y no hay interacciones entre ellos, con el análisis del ANCOVA queda claro que hay diferencias significativas entre las dos categorías de ambos factores.

Como se muestra en la Figura 4.15, la activación de la mutación de partículas por la variable *mutation* ha demostrado ser efectiva, al igual que la activación de la función de límite de velocidad para frenar las partículas cuando excedían cierta velocidad en sus respectivas dimensiones del vector velocidad.

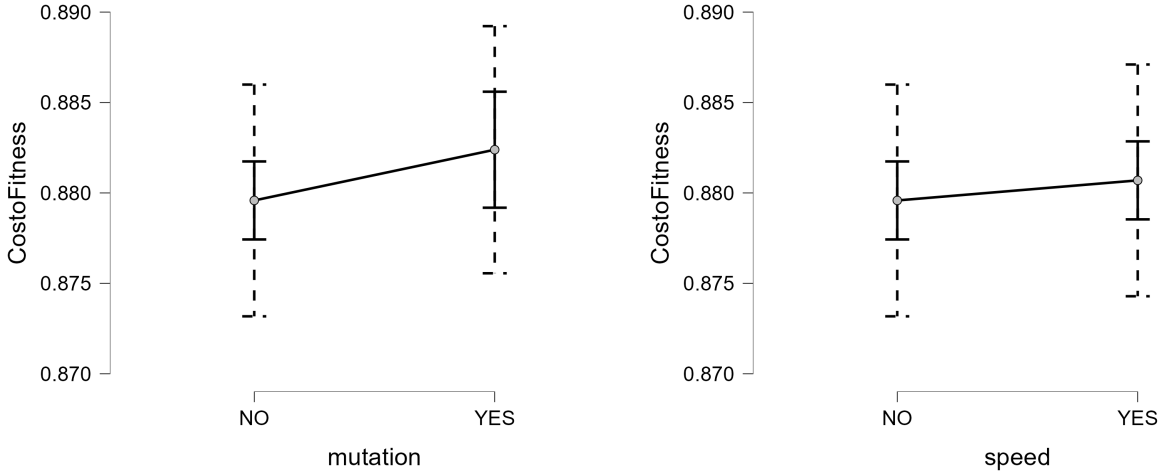


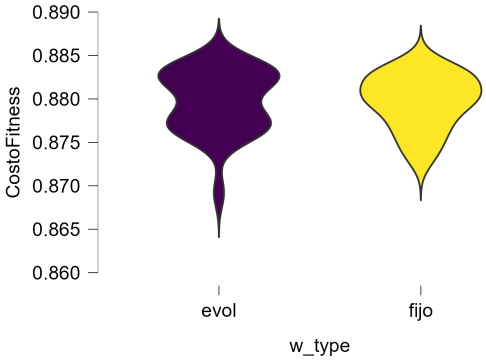
Figura 4.15: Medias marginales de *mutation* y *speed*

En cuanto a la inercia utilizada, *w_type*, se ha visto que no es significativa. Esto se explica porque las medias y las desviaciones estándar de las distribuciones son prácticamente indénticas, como se muestra en la tabla 4.10.

Cuadro 4.10: Descriptiva de *w_type*

	CostoFitness	
	evol	fijo
Valid	27	27
Missing	0	0
Mean	0,880	0,880
Std. Deviation	0,004	0,003
Minimum	0,869	0,873
Maximum	0,884	0,884

Figura 4.16: Distribución violín de *w_type*



Este resultado confirma la idea de que una inercia fijada en 0.9 ha sido fiable en la búsqueda realizada de hiperparámetros discretos, tolerando los saltos entre número natural y número natural. Este resultado que se acaba de detallar no es trivial. En este trabajo ha funcionado muy bien, pero no tendría por qué haber tenido este comportamiento. De hecho en otros trabajos de búsqueda de hiperparámetros discretos por PSO [25], se ha tenido que utilizar un w evolutivo como consecuencia del estancamiento de las partículas al no poder terminar de dar el salto mencionado.

Aún así, si se observa con detalle la distribución de la variable *w_type* en la Figura 4.16, se puede observar que en $w = 0{,}9$, la distribución es uniforme mientras que la distribución de w_{dyn} no lo es. Esto último podría estar sugiriendo que, aunque *w_type* por sí sola no sea significativa en el modelo, podría estar interactuando con otro factor que podría dar mayor poder explicativo en una supuesta regresión lineal.

Para estudiar esta interacción se plantean mediante un ANCOVA bayesiano las 19 combinaciones diferentes de modelos ANCOVA que se pueden crear y se ordenan por probabilidad de mayor a menor. En la tabla 4.11 se muestran los 10 mejores modelos de los 19 que se han evaluado.

Cuadro 4.11: Estadística bayesiana: comparación de modelos

Models	P(M)	P(M|data)	BF_M	BF_{10}	error %
mutation + speed + w_type + speed * w_type	0,053	0,438	14,007	1,000	
mutation + speed + w_type + mutation * speed + speed * w_type	0,053	0,227	5,284	0,519	6,326
mutation + speed + w_type + mutation * w_type + speed * w_type	0,053	0,161	3,458	0,368	2,928
mutation + speed + w_type + mutation * speed + mutation * w_type + speed * w_type	0,053	0,088	1,731	0,201	2,725
speed + w_type + speed * w_type	0,053	0,043	0,805	0,098	10,409
mutation + speed + w_type + mutation * speed + mutation * w_type + speed * w_type + mutation * speed * w_type	0,053	0,037	0,699	0,085	2,942
mutation + speed	0,053	0,002	0,044	0,006	1,714
mutation + speed + mutation * speed	0,053	0,001	0,019	0,002	1,864
mutation + speed + w_type	0,053	$6{,}730 \times 10^{-4}$	0,012	0,002	1,900
speed	0,053	$5{,}817 \times 10^{-4}$	0,010	0,001	1,645

Resulta que el modelo 4.20 en el que se incluye una interacción de *w_type* con *speed*, es el que, aparentemente, mejor explica la variable objetivo. Como muestra la tabla, la probabilidad de que este modelo sea el mejor de los 19 es del 43.8 %. Además, el Factor de

Bayes del primer modelo BF_M es de 14.007 y es considerado como una evidencia fuerte a su favor respecto del resto a través de la escala de interpretación del Factor de Bayes tradicional. Adicionalmente, el BF_{01} del primer modelo es prácticamente el doble que el del segundo 0.519, es decir, tiene el doble de probabilidad de ser elegido como el mejor modelo respecto del segundo. En consecuencia, se ha elegido el modelo para estudiar la interacción de las variables ya mencionadas respecto de la objetivo:

$$Y \sim Pob + mutation + speed + w_type + w_type * speed \tag{4.20}$$

Los resultados del ANCOVA mostrados en 4.12 son excepcionales en el sentido de que todas las variables estudiadas son altamente significativas concordando con lo visto en el ANCOVA sin interacciones anteriormente. Asimismo, se confirma la hipótesis que se había planteado; una interacción significativa de *w_type* con algún otro factor, que ha resultado ser *speed*.

Cuadro 4.12: ANCOVA - CostoFitness

Cases	Sum of Squares	df	Mean Square	F	p
mutation	$5{,}616 \times 10^{-5}$	1	$5{,}616 \times 10^{-5}$	8,006	0,007
speed	$6{,}118 \times 10^{-5}$	1	$6{,}118 \times 10^{-5}$	8,721	0,005
w_type	$3{,}158 \times 10^{-8}$	1	$3{,}158 \times 10^{-8}$	0,005	0,947
speed * w_type	$1{,}387 \times 10^{-4}$	1	$1{,}387 \times 10^{-4}$	19,773	$< .001$
Residuals	$3{,}437 \times 10^{-4}$	49	$7{,}015 \times 10^{-6}$		

Aunque el ANCOVA sea fantástico, nos encontramos con un problema grave de heterocedasticidad que desacredita el modelo: las pruebas de homocedasticidad de Levene 4.18 y Breusch-Pagan 4.17 aceptan la hipótesis alternativa. Esto es un claro indicio de heterocedasticidad acreditada por dos test a la vez.

Test de Igualdad de Varianzas	
Prueba	Breusch - Pagan
Datos	Residuos
F-Estadístico	2.57
***p*-valor**	0.0386

Figura 4.17: Resultados del test de Breusch - Pagan.

Test de Igualdad de Varianzas	
Prueba	Levene
Datos	Residuos
F-Estadístico	3.14
***p*-valor**	0.081

Figura 4.18: Resultados del test de Levene.

Indigando en el motivo de este problema, se ha visto que en todos y cada uno de los modelos que incluyen esta interacción, fallan los test de Levene y Breusch-Pagan. Esto ha motivado la búsqueda de una razón que explique el problema que se está encontrando. Efectivamente, como muestra la Figura 4.19, la heterocedasticidad viene inducida por la inclusión de interacción $w_type * speed$ debido a que las varianzas de las distribuciones de los residuos de las cuatro categorías que genera esta interacción son muy dispersas entre sí:

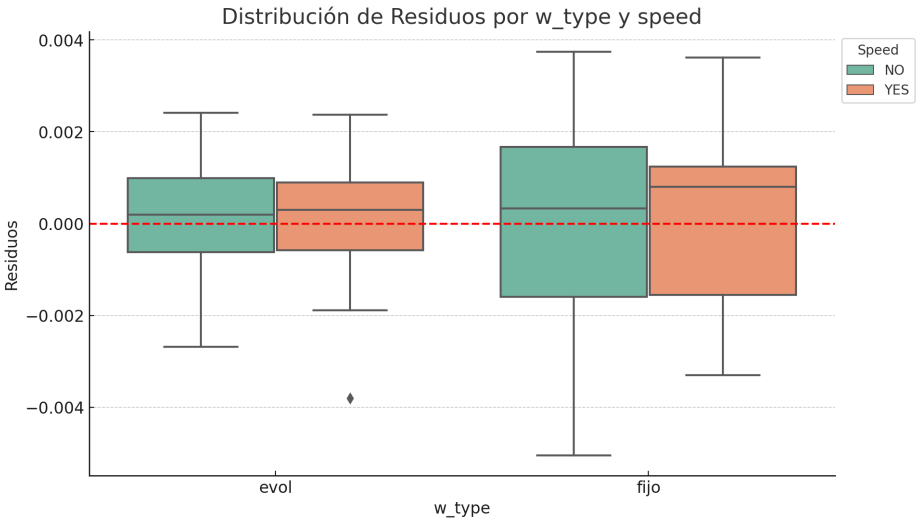


Figura 4.19: Distribución de los residuos de la interacción $w_type * speed$

Se han probado distintas transformaciones de la variable respuesta como Cox-Box con distintos lambdas, logarítmicas y exponenciales sin éxito. Por ello, las única opcion que queda para mitigar la heterocedasticidad es el uso de correctores como HC3 que ajusten los errores estándar. Sin embargo, no se dispone de una herramienta de trabajo tan potente para evaluar este modelo.

En conclusión, aunque las pruebas de independencia y normalidad de los residuos resultaran favorables, no se puede afirmar que el modelo que incluye esta interacción sea fiable. De hecho, ningún modelo que contenga esta interacción lo es. Esto es consecuencia de lo mencionado anteriormente: la heterocedasticidad está presente en todos aquellos modelos que contengan la interacción $w_type * speed$. Sin embargo, hay razones para creer en la significancia de esta interacción por la evolución del enjambre, la cual no se ha tenido en cuenta en este estudio estadístico al sólamente incluir la trigésima iteración de los 56 modelos estudiados en el dataset.

Se estudia a continuación cómo se ha comportado el PSO en la búsqueda de hiperparámetros del Random Forest Classifier. Lo primero que hay que destacar es sin lugar a dudas que el PSO ha encontrado mejores combinaciones de hiperparámetros que el Grid Search. El lector se podrá sorprender por este resultado. Se ha explicado en el capítulo 3 que el PSO es un método heurístico y normalmente no encuentra la mejor posición global del espacio de búsqueda sino una aproximación. Pero se debe recordar que el espacio de búsqueda del PSO es continuo. Cuando el Grid Search es ejecutado, se tienen predefinidas las combinaciones que se van a estudiar. En el PSO, a diferencia del Grid Search, va explorando el espacio de búsqueda de forma continua, es decir, tiene 'acceso' a más valores en hiperparámetros que no sean categóricos como pueden ser $n_estimators$ o max_depth. Mientras que el Grid Search solo puede acceder a los valores prefijados 10,50,100,150,200 y 250 y 5,10 y 20 respectivamente, el PSO tiene acceso al estudio de los rangos (10,200) y (5,20) respectivamente. Esto último se ve claramente al analizar los ajustes de los modelos PSO que han sobrepasado al modelo encontrado por Grid Search:

CostoFitness	**TiempoRun (s)**	**Parámetros RFC**
0.8831	31995	[100, 20, entropy, 5, 1, None,0]

Cuadro 4.13: Tabla de parámetros y resultados Grid Search

Adicionalmente, se puede observar que en cada uno de los modelos detallados a continuación en 4.14, el tiempo de ejecución es muy inferior al del Grid Search.

CostoFitness	**TiempoRun (s)**	**Parámetros RFC**	**Parámetros PSO**
0.8834	1405	[500, 12, log_loss, 2, 1, log2,0]	[7,NO,YES,evol]
0.8841	3534	[500, 20, log_loss, 4, 1, None,0]	[13,YES,NO,fijo]
0.8833	5320	[500,20,log_loss,2,1,None,0]	[15,NO,YES,evol]
0.8837	10527	[206, 19, log_loss, 4, 1, None,0]	[13,YES,NO,fijo]
0.8832	11345	[250,14,log_loss,4,1,log2,0]	[15,YES,NO,fijo]
0.8840	1410	[265, 15, entropy, 6, 2, log2,0]	[13,NO,NO,fijo]
0.8838	1098	[466, 5, gini, 7, 1, sqrt,0]	[7,YES,YES,evol]
0.8841	2067	[500, 14, log_loss, 2, 1, None,0]	[15,YES,YES,evol]

Cuadro 4.14: Tabla de parámetros y resultados PSO

Es cierto que se pueden observar valores muy buenos sin limitación de velocidad y sin mutaciones, pero esto es mera coincidencia. No olvidemos que al final, el PSO tiene una parte importante de aleatoriedad tanto en su inicialización como en las mutaciones. Lo importante es que a nivel general, como ha demostrado el estudio estadístico, sí que se pueden afirmar diferencias significativas entre las distintas configuraciones del algoritmo.

Capítulo 5

Conclusiones y trabajo futuro

En este trabajo se ha estudiado en profundidad el algoritmo PSO, una técnica de optimización heurística que ha sido utilizada para el ajuste de hiperparámetros del modelo *Random Forest Classifier* en un problema de clasificación.

5.1. Conclusiones

Al comparar los resultados obtenidos de métodos más exhaustivos como *Grid Search* con los resultados del PSO, se han evidenciado:

1. Una clara fiabilidad y eficacia a la hora de ajustar hiperparámetros del modelo, llegando incluso a mejorar los resultados obtenidos en *Grid Search*.
2. Ventajas a nivel de coste computacional, lo cual era esperado, pero no lo era el balance tan bueno entre este y los resultados obtenidos.

Puede que el modelo tuviera un gran número de óptimos locales con desempeño similar al óptimo global. Esta hipótesis es improbable debido a que la convergencia de los enjambres se ha dado de manera parecida en todas las configuraciones, indicando así que el enjambre, buscó y encontró de manera precisa, no fue mera casualidad. Adicionalmente, todos y cada uno de las ejecuciones partieron de modelos con un *Fitness* mucho menor, rondando el 83 % y todos terminaron mejorando este resultado en un 5 %, alcanzando cotas que sobrepasaban en muchos casos un 88 % de puntuación.

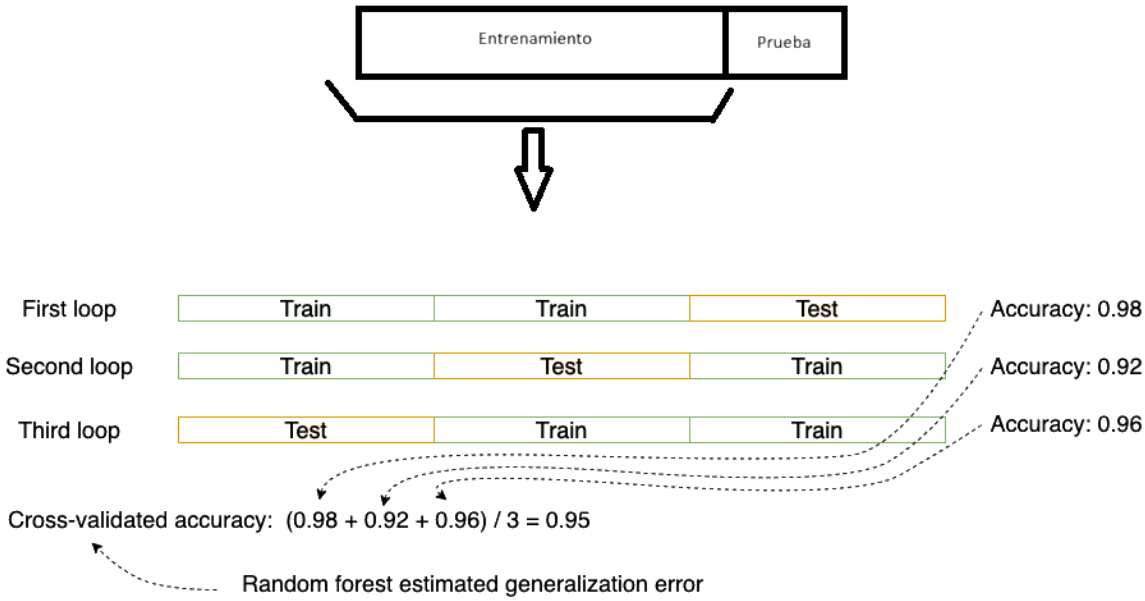
Como conclusión del resultado estadístico, se deben tener cuenta las distintas configuraciones del PSO para conseguir una buena actuación del modelo sobre la optimización. De no ser así, el espacio de búsqueda podría no ser explorado correctamente y dar resultados alejados del óptimo global. Esto ha ocurrido concretamente en dos ejecuciones donde el número de partículas utilizado ha sido el menor y el enjambre ha quedado estancado en óptimos locales.

5.2. Trabajo futuro

Si bien los resultados de las pruebas realizadas han resultado prometedores, existen múltiples líneas de investigación todavía por abordar. Entre ellas, la exploración de otras variantes del PSO, incluyendo el estudio de otras inercias evolutivas, combinaciones distintas de parámetros cognitvo-sociales o la variación de número mayor de partículas empleadas. En referencia a esto último, me he visto limitado por la capacidad de computación para llevar a cabo esta exploración. Las ejecuciones con 11, 13, 15 partículas han llevado horas lo que han impedido una exploración más amplia de esta variable. También se ha dejado sin tratar la aplicación del PSO a otros métodos de optimización de Aprendizaje Automático como las Redes Neuronales, que son más complejos y requieren de un ajuste más preciso.

Capítulo 6

Anexo



(a) *Nested Cross-Validation*

Figura 6.2: Comparación de métodos de validación

Cuadro 6.1: Datos del archivo `w_evol_mut_nospeedlimit.csv`

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
1	1	0.8298	4.0979	
1	2	0.8298	3.7539	
1	3	0.8523	7.2996	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
1	4	0.8747	15.8004	
1	5	0.8747	11.3073	
1	6	0.8747	13.3365	
1	7	0.8747	12.9389	
1	8	0.8752	15.5220	
1	9	0.8752	12.8153	
1	10	0.8761	15.6424	
1	11	0.8761	11.6581	
1	12	0.8761	13.3699	
1	13	0.8761	11.2725	
1	14	0.8761	13.7830	1
1	15	0.8761	11.7972	
1	16	0.8768	13.5053	
1	17	0.8768	11.1788	
1	18	0.8768	13.2209	
1	19	0.8769	16.9137	1
1	20	0.8769	15.2871	
1	21	0.8769	14.5230	
1	22	0.8769	16.8861	
1	23	0.8769	12.9674	1
1	24	0.8769	16.8283	
1	25	0.8776	12.8968	
1	26	0.8776	15.5166	
1	27	0.8776	13.0404	
1	28	0.8776	15.3184	
1	29	0.8776	12.7563	1
1	30	0.8776	15.3339	
2	1	0.8320	8.7188	
2	2	0.8736	1.9480	
2	3	0.8736	9.3325	
2	4	0.8736	5.1577	
2	5	0.8736	17.3767	
2	6	0.8736	3.3772	
2	7	0.8736	19.7745	
2	8	0.8736	3.5454	
2	9	0.8736	17.5750	
2	10	0.8736	3.4988	
2	11	0.8736	18.2224	
2	12	0.8736	3.4577	
2	13	0.8736	21.0910	1
2	14	0.8736	3.5943	
2	15	0.8736	17.3787	
2	16	0.8736	3.5245	
2	17	0.8736	17.4884	
2	18	0.8736	6.0291	1

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
2	19	0.8736	17.7070	
2	20	0.8736	3.5496	
2	21	0.8736	18.4063	
2	22	0.8736	3.4698	
2	23	0.8736	19.4217	1
2	24	0.8768	3.4673	
2	25	0.8768	17.1836	
2	26	0.8768	3.3930	
2	27	0.8768	17.3027	
2	28	0.8768	5.7913	1
2	29	0.8768	18.4963	
2	30	0.8768	4.3575	
3	1	0.8534	16.8824	
3	2	0.8692	41.8187	
3	3	0.8692	31.5820	
3	4	0.8692	18.3913	
3	5	0.8692	16.4690	
3	6	0.8692	18.3017	
3	7	0.8692	18.4052	
3	8	0.8692	21.2661	
3	9	0.8692	18.4385	
3	10	0.8692	20.8888	
3	11	0.8692	18.6127	
3	12	0.8752	26.2423	2
3	13	0.8752	16.5052	
3	14	0.8752	18.6522	
3	15	0.8752	17.3818	
3	16	0.8752	19.3864	
3	17	0.8752	30.2048	2
3	18	0.8752	19.7596	
3	19	0.8752	16.6091	
3	20	0.8752	20.1414	
3	21	0.8752	17.1350	
3	22	0.8752	18.3440	2
3	23	0.8752	17.6203	
3	24	0.8752	18.6034	
3	25	0.8752	19.7023	
3	26	0.8752	25.5287	
3	27	0.8752	22.7790	2
3	28	0.8752	22.3582	
3	29	0.8752	18.2543	
3	30	0.8752	21.0928	
4	1	0.8370	22.5263	
4	2	0.8762	52.2069	
4	3	0.8784	10.3649	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
4	4	0.8788	38.2364	
4	5	0.8788	8.6684	
4	6	0.8788	32.3289	
4	7	0.8800	7.4878	
4	8	0.8800	29.1871	
4	9	0.8800	7.4903	
4	10	0.8800	29.5904	
4	11	0.8800	7.4764	2
4	12	0.8800	30.1725	
4	13	0.8800	7.5654	
4	14	0.8800	29.1598	
4	15	0.8800	7.6023	
4	16	0.8800	30.2915	2
4	17	0.8800	10.4428	
4	18	0.8800	29.7858	
4	19	0.8808	7.7408	
4	20	0.8808	30.2569	
4	21	0.8808	7.4470	2
4	22	0.8808	34.7626	
4	23	0.8808	8.0353	
4	24	0.8808	30.9376	
4	25	0.8808	7.8303	
4	26	0.8808	31.0627	2
4	27	0.8808	14.4063	
4	28	0.8808	32.5807	
4	29	0.8808	8.3465	
4	30	0.8808	31.7857	
5	1	0.8356	16.4459	
5	2	0.8747	28.6538	
5	3	0.8783	25.7704	
5	4	0.8783	32.3283	
5	5	0.8783	28.8878	
5	6	0.8784	38.7837	
5	7	0.8784	32.4009	
5	8	0.8784	29.9239	
5	9	0.8784	27.8288	
5	10	0.8788	31.7822	
5	11	0.8788	30.0617	
5	12	0.8788	42.9540	3
5	13	0.8792	30.7361	
5	14	0.8792	32.7049	
5	15	0.8792	25.8182	
5	16	0.8792	34.2368	
5	17	0.8792	36.5949	3
5	18	0.8792	31.3992	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
5	19	0.8792	27.3772	
5	20	0.8792	33.7168	
5	21	0.8792	25.6170	
5	22	0.8792	51.7772	3
5	23	0.8792	29.1807	
5	24	0.8792	30.9581	
5	25	0.8792	25.1062	
5	26	0.8792	34.5907	
5	27	0.8792	29.0618	3
5	28	0.8792	30.1872	
5	29	0.8792	24.0273	
5	30	0.8792	28.8236	
6	1	0.8552	32.0583	
6	2	0.8780	60.8698	
6	3	0.8809	30.7606	
6	4	0.8809	15.7184	
6	5	0.8809	32.5559	
6	6	0.8809	15.3087	
6	7	0.8809	30.3692	
6	8	0.8809	15.4837	
6	9	0.8809	31.0144	
6	10	0.8809	14.8566	
6	11	0.8809	38.9084	3
6	12	0.8809	14.1294	
6	13	0.8809	29.2774	
6	14	0.8809	14.7831	
6	15	0.8809	31.0985	
6	16	0.8809	20.2221	3
6	17	0.8809	32.0280	
6	18	0.8809	15.1442	
6	19	0.8809	29.7848	
6	20	0.8809	14.8554	
6	21	0.8809	45.7706	3
6	22	0.8809	15.0195	
6	23	0.8809	31.7399	
6	24	0.8809	14.8353	
6	25	0.8809	31.8859	
6	26	0.8809	24.0014	3
6	27	0.8809	31.7660	
6	28	0.8809	15.1513	
6	29	0.8809	32.0175	
6	30	0.8809	15.4975	
7	1	0.8549	29.5212	
7	2	0.8739	13.6199	
7	3	0.8739	26.8858	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
7	4	0.8739	19.0212	
7	5	0.8739	24.4357	
7	6	0.8739	23.1336	
7	7	0.8739	33.9932	
7	8	0.8739	21.7922	
7	9	0.8739	29.7118	
7	10	0.8739	23.1697	
7	11	0.8739	30.4197	
7	12	0.8775	30.2709	4
7	13	0.8775	31.5371	
7	14	0.8775	22.9295	
7	15	0.8775	23.6149	
7	16	0.8775	19.0461	
7	17	0.8775	31.9480	
7	18	0.8775	19.7871	4
7	19	0.8775	28.5277	
7	20	0.8775	22.4694	
7	21	0.8775	28.8796	
7	22	0.8775	27.3108	4
7	23	0.8775	29.2367	
7	24	0.8775	21.7890	
7	25	0.8775	25.1525	
7	26	0.8775	18.7842	
7	27	0.8775	30.0455	4
7	28	0.8775	22.4225	
7	29	0.8775	23.1071	
7	30	0.8775	20.0763	

Cuadro 6.2: Datos del archivo `w_evol_mut_speedlimit.csv`

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
1	1	0.8541	4.8106	
1	2	0.8755	7.172	
1	3	0.8755	2.37	
1	4	0.8755	1.1794	
1	5	0.8755	5.4838	
1	6	0.876	10.0848	
1	7	0.8803	18.9008	
1	8	0.8808	11.9656	
1	9	0.8819	13.6133	
1	10	0.8819	16.8522	
1	11	0.8819	18.6323	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
1	12	0.8819	18.7814	
1	13	0.8819	14.953	
1	14	0.8819	13.8526	
1	15	0.8819	14.486	
1	16	0.8819	16.3241	
1	17	0.8819	15.4848	
1	18	0.8819	22.4959	1
1	19	0.8819	15.5364	
1	20	0.8823	15.6823	
1	21	0.8823	10.0646	
1	22	0.8823	9.9051	
1	23	0.8823	9.3874	
1	24	0.8823	12.6423	
1	25	0.8823	12.8622	
1	26	0.8823	9.6417	
1	27	0.8823	13.7648	
1	28	0.8823	9.3752	
1	29	0.8823	12.4292	
1	30	0.8823	13.6045	
2	1	0.8325	8.7547	
2	2	0.8452	1.7774	
2	3	0.866	21.9706	
2	4	0.8803	32.5174	
2	5	0.8815	24.6271	
2	6	0.8815	20.2721	
2	7	0.8815	28.4256	
2	8	0.8815	25.427	
2	9	0.8815	25.8283	
2	10	0.8815	25.9831	
2	11	0.8815	26.2874	
2	12	0.8815	25.3747	
2	13	0.8815	26.8855	
2	14	0.8815	27.8153	1
2	15	0.8815	25.0328	
2	16	0.8815	23.901	
2	17	0.8815	25.6171	
2	18	0.8815	24.253	
2	19	0.8815	25.9034	
2	20	0.8815	21.0903	
2	21	0.8815	39.6437	
2	22	0.8815	35.1793	
2	23	0.8815	21.8181	
2	24	0.8815	21.4251	
2	25	0.8815	30.7163	
2	26	0.8815	33.1251	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
2	27	0.8815	23.2785	
2	28	0.8815	24.5119	
2	29	0.8815	22.8154	
2	30	0.8815	26.0988	
3	1	0.8303	8.5717	
3	2	0.8626	8.7095	
3	3	0.8778	10.9137	
3	4	0.8778	15.0417	
3	5	0.8794	15.7514	
3	6	0.8794	21.2984	
3	7	0.8794	35.6749	
3	8	0.8794	25.1106	
3	9	0.883	34.2732	
3	10	0.883	39.8271	
3	11	0.883	42.6893	
3	12	0.883	23.9982	
3	13	0.883	19.246	
3	14	0.883	40.4677	
3	15	0.8837	55.0843	
3	16	0.8837	38.1048	
3	17	0.8837	38.0536	
3	18	0.8837	53.5058	
3	19	0.8837	55.1565	
3	20	0.8837	57.1086	
3	21	0.8837	46.199	
3	22	0.8837	34.4179	
3	23	0.8837	64.6839	
3	24	0.8837	39.2879	
3	25	0.8837	34.9739	
3	26	0.8837	43.1171	
3	27	0.8837	43.5563	
3	28	0.8838	58.4548	
3	29	0.8838	42.9075	
3	30	0.8838	52.4064	
4	1	0.8374	20.8275	
4	2	0.8526	33.4832	
4	3	0.881	37.3757	
4	4	0.881	80.9042	
4	5	0.881	65.5197	
4	6	0.881	47.3079	
4	7	0.881	43.6285	
4	8	0.881	73.3259	
4	9	0.8827	66.004	
4	10	0.8827	48.4925	
4	11	0.8827	17.8917	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
4	12	0.8827	47.9491	
4	13	0.8827	89.4851	
4	14	0.8827	41.6919	
4	15	0.8827	7.9646	
4	16	0.8827	31.9251	
4	17	0.8827	75.2561	
4	18	0.8827	61.7526	
4	19	0.8827	15.1435	2
4	20	0.8827	32.5698	
4	21	0.8827	58.3379	
4	22	0.8827	42.4924	
4	23	0.8827	22.1729	
4	24	0.8827	43.7257	
4	25	0.8827	62.5613	
4	26	0.8827	51.0104	
4	27	0.8827	48.6865	
4	28	0.8827	44.3424	
4	29	0.8827	48.9805	2
4	30	0.8827	44.3193	
5	1	0.8433	20.7224	
5	2	0.8575	5.3076	
5	3	0.8575	10.8242	
5	4	0.882	28.9871	
5	5	0.882	95.0139	
5	6	0.882	52.9731	
5	7	0.882	7.5758	
5	8	0.882	33.1974	
5	9	0.882	122.3749	
5	10	0.882	66.5888	
5	11	0.882	3.8953	
5	12	0.8827	36.8071	
5	13	0.883	114.5252	
5	14	0.883	87.7651	
5	15	0.883	47.0911	
5	16	0.883	68.7186	
5	17	0.883	98.5778	
5	18	0.883	96.6757	
5	19	0.883	92.7615	
5	20	0.8841	68.2215	
5	21	0.8841	100.0968	
5	22	0.8841	88.7332	
5	23	0.8841	91.0884	
5	24	0.8841	103.5046	
5	25	0.8841	78.7071	
5	26	0.8841	93.4267	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
5	27	0.8841	98.3634	
5	28	0.8841	83.0072	
5	29	0.8841	72.2618	
5	30	0.8841	99.4417	
6	1	0.836	19.7033	
6	2	0.8731	48.5455	
6	3	0.8789	49.8579	
6	4	0.8791	22.8327	
6	5	0.8791	25.951	
6	6	0.8791	53.0449	
6	7	0.8828	78.3004	
6	8	0.8828	53.1855	
6	9	0.8828	36.1592	
6	10	0.8828	64.0462	
6	11	0.8828	80.6933	
6	12	0.8828	76.787	
6	13	0.8828	75.7612	
6	14	0.8828	112.1367	
6	15	0.8828	111.0465	
6	16	0.8828	68.7545	
6	17	0.8828	75.3991	
6	18	0.8828	85.5105	
6	19	0.8828	111.1388	
6	20	0.883	84.4899	
6	21	0.883	33.1913	
6	22	0.883	48.8122	
6	23	0.883	82.3175	
6	24	0.883	93.0244	
6	25	0.883	85.774	
6	26	0.883	56.2989	
6	27	0.883	106.3589	
6	28	0.883	84.8595	
6	29	0.883	114.5313	
6	30	0.883	124.1439	
7	1	0.8315	27.231	
7	2	0.8793	53.6881	
7	3	0.8802	46.7098	
7	4	0.8813	44.7582	
7	5	0.8813	59.6308	
7	6	0.8816	78.83	
7	7	0.8816	56.2719	
7	8	0.8816	59.6012	4
7	9	0.8816	66.1921	
7	10	0.8817	68.5189	
7	11	0.8817	54.0236	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
7	12	0.8817	54.0039	
7	13	0.8817	50.0227	4
7	14	0.8817	76.3182	
7	15	0.8817	73.7807	
7	16	0.8817	60.6565	
7	17	0.8817	55.0019	
7	18	0.8817	66.0433	
7	19	0.8817	76.2161	
7	20	0.8818	73.0544	
7	21	0.8827	47.2988	
7	22	0.8827	58.6635	
7	23	0.8827	82.1675	
7	24	0.8827	74.5074	
7	25	0.8827	59.1761	
7	26	0.8827	60.1756	
7	27	0.8827	81.7394	
7	28	0.8827	66.9321	
7	29	0.8827	59.101	
7	30	0.8827	70.6372	

Cuadro 6.3: Datos del archivo `w_evol_nomut_nospeedlimit.csv`

Run	Iteración	Costo/Fitness	TiempoIter
1	0	0.8275360060671322	42.22018575668335
1	1	0.8282123432282577	5.541130065917969
1	2	0.8763623948134542	14.36892032623291
1	3	0.8763623948134542	4.5662925243377686
1	4	0.8763623948134542	0.5898022651672363
1	5	0.8763623948134542	10.279666423797607
1	6	0.8763623948134542	0.5593276023864746
1	7	0.8763623948134542	10.699994087219238
1	8	0.8763623948134542	1.0453853607177734
1	9	0.8763623948134542	10.634629249572754
1	10	0.8763623948134542	0.6493310928344727
1	11	0.8763623948134542	10.458587884902954
1	12	0.8763623948134542	0.5929393768310547
1	13	0.8763623948134542	9.79326868057251
1	14	0.8763623948134542	0.6038241386413574
1	15	0.8763623948134542	9.440687894821167
1	16	0.8763623948134542	0.6179931163787842
1	17	0.8763623948134542	12.441182851791382
1	18	0.8763623948134542	0.6058955192565918
1	19	0.8763623948134542	12.198275089263916
1	20	0.8763623948134542	0.6944756507873535

Run	Iteración	Costo/Fitness	TiempoIter
1	21	0.8763623948134542	11.31234335899353
1	22	0.8763623948134542	0.6281871795654297
1	23	0.8763623948134542	11.360926866531372
1	24	0.8763623948134542	0.7276842594146729
1	25	0.8763623948134542	12.181276321411133
1	26	0.8763623948134542	0.6976668834686279
1	27	0.8763623948134542	9.990837335586548
1	28	0.8763623948134542	0.6643822193145752
1	29	0.8763623948134542	10.23689341545105
1	30	0.8763623948134542	0.6018507480621338
2	0	0.8324352985400504	0.6018507480621338
2	1	0.8324352985400504	14.22589898109436
2	2	0.8748992649859382	21.26158833503723
2	3	0.8748992649859382	10.999989032745361
2	4	0.8777751384498723	22.964287996292114
2	5	0.8777751384498723	6.087380647659302
2	6	0.8777751384498723	23.8044490814209
2	7	0.8777751384498723	6.080365180969238
2	8	0.8777751384498723	24.497936725616455
2	9	0.8777751384498723	6.063138961791992
2	10	0.8777751384498723	25.364294290542603
2	11	0.8777751384498723	6.555506229400635
2	12	0.8777751384498723	28.344622373580933
2	13	0.8777751384498723	6.715804576873779
2	14	0.8777751384498723	29.316497325897217
2	15	0.8777751384498723	7.433908462524414
2	16	0.8777751384498723	30.926580667495728
2	17	0.8777751384498723	8.18977403640747
2	18	0.8777751384498723	33.42683959007263
2	19	0.8777751384498723	8.484198570251465
2	20	0.8777751384498723	37.52506709098816
2	21	0.8777751384498723	10.38749361038208
2	22	0.8777751384498723	43.5918242931366
2	23	0.8777751384498723	10.472052335739136
2	24	0.8777751384498723	39.338311433792114
2	25	0.8777751384498723	9.582179307937622
2	26	0.8777751384498723	35.42523527145386
2	27	0.8777751384498723	8.057632446289062
2	28	0.8777751384498723	31.207144260406494
2	29	0.8777751384498723	7.64713978767395
2	30	0.8777751384498723	32.0378053188324
3	0	0.8394448020623356	32.0378053188324
3	1	0.8394448020623356	30.378413200378418
3	2	0.8757581001739307	101.18043112754822
3	3	0.8757581001739307	61.11742639541626

Run	Iteración	Costo/Fitness	TiempoIter
3	4	0.8757581001739307	23.36518955230713
3	5	0.8757581001739307	74.08494734764099
3	6	0.8757581001739307	28.01695942878723
3	7	0.8757581001739307	88.22590899467468
3	8	0.8757581001739307	37.50476789474487
3	9	0.8757581001739307	91.91916036605835
3	10	0.8757581001739307	33.05132007598877
3	11	0.8757581001739307	85.67639803886414
3	12	0.8757581001739307	32.25181818008423
3	13	0.8757581001739307	89.1349458694458
3	14	0.8757581001739307	33.121193170547485
3	15	0.8757581001739307	87.18852210044861
3	16	0.8757581001739307	32.08970260620117
3	17	0.8757581001739307	87.51269149780273
3	18	0.8757581001739307	35.80463433265686
3	19	0.8757581001739307	85.9949357509613
3	20	0.8757581001739307	32.39608311653137
3	21	0.8757581001739307	90.1451940536499
3	22	0.8757581001739307	32.118282079696655
3	23	0.8757581001739307	85.46888089179993
3	24	0.8757581001739307	31.648415327072144
3	25	0.8757581001739307	86.04219365119934
3	26	0.8757581001739307	31.652469396591187
3	27	0.8757581001739307	85.08001708984375
3	28	0.8757581001739307	33.74588370323181
3	29	0.8757581001739307	93.37684488296509
3	30	0.8757581001739307	31.794617414474487
4	0	0.8538627685777762	31.794617414474487
4	1	0.8538627685777762	36.23074746131897
4	2	0.8691918267551084	87.64770579338074
4	3	0.8691918267551084	23.098876953125
4	4	0.8691918267551084	58.16057801246643
4	5	0.8691918267551084	13.272279977798462
4	6	0.8691918267551084	68.03543710708618
4	7	0.8691918267551084	15.013749837875366
4	8	0.8691918267551084	63.90864109992981
4	9	0.8691918267551084	12.841383695602417
4	10	0.8691918267551084	61.266603231430054
4	11	0.8691918267551084	13.10023808479309
4	12	0.8691918267551084	61.0518593788147
4	13	0.8691918267551084	12.987066268920898
4	14	0.8691918267551084	61.494340896606445
4	15	0.8691918267551084	12.907413959503174
4	16	0.8691918267551084	60.97900700569153
4	17	0.8691918267551084	12.781996726989746

Run	Iteración	Costo/Fitness	TiempoIter
4	18	0.8691918267551084	64.43903613090515
4	19	0.8691918267551084	16.21690583229065
4	20	0.8691918267551084	73.81978559494019
4	21	0.8691918267551084	14.769989490509033
4	22	0.8691918267551084	64.49785709381104
4	23	0.8691918267551084	12.847094535827637
4	24	0.8691918267551084	61.9381742477417
4	25	0.8691918267551084	12.981039762496948
4	26	0.8691918267551084	61.20737934112549
4	27	0.8691918267551084	13.316846370697021
4	28	0.8691918267551084	61.68372702598572
4	29	0.8691918267551084	13.148306131362915
4	30	0.8691918267551084	61.41005778312683
5	0	0.8466119577930492	61.41005778312683
5	1	0.847916047605486	72.80234050750732
5	2	0.8757014484438261	171.70567202568054
5	3	0.8757014484438261	39.02820372581482
5	4	0.8757014484438261	251.50267815589905
5	5	0.8757014484438261	2.918398141860962
5	6	0.8757014484438261	239.2975676059723
5	7	0.8757014484438261	2.637321710586548
5	8	0.8757014484438261	235.59036445617676
5	9	0.8757014484438261	2.655172824859619
5	10	0.8757014484438261	237.2143359184265
5	11	0.8757014484438261	2.7687761783599854
5	12	0.8757014484438261	237.57294988632202
5	13	0.8757014484438261	2.668231725692749
5	14	0.8757014484438261	157.0790412425995
5	15	0.8757014484438261	1.3634917736053467
5	16	0.8762252274253278	113.01089286804199
5	17	0.8762252274253278	1.4454963207244873
5	18	0.8762252274253278	126.57379579544067
5	19	0.8762252274253278	1.6641619205474854
5	20	0.8762252274253278	162.20394468307495
5	21	0.8762252274253278	3.047508478164673
5	22	0.8762252274253278	226.7568497657776
5	23	0.8762252274253278	1.9998669624328613
5	24	0.8762252274253278	166.43065881729126
5	25	0.8762252274253278	2.0113961696624756
5	26	0.8762252274253278	219.60329127311707
5	27	0.8762252274253278	2.7843234539031982
5	28	0.8762252274253278	235.984126329422
5	29	0.8762252274253278	3.0398142337799072
5	30	0.8762252274253278	249.49792981147766
6	0	0.8334650277354225	249.49792981147766

Run	Iteración	Costo/Fitness	TiempoIter
6	1	0.8334650277354225	51.94576573371887
6	2	0.8618925180993905	118.12940573692322
6	3	0.8743707363048021	85.82509303092957
6	4	0.8743707363048021	64.45619344711304
6	5	0.8743707363048021	89.10531425476074
6	6	0.8747683776116835	64.9524974822998
6	7	0.8747683776116835	90.5434980392456
6	8	0.8747683776116835	64.32373714447021
6	9	0.8754858935328693	89.23510432243347
6	10	0.8761122754631068	65.17541122436523
6	11	0.8761122754631068	96.40360426902771
6	12	0.8761122754631068	65.7185001373291
6	13	0.8761122754631068	92.95095181465149
6	14	0.8761122754631068	65.4908504486084
6	15	0.8761122754631068	89.77911186218262
6	16	0.8772823956776128	66.4087393283844
6	17	0.8772823956776128	89.42851614952087
6	18	0.8772823956776128	68.0529932975769
6	19	0.8772823956776128	97.6088650226593
6	20	0.8772823956776128	64.52768301963806
6	21	0.8772823956776128	91.4992527961731
6	22	0.8772823956776128	70.10112738609314
6	23	0.8772823956776128	97.80147457122803
6	24	0.8772823956776128	64.23042011260986
6	25	0.8772823956776128	88.9346981048584
6	26	0.8772823956776128	64.57086277008057
6	27	0.8772823956776128	88.51829504966736
6	28	0.8772823956776128	64.81973958015442
6	29	0.8772823956776128	90.558034658432
6	30	0.8772823956776128	63.86492943763733
7	0	0.836270232484351	63.86492943763733
7	1	0.8364182266727086	44.33758997917175
7	2	0.8785711336150486	10.023351907730103
7	3	0.8785711336150486	70.21984100341797
7	4	0.8785711336150486	42.59156274795532
7	5	0.8785711336150486	74.33449196815491
7	6	0.8785711336150486	41.26298809051514
7	7	0.8785711336150486	69.11036252975464
7	8	0.8785711336150486	41.32251191139221
7	9	0.8785711336150486	69.95498776435852
7	10	0.8785711336150486	43.028889179229736
7	11	0.8785711336150486	70.26866459846497
7	12	0.8785711336150486	41.41081142425537
7	13	0.8785711336150486	69.21673893928528
7	14	0.8785711336150486	41.195297956466675

Run	Iteración	Costo/Fitness	TiempoIter
7	15	0.8785711336150486	69.71143198013306
7	16	0.8785711336150486	42.11408996582031
7	17	0.8785711336150486	69.31621837615967
7	18	0.8785711336150486	41.503812074661255
7	19	0.8785711336150486	75.12260150909424
7	20	0.8785711336150486	48.505046367645264
7	21	0.8785711336150486	75.07052159309387
7	22	0.8785711336150486	47.647082805633545
7	23	0.8785711336150486	70.8819887638092
7	24	0.8785711336150486	45.65255355834961
7	25	0.8785711336150486	72.81174778938293
7	26	0.8785711336150486	41.13087010383606
7	27	0.8785711336150486	69.5377197265625
7	28	0.8785711336150486	41.13065218925476
7	29	0.8785711336150486	69.6032862663269
7	30	0.8785711336150486	41.37213754653931

Cuadro 6.4: Datos del archivo `w_evol_nomut_speedlimit.csv`

Run	Iteración	Costo/Fitness	TiempoIter
1	0	0.8245859408735752	2.090425729751587
1	1	0.8245859408735752	2.545794725418091
1	2	0.8517762936343786	7.600373983383179
1	3	0.8517762936343786	5.130711555480957
1	4	0.8517762936343786	6.800297498703003
1	5	0.8517762936343786	6.387768507003784
1	6	0.8521308304570955	6.58357048034668
1	7	0.8521308304570955	16.128064155578613
1	8	0.8521308304570955	13.594764947891235
1	9	0.8521308304570955	11.658144235610962
1	10	0.8521308304570955	13.965043783187866
1	11	0.8529089852608118	17.613783597946167
1	12	0.8529089852608118	17.97335934638977
1	13	0.8529089852608118	18.19292116165161
1	14	0.8530268535441762	18.582812547683716
1	15	0.8535100293936004	25.463227033615112
1	16	0.8535100293936004	24.25960636138916
1	17	0.8535100293936004	22.66602897644043
1	18	0.8535100293936004	22.502595901489258
1	19	0.8535100293936004	20.040184259414673
1	20	0.8535100293936004	20.53316354751587
1	21	0.8535100293936004	21.89090919494629
1	22	0.8535100293936004	23.477795600891113
1	23	0.8535100293936004	23.16916060447693

Run	Iteración	Costo/Fitness	TiempoIter
1	24	0.8535100293936004	22.94689702987671
1	25	0.8535100293936004	23.005090713500977
1	26	0.8535100293936004	22.957441091537476
1	27	0.8535100293936004	23.689898014068604
1	28	0.8535912639783481	23.463855504989624
1	29	0.8535912639783481	24.37518048286438
1	30	0.8535912639783481	22.747451543807983
2	0	0.8387038791907877	14.999099254608154
2	1	0.8387038791907877	18.135718822479248
2	2	0.8740548317524709	46.513808250427246
2	3	0.8740548317524709	23.35088562965393
2	4	0.8740548317524709	4.2890894412994385
2	5	0.8747154782682106	27.458579778671265
2	6	0.8747154782682106	36.30686044692993
2	7	0.8764403921592431	53.344852685928345
2	8	0.8764403921592431	53.29982662200928
2	9	0.8764403921592431	50.22508645057678
2	10	0.8764403921592431	54.747384786605835
2	11	0.8764403921592431	52.96277856826782
2	12	0.8764403921592431	52.693135261535645
2	13	0.8764403921592431	60.53748440742493
2	14	0.8764403921592431	60.87141942977905
2	15	0.8764403921592431	60.83897948265076
2	16	0.8764403921592431	54.32362508773804
2	17	0.8764403921592431	59.56345510482788
2	18	0.8764403921592431	70.05283641815186
2	19	0.8764403921592431	64.78808879852295
2	20	0.8764403921592431	54.997766971588135
2	21	0.8764403921592431	66.97587704658508
2	22	0.8764403921592431	66.4182698726654
2	23	0.8764403921592431	56.01342225074768
2	24	0.8764403921592431	61.327839374542236
2	25	0.8764403921592431	66.684166431427
2	26	0.8764403921592431	63.82017803192139
2	27	0.8767707299990988	59.09607481956482
2	28	0.8767707299990988	65.58786082267761
2	29	0.8767707299990988	70.4912211894989
2	30	0.8767707299990988	68.9190845489502
3	0	0.8246554210521502	13.976362705230713
3	1	0.8302265943136211	13.712505340576172
3	2	0.8533136891113713	24.086966037750244
3	3	0.8793213587600048	21.41502547264099
3	4	0.8793213587600048	10.247036695480347
3	5	0.8793213587600048	23.29821801185608
3	6	0.8793213587600048	28.91284418106079

Run	Iteración	Costo/Fitness	TiempoIter
3	7	0.88093188660153	41.54310584068298
3	8	0.8823343488183102	56.243584394454956
3	9	0.8823343488183102	28.34804677963257
3	10	0.8823343488183102	54.11873531341553
3	11	0.8823343488183102	47.677605628967285
3	12	0.8823343488183102	27.07407784461975
3	13	0.8823343488183102	69.27409720420837
3	14	0.8823343488183102	70.74822521209717
3	15	0.8823343488183102	1.7229628562927246
3	16	0.88253197381256	55.114365339279175
3	17	0.88253197381256	87.05161952972412
3	18	0.8834446755953904	51.288352251052856
3	19	0.8834446755953904	31.112648963928223
3	20	0.8834446755953904	63.39194679260254
3	21	0.8834446755953904	63.87291359901428
3	22	0.8834446755953904	54.53266906738281
3	23	0.8834446755953904	46.04853057861328
3	24	0.8834446755953904	94.75475025177002
3	25	0.8834446755953904	53.04498028755188
3	26	0.8834446755953904	60.643253803253174
3	27	0.8834446755953904	63.63438582420349
3	28	0.8834446755953904	65.47838068008423
3	29	0.8834446755953904	42.59821105003357
3	30	0.8834446755953904	40.19867467880249
4	0	0.8493482856100153	17.108388423919678
4	1	0.8499407741229525	17.55167508125305
4	2	0.8787810109091788	18.129011869430542
4	3	0.8787810109091788	31.844319820404053
4	4	0.8787810109091788	25.16534662246704
4	5	0.8787810109091788	15.95907187461853
4	6	0.8787810109091788	52.5474374294281
4	7	0.8787810109091788	54.58143138885498
4	8	0.8787810109091788	54.1643009185791
4	9	0.8787810109091788	54.293142318725586
4	10	0.8805327986174681	73.969078540802
4	11	0.881578734469902	59.17855763435364
4	12	0.881578734469902	67.20896339416504
4	13	0.881578734469902	48.09193396568298
4	14	0.881578734469902	32.8657341003418
4	15	0.881578734469902	68.52475953102112
4	16	0.881578734469902	58.66793632507324
4	17	0.881578734469902	43.4172089099884
4	18	0.881578734469902	59.698578119277954
4	19	0.881578734469902	43.914180278778076
4	20	0.881578734469902	42.12810802459717

Run	Iteración	Costo/Fitness	TiempoIter
4	21	0.881578734469902	35.714767932891846
4	22	0.8817397452543654	34.69435214996338
4	23	0.8817397452543654	38.92806577682495
4	24	0.8817397452543654	75.69061088562012
4	25	0.8817397452543654	52.45639634132385
4	26	0.8817397452543654	42.17553758621216
4	27	0.8817397452543654	66.45642685890198
4	28	0.8817397452543654	58.8422794342041
4	29	0.8817397452543654	39.7245991230011
4	30	0.8817397452543654	64.25132441520691
5	0	0.8301600961087449	25.139694213867188
5	1	0.8301600961087449	22.93292808532715
5	2	0.8576861106267597	13.022144556045532
5	3	0.8576861106267597	29.876628637313843
5	4	0.8680876579967052	19.59432864189148
5	5	0.8763167885221739	31.58596682548523
5	6	0.8798281981862367	74.11887311935425
5	7	0.8798281981862367	45.17434883117676
5	8	0.8798281981862367	47.857184648513794
5	9	0.8798281981862367	90.90072989463806
5	10	0.8798281981862367	79.28059220314026
5	11	0.8798281981862367	51.82546353340149
5	12	0.8798281981862367	68.3389208316803
5	13	0.8798281981862367	86.55403661727905
5	14	0.8798281981862367	76.21075534820557
5	15	0.8805244291831678	67.81999063491821
5	16	0.8805244291831678	106.2741870880127
5	17	0.882358909068083	159.7520558834076
5	18	0.882358909068083	106.74019026756287
5	19	0.882358909068083	128.39026498794556
5	20	0.8829724620317105	183.21098804473877
5	21	0.8829724620317105	144.73588109016418
5	22	0.8829724620317105	158.72169876098633
5	23	0.8829724620317105	156.00757431983948
5	24	0.8829724620317105	172.31859850883484
5	25	0.8829724620317105	159.4198353290558
5	26	0.8829724620317105	166.51642036437988
5	27	0.8829724620317105	159.22528100013733
5	28	0.8829724620317105	204.29582262039185
5	29	0.8829724620317105	227.84450030326843
5	30	0.8829724620317105	216.7345790863037
6	0	0.847332988425587	45.71681761741638
6	1	0.849736446727734	48.11450672149658
6	2	0.876175316120946	84.10066318511963
6	3	0.876175316120946	56.47775340080261

Run	Iteración	Costo/Fitness	TiempoIter
6	4	0.876175316120946	21.593621730804443
6	5	0.876175316120946	82.14498686790466
6	6	0.8769003730595367	58.985007524490356
6	7	0.8807343448143763	60.506614446640015
6	8	0.8807343448143763	56.425095319747925
6	9	0.8807343448143763	129.99676871299744
6	10	0.8807343448143763	90.22449779510498
6	11	0.8807343448143763	188.423073768616
6	12	0.8812603264235749	115.70341801643372
6	13	0.8812603264235749	123.12127947807312
6	14	0.8812603264235749	129.2516860961914
6	15	0.8812603264235749	99.95057964324951
6	16	0.8812603264235749	138.26157236099243
6	17	0.8812603264235749	114.28505325317383
6	18	0.8812603264235749	126.7608630657196
6	19	0.8813104664924856	137.88607382774353
6	20	0.8813104664924856	34.90640115737915
6	21	0.8813104664924856	125.12448716163635
6	22	0.8813104664924856	160.73576378822327
6	23	0.8816716862497843	95.34022998809814
6	24	0.8822305580370786	166.74206924438477
6	25	0.8822305580370786	200.02930760383606
6	26	0.8822305580370786	145.33199095726013
6	27	0.8822305580370786	209.43302536010742
6	28	0.8822305580370786	183.26618766784668
6	29	0.8822305580370786	178.26417875289917
6	30	0.884121227315416	127.68113493919373
7	0	0.8303432171649723	25.05295181274414
7	1	0.8303432171649723	25.2770574092865
7	2	0.878958730625443	64.87925839424133
7	3	0.878958730625443	39.72606682777405
7	4	0.878958730625443	28.73063015937805
7	5	0.878958730625443	70.76614761352539
7	6	0.8790610761264578	63.754722118377686
7	7	0.8790610761264578	89.8459119796753
7	8	0.8813413618249131	144.90236473083496
7	9	0.8813413618249131	135.92153143882751
7	10	0.8822959395511877	170.63057780265808
7	11	0.8822959395511877	200.4325029850006
7	12	0.8822959395511877	196.2502715587616
7	13	0.8822959395511877	200.32373142242432
7	14	0.8822959395511877	205.37205839157104
7	15	0.8822959395511877	263.4995427131653
7	16	0.8826666584879097	177.78765606880188
7	17	0.8830330398950208	190.7220287322998

Run	Iteración	Costo/Fitness	TiempoIter
7	18	0.8833697582139486	202.10478496551514
7	19	0.8833697582139486	184.49653601646423
7	20	0.8833697582139486	201.3509521484375
7	21	0.8833697582139486	250.299485206604
7	22	0.8833697582139486	274.29330587387085
7	23	0.8833697582139486	265.2995619773865
7	24	0.8833697582139486	215.3968563079834
7	25	0.8833697582139486	259.2621352672577
7	26	0.8833697582139486	230.0550193786621
7	27	0.8833697582139486	244.5940191745758
7	28	0.8833697582139486	217.18027067184448
7	29	0.8833697582139486	238.02133345603943
7	30	0.8833697582139486	244.42544960975647

Cuadro 6.5: Datos del archivo `w9_mut_nospeedlimit.csv`

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
1	0	0.824512	7.861	
1	1	0.827945	6.3166	
1	2	0.827945	6.0836	
1	3	0.827945	5.0616	
1	4	0.827945	4.3648	
1	5	0.827945	2.7721	
1	6	0.828679	1.5208	
1	7	0.828679	8.083	
1	8	0.828679	1.5142	
1	9	0.829918	2.6702	
1	10	0.831425	3.6477	
1	11	0.831425	5.3109	
1	12	0.831425	5.0197	
1	13	0.831425	4.1354	
1	14	0.831425	2.8239	
1	15	0.831425	1.7006	
1	16	0.831425	1.181	
1	17	0.831425	1.2029	
1	18	0.831425	1.9983	
1	19	0.831425	2.7982	
1	20	0.831425	3.6022	
1	21	0.831425	3.6533	
1	22	0.845730	3.8353	1
1	23	0.845730	2.7849	
1	24	0.855925	2.3377	
1	25	0.857069	1.3056	
1	26	0.862588	1.8844	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
1	27	0.862588	1.572	
1	28	0.862588	2.7956	
1	29	0.862588	2.7587	
1	30	0.867575	2.3296	
2	0	0.877817	2.329	
2	1	0.877817	166.445	
2	2	0.879626	272.112	
2	3	0.879626	274.249	
2	4	0.879626	212.363	
2	5	0.879626	206.615	
2	6	0.879626	23.288	
2	7	0.880327	282.156	
2	8	0.880327	291.534	
2	9	0.880327	309.739	
2	10	0.880327	171.954	1
2	11	0.880327	153.705	
2	12	0.880327	252.609	
2	13	0.880327	276.483	
2	14	0.880327	287.928	
2	15	0.880327	25.651	
2	16	0.880327	32.892	
2	17	0.880327	26.835	
2	18	0.880327	240.518	
2	19	0.880327	22.703	
2	20	0.880327	221.949	
2	21	0.880327	35.285	
2	22	0.880327	43.494	
2	23	0.880327	461.959	
2	25	0.880718	282.707	
2	26	0.880718	21.853	
2	27	0.880718	205.613	
2	28	0.880718	18.592	
2	29	0.880718	322.515	1
2	30	0.880718	246.365	
3	0	0.839590	24.636	
3	1	0.848645	18.027	
3	2	0.858634	222.618	
3	3	0.880686	245.686	
3	4	0.880686	252.706	
3	5	0.880686	201.537	
3	6	0.880686	183.439	
3	7	0.880686	276.468	
3	8	0.880686	30.942	
3	9	0.882178	28.435	
3	10	0.882178	318.425	2

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
3	11	0.882178	391.884	
3	12	0.882178	356.977	
3	13	0.882178	334.557	
3	14	0.882178	255.049	
3	15	0.882178	280.559	
3	16	0.882178	338.856	
3	17	0.882178	295.288	
3	18	0.882178	215.483	
3	19	0.882178	242.738	
3	20	0.882178	337.916	
3	21	0.882178	328.493	
3	22	0.882178	357.348	
3	23	0.882178	262.954	
3	25	0.882407	257.464	
3	26	0.882407	297.801	
3	27	0.882407	279.237	
3	28	0.882407	288.549	
3	29	0.882407	340.812	2
3	30	0.882407	267.432	
4	0	0.849288	267.43157	
4	1	0.849288	151.744	
4	2	0.849288	194.036	
4	3	0.875392	187.395	
4	4	0.875709	131.033	
4	5	0.876739	12.7075	
4	6	0.876739	11.7851	
4	7	0.878308	232.838	
4	8	0.878308	392.388	
4	9	0.879422	414.197	
4	10	0.879422	384.747	
4	11	0.879422	299.973	
4	12	0.879422	310.932	
4	13	0.880883	289.629	
4	14	0.880883	26.928	
4	15	0.880883	265.958	
4	16	0.880883	460.629	2
4	17	0.880883	384.942	
4	18	0.880883	430.416	
4	19	0.880883	489.554	
4	20	0.880883	358.808	
4	22	0.880883	336.685	
4	23	0.880883	398.396	
4	24	0.880883	363.846	
4	25	0.880883	480.325	
4	26	0.880883	352.731	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
4	27	0.880883	432.452	
4	28	0.880883	361.349	
4	29	0.880883	37.968	
4	30	0.880883	354.418	2
5	0	0.826111	354.4175	
5	1	0.826424	164.138	
5	2	0.827139	184.223	
5	3	0.827334	208.602	
5	4	0.827334	242.165	
5	5	0.828952	271.082	
5	6	0.828952	29.749	
5	7	0.828985	293.131	
5	8	0.865312	396.877	
5	9	0.871808	452.584	
5	10	0.878627	644.516	3
5	11	0.879399	582.321	
5	12	0.879399	462.893	
5	13	0.879399	401.862	
5	14	0.879399	491.898	
5	15	0.879399	636.476	
5	16	0.879399	59.174	
5	17	0.879399	573.124	
5	18	0.879399	485.331	
5	19	0.879399	525.853	
5	20	0.880350	614.858	
5	21	0.880350	450.284	
5	22	0.880350	443.622	
5	23	0.880350	348.276	
5	24	0.880350	417.269	
5	25	0.880350	580.745	
5	26	0.880350	434.852	
5	27	0.880350	504.417	
5	28	0.880350	392.839	
5	29	0.880350	41.617	
5	30	0.880350	445.557	
6	0	0.851836	445.578	
6	1	0.853935	234.615	
6	2	0.879581	258.572	
6	3	0.880357	260.877	
6	4	0.880357	293.099	
6	5	0.880357	185.558	
6	6	0.880357	135.243	
6	7	0.880357	233.061	
6	8	0.882997	304.284	
6	9	0.882997	306.008	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
6	10	0.882997	375.808	
6	11	0.882997	49.759	
6	12	0.882997	407.355	
6	13	0.882997	391.107	
6	14	0.882997	405.282	3
6	15	0.882997	293.825	
6	16	0.882997	317.792	
6	17	0.882997	378.933	
6	18	0.882997	389.425	
6	19	0.882997	441.058	
6	20	0.882997	427.213	
6	21	0.882997	435.448	
6	22	0.882997	411.248	
6	23	0.883743	460.113	
6	24	0.883743	403.855	3
6	25	0.883743	276.313	
6	26	0.883743	384.654	
6	27	0.883743	44.778	
6	28	0.883743	490.068	
6	29	0.883743	554.073	
6	30	0.883743	532.752	
7	0	0.870612	532.751	
7	1	0.871833	253.452	
7	2	0.871833	311.111	
7	3	0.873963	372.388	
7	4	0.873963	330.644	
7	5	0.873963	270.656	
7	6	0.873963	274.535	
7	7	0.874195	282.818	
7	8	0.874195	301.885	
7	9	0.874195	307.277	
7	10	0.874195	274.857	
7	11	0.874195	335.012	
7	12	0.874195	397.679	
7	13	0.874195	593.288	4
7	14	0.874195	365.717	
7	15	0.874195	340.483	
7	16	0.874195	325.179	
7	17	0.874195	359.472	
7	18	0.877350	52.482	4
7	19	0.877350	485.115	
7	20	0.877580	641.944	
7	21	0.877952	682.653	
7	22	0.881748	538.326	
7	23	0.881748	477.986	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
7	24	0.882177	454.732	
7	25	0.882177	419.006	
7	26	0.882177	481.572	
7	27	0.882177	48.181	4
7	28	0.882369	382.859	
7	29	0.883276	37.927	
7	30	0.883276	413.186	

Cuadro 6.6: Datos del archivo `w9_mut_speedlimit.csv`

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
1	0	0.825881	4.1318	
1	1	0.827533	3.2016	
1	2	0.827533	4.0425	
1	3	0.832692	5.6072	
1	4	0.875979	8.7808	
1	5	0.875979	10.1393	
1	6	0.875979	140.336	
1	7	0.875979	8.6663	
1	8	0.877959	12.0579	
1	9	0.877959	9.1539	
1	10	0.877959	7.7933	
1	11	0.877959	6.6041	
1	12	0.877959	5.5178	
1	13	0.878718	7.7452	
1	14	0.878718	8.4433	
1	15	0.878718	7.9166	
1	16	0.878718	9.3315	
1	17	0.878718	9.2946	
1	18	0.878718	7.2534	
1	19	0.878718	8.0968	
1	20	0.878718	7.0807	
1	21	0.878718	6.5187	
1	22	0.878718	6.4564	
1	23	0.878718	8.3628	
1	24	0.878718	9.0714	
1	25	0.878718	9.8108	
1	26	0.878718	10.3470	
1	27	0.878718	9.5869	
1	28	0.880127	8.9494	
1	29	0.880127	7.6612	
1	30	0.880127	7.2015	
2	0	0.828843	5.4525	
2	1	0.831816	8.6130	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
2	2	0.842522	7.7108	
2	3	0.873516	6.1494	
2	4	0.873516	5.2843	
2	5	0.873516	5.5681	
2	6	0.873516	6.0431	
2	7	0.873516	6.8120	
2	8	0.873516	7.6582	
2	9	0.873516	7.5373	
2	10	0.875576	5.5913	
2	11	0.875576	6.1146	
2	12	0.875576	6.0207	
2	13	0.875576	6.5452	
2	14	0.875576	6.1957	
2	15	0.876985	5.9443	
2	16	0.876985	5.9024	
2	17	0.876985	6.6488	
2	18	0.878052	6.2807	
2	19	0.878052	6.9867	
2	20	0.878052	6.7913	
2	21	0.878052	7.3231	
2	22	0.878052	8.8316	
2	23	0.878052	7.6613	
2	24	0.878052	6.5238	
2	25	0.878052	4.7375	
2	26	0.878052	5.3814	
2	27	0.878052	6.7250	
2	28	0.880194	10.7595	
2	29	0.880194	12.4183	
2	30	0.880194	11.6634	
3	0	0.863072	11.663	
3	1	0.863072	7.3195	
3	2	0.867031	5.7675	
3	3	0.868312	3.3668	
3	4	0.868312	2.2845	
3	5	0.868312	3.6772	
3	6	0.868312	5.3711	
3	7	0.868312	4.8967	
3	8	0.870031	4.4741	
3	9	0.870031	3.6626	
3	10	0.871422	3.9158	
3	11	0.871422	5.4703	
3	12	0.871422	6.5594	
3	13	0.871422	8.2697	
3	14	0.871422	9.2298	2
3	15	0.872010	7.1067	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
3	16	0.872010	6.7044	
3	17	0.872010	5.8725	
3	18	0.873030	8.1613	
3	19	0.876119	164.849	
3	20	0.876119	10.8731	
3	21	0.876119	10.0618	
3	22	0.876119	7.9083	
3	23	0.876119	7.0869	
3	24	0.876119	8.1561	
3	25	0.876119	9.1122	
3	26	0.876119	11.2581	
3	27	0.876119	153.730	
3	28	0.876119	11.1387	
3	30	0.876119	7.7866	
4	0	0.871665	178.656	
4	1	0.873441	197.483	
4	2	0.874843	276.253	
4	3	0.876200	363.984	
4	4	0.876200	359.012	
4	5	0.876200	291.051	
4	6	0.876200	312.261	
4	7	0.876200	331.919	
4	8	0.876629	324.819	
4	9	0.876629	344.273	
4	10	0.876629	319.946	
4	11	0.877315	366.962	
4	12	0.877315	345.508	
4	13	0.877315	379.816	
4	14	0.878624	376.741	
4	15	0.878686	396.766	
4	16	0.878686	383.234	
4	17	0.878686	419.527	
4	18	0.878686	405.293	2
4	19	0.878686	457.306	
4	20	0.878686	326.880	
4	21	0.878686	368.560	
4	22	0.878686	376.022	
4	23	0.878686	384.443	
4	24	0.879072	448.451	
4	25	0.879072	391.259	
4	26	0.879079	431.122	
4	27	0.880038	467.614	
4	28	0.880038	467.261	2
4	29	0.880038	494.192	
4	30	0.880038	431.741	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
5	0	0.857666	431.741	
5	1	0.858313	266.721	
5	2	0.862918	428.397	
5	3	0.864301	575.112	
5	4	0.864301	579.818	
5	5	0.864670	560.580	
5	6	0.864670	561.820	
5	7	0.867536	639.454	
5	8	0.873194	459.849	
5	9	0.873194	316.292	
5	10	0.873194	263.433	
5	11	0.876231	245.891	
5	12	0.876231	250.996	
5	13	0.876231	280.577	
5	14	0.876231	295.204	
5	15	0.876231	296.645	
5	16	0.876231	275.798	3
5	17	0.878480	576.721	
5	18	0.878480	475.273	
5	19	0.879255	631.040	
5	20	0.879255	610.683	
5	21	0.879255	612.706	3
5	22	0.879255	783.862	
5	23	0.879255	538.814	
5	24	0.879920	582.618	
5	25	0.879920	528.747	
5	26	0.880526	516.330	
5	27	0.880526	453.305	
5	28	0.881894	528.980	
5	29	0.881894	432.765	
5	30	0.881894	368.830	
6	0	0.849335	368.829	
6	1	0.849713	324.985	
6	2	0.871447	436.872	
6	3	0.871793	648.137	
6	4	0.872410	639.119	
6	5	0.872410	434.909	
6	6	0.872410	416.375	
6	7	0.872410	468.771	
6	8	0.874595	651.546	
6	9	0.874595	577.646	
6	10	0.875708	488.895	
6	11	0.875708	471.132	
6	12	0.875708	539.533	
6	13	0.877276	606.748	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
6	14	0.877276	610.508	
6	15	0.877276	554.285	
6	16	0.877276	502.435	
6	17	0.877276	582.564	
6	18	0.877276	637.424	
6	19	0.877276	563.271	
6	20	0.877276	590.605	
6	21	0.877276	573.472	
6	22	0.877276	624.155	
6	23	0.877276	588.227	3
6	24	0.877276	682.194	
6	25	0.877276	625.910	
6	26	0.877276	590.922	
6	27	0.877276	583.525	
6	28	0.877276	592.429	3
6	29	0.879355	752.298	
6	30	0.879355	684.847	
7	0	0.855859	68.484	
7	1	0.855859	335.076	
7	2	0.856578	485.797	
7	3	0.876926	826.563	
7	4	0.876926	1271.614	
7	5	0.876926	902.525	
7	6	0.876926	723.472	
7	7	0.876926	743.308	
7	8	0.877641	930.923	
7	9	0.878347	942.140	
7	10	0.879709	665.093	
7	11	0.879709	547.027	
7	12	0.879709	500.149	
7	13	0.881105	628.475	
7	14	0.881105	631.931	4
7	15	0.881105	811.096	
7	16	0.881105	593.437	
7	17	0.881105	605.191	
7	18	0.881105	670.770	
7	19	0.881105	578.692	
7	20	0.881105	516.371	
7	21	0.881105	543.042	
7	22	0.881105	687.261	
7	23	0.881105	704.864	
7	24	0.881105	647.085	4
7	25	0.881105	695.167	
7	26	0.881105	587.468	
7	27	0.882148	675.571	

Run	Iteración	Costo/Fitness	TiempoIter	NumPartMut
7	28	0.882148	658.281	
7	29	0.882148	517.534	4
7	30	0.882531	730.567	

Cuadro 6.7: Datos del archivo `w9_nomut_nospeedlimit.csv`

Run	Iteración	Costo/Fitness	TiempoIter
1	0	0.844326710572133	41.37213754653931
1	1	0.8481331414967961	10.05539846420288
1	2	0.8481724826617262	10.212698936462402
1	3	0.8708445526907489	8.590256452560425
1	4	0.8708445526907489	6.629370450973511
1	5	0.8708445526907489	8.425668716430664
1	6	0.8708445526907489	8.073707580566406
1	7	0.8708445526907489	6.809502840042114
1	8	0.8710430191271679	7.345610857009888
1	9	0.8710430191271679	6.569413661956787
1	10	0.8710430191271679	10.507915019989014
1	11	0.8720766711429091	10.646379947662354
1	12	0.8720766711429091	10.283257961273193
1	13	0.8720766711429091	12.22114610671997
1	14	0.8720766711429091	10.786816358566284
1	15	0.8724283248887381	9.104193687438965
1	16	0.8724283248887381	5.9587082862854
1	17	0.8724283248887381	6.2452802658081055
1	18	0.8724283248887381	7.031435489654541
1	19	0.8724283248887381	8.34565258026123
1	20	0.8740320518368424	11.881030559539795
1	21	0.8740320518368424	13.212804317474365
1	22	0.8740320518368424	14.138024091720581
1	23	0.8740320518368424	13.064999103546143
1	24	0.874173686089545	12.507817029953003
1	25	0.8744465838315538	10.433298587799072
1	26	0.8766933041395315	11.054393768310547
1	27	0.8766933041395315	9.732257843017578
1	28	0.8766933041395315	11.062316417694092
1	29	0.8766933041395315	12.193381071090698
1	30	0.8766933041395315	13.143821001052856
2	0	0.8379846570887945	13.143821001052856
2	1	0.8379846570887945	17.87377166748047
2	2	0.8379846570887945	29.43229579925537
2	3	0.8708477731145308	67.71758532524109
2	4	0.8708477731145308	71.00482106208801
2	5	0.8708477731145308	62.481565952301025

Run	Iteración	Costo/Fitness	TiempoIter
2	6	0.8708477731145308	59.03740954399109
2	7	0.8708477731145308	56.869017124176025
2	8	0.8708477731145308	61.301669120788574
2	9	0.8708477731145308	61.5732536315918
2	10	0.8708477731145308	47.11934781074524
2	11	0.8708477731145308	46.79470252990723
2	12	0.8708477731145308	53.17241072654724
2	13	0.8711467874782504	59.92454481124878
2	14	0.8711467874782504	65.33103656768799
2	15	0.8711467874782504	62.080111026763916
2	16	0.8711467874782504	52.97627091407776
2	17	0.8711467874782504	55.31888127326965
2	18	0.8711467874782504	61.54970574378967
2	19	0.8711467874782504	67.68930125236511
2	20	0.8711467874782504	53.46438789367676
2	21	0.8711467874782504	50.00210666656494
2	22	0.8711467874782504	56.91161870956421
2	23	0.8711467874782504	72.2042407989502
2	24	0.8711467874782504	59.516817808151245
2	25	0.8711467874782504	50.20915722846985
2	26	0.8711467874782504	50.181400537490845
2	27	0.8711467874782504	53.13013982772827
2	28	0.8738521851140335	68.86696195602417
2	29	0.8738521851140335	56.50146460533142
2	30	0.8738521851140335	26.10281777381897
3	0	0.8485384070680538	26.10281777381897
3	1	0.8486743281748387	22.129594326019287
3	2	0.8805924003537969	48.9615592956543
3	3	0.8806268339023697	81.31240129470825
3	4	0.8806268339023697	51.58256196975708
3	5	0.8806268339023697	38.580376863479614
3	6	0.8806268339023697	44.592626094818115
3	7	0.8831403443644102	88.19870948791504
3	8	0.8831403443644102	163.9176731109619
3	9	0.8831403443644102	97.52726340293884
3	10	0.8831403443644102	54.15530586242676
3	11	0.8831403443644102	64.04332423210144
3	12	0.8831403443644102	119.25057077407837
3	13	0.8831403443644102	150.58164286613464
3	14	0.8831403443644102	99.28097462654114
3	15	0.8831403443644102	84.98482894897461
3	16	0.8831403443644102	113.82338953018188
3	17	0.8831403443644102	154.38994216918945
3	18	0.8831403443644102	103.28482055664062
3	19	0.8831403443644102	104.6227388381958

Run	Iteración	Costo/Fitness	TiempoIter
3	20	0.8831403443644102	108.21200752258301
3	21	0.8831403443644102	161.9179048538208
3	22	0.8831403443644102	132.93714666366577
3	23	0.8831403443644102	117.09424877166748
3	24	0.8831403443644102	113.3999834060669
3	25	0.8831403443644102	142.53369665145874
3	26	0.8831403443644102	139.97808361053467
3	27	0.8831403443644102	107.72324705123901
3	28	0.8831403443644102	116.43120312690735
3	29	0.8831403443644102	151.490540266037
3	30	0.8831403443644102	173.2506444454193
4	0	0.8334742629883033	173.2506444454193
4	1	0.8341506133873902	39.502501010894775
4	2	0.8471662777619583	33.851507902145386
4	3	0.8632718982738531	36.37077713012695
4	4	0.8632718982738531	37.21962761878967
4	5	0.8658685136326938	34.44337224960327
4	6	0.8658685136326938	34.072158336639404
4	7	0.8662062935892783	37.573140144348145
4	8	0.8668879233685823	39.725138425827026
4	9	0.8668879233685823	41.23065423965454
4	10	0.8674320960867001	41.97542357444763
4	11	0.8702190238420195	39.40195441246033
4	12	0.8702190238420195	37.21184778213501
4	13	0.8702190238420195	36.4436821937561
4	14	0.8702190238420195	35.3975555896759
4	15	0.8702190238420195	39.6717746257782
4	16	0.8708936473803675	42.33515501022339
4	17	0.8708936473803675	44.645939111709595
4	18	0.8708936473803675	45.742677450180054
4	19	0.8708936473803675	43.78571271896362
4	20	0.8708936473803675	43.3942437171936
4	21	0.8708936473803675	44.45452427864075
4	22	0.8708936473803675	39.867303133010864
4	23	0.8708936473803675	38.43020534515381
4	24	0.8708936473803675	42.29229021072388
4	25	0.872740064918094	41.3921172618866
4	26	0.872740064918094	43.69087028503418
4	27	0.872740064918094	45.34081268310547
4	28	0.872740064918094	49.955387115478516
4	29	0.872740064918094	50.33408045768738
4	30	0.872740064918094	46.67501401901245
5	0	0.8276701993377913	46.67501401901245
5	1	0.8293651082877057	40.733602523804
5	2	0.8299001784266042	24.166261672973633

Run	Iteración	Costo/Fitness	TiempoIter
5	3	0.8299001784266042	15.361186742782593
5	4	0.843673015776415	12.279696702957153
5	5	0.8789587510202113	12.424315214157104
5	6	0.8789587510202113	8.232587337493896
5	7	0.8812369614284628	9.126033306121826
5	8	0.8812369614284628	12.165823459625244
5	9	0.8812369614284628	16.87856364250183
5	10	0.8812369614284628	19.929811716079712
5	11	0.8812369614284628	18.9193217754364
5	12	0.8812369614284628	14.497676610946655
5	13	0.8812369614284628	12.519175291061401
5	14	0.8812369614284628	11.95114517211914
5	15	0.8812369614284628	15.039881944656372
5	16	0.8812369614284628	20.52468752861023
5	17	0.8822713091858361	21.087368488311768
5	18	0.8822713091858361	22.319454669952393
5	19	0.8822713091858361	22.030002117156982
5	20	0.8822713091858361	14.88034701347351
5	21	0.8822713091858361	12.046628713607788
5	22	0.8822713091858361	9.777525186538696
5	23	0.8822713091858361	13.742020606994629
5	24	0.8822713091858361	16.602872610092163
5	25	0.8822713091858361	19.033374547958374
5	26	0.8822713091858361	20.087212324142456
5	27	0.8822713091858361	15.084148168563843
5	28	0.8825912850403886	15.02293348312378
5	29	0.8825912850403886	13.37075924873352
5	30	0.8825912850403886	13.136521577835083
6	0	0.8476866258529789	13.136521577835083
6	1	0.8476866258529789	35.233476400375366
6	2	0.8487101500281142	36.784470558166504
6	3	0.8699519070834111	52.779778480529785
6	4	0.8709544825583626	43.020670652389526
6	5	0.8709544825583626	32.61844277381897
6	6	0.8719622966643096	32.658588886260986
6	7	0.8724143174388793	47.616060733795166
6	8	0.8739452706039474	74.08663725852966
6	9	0.8739452706039474	52.83215832710266
6	10	0.8754842766167448	49.070942640304565
6	11	0.8754842766167448	47.47036695480347
6	12	0.8804162372405642	67.34139347076416
6	13	0.8804162372405642	57.71233153343201
6	14	0.8804162372405642	37.68859362602234
6	15	0.8804162372405642	37.43153405189514
6	16	0.8804162372405642	39.0477991104126

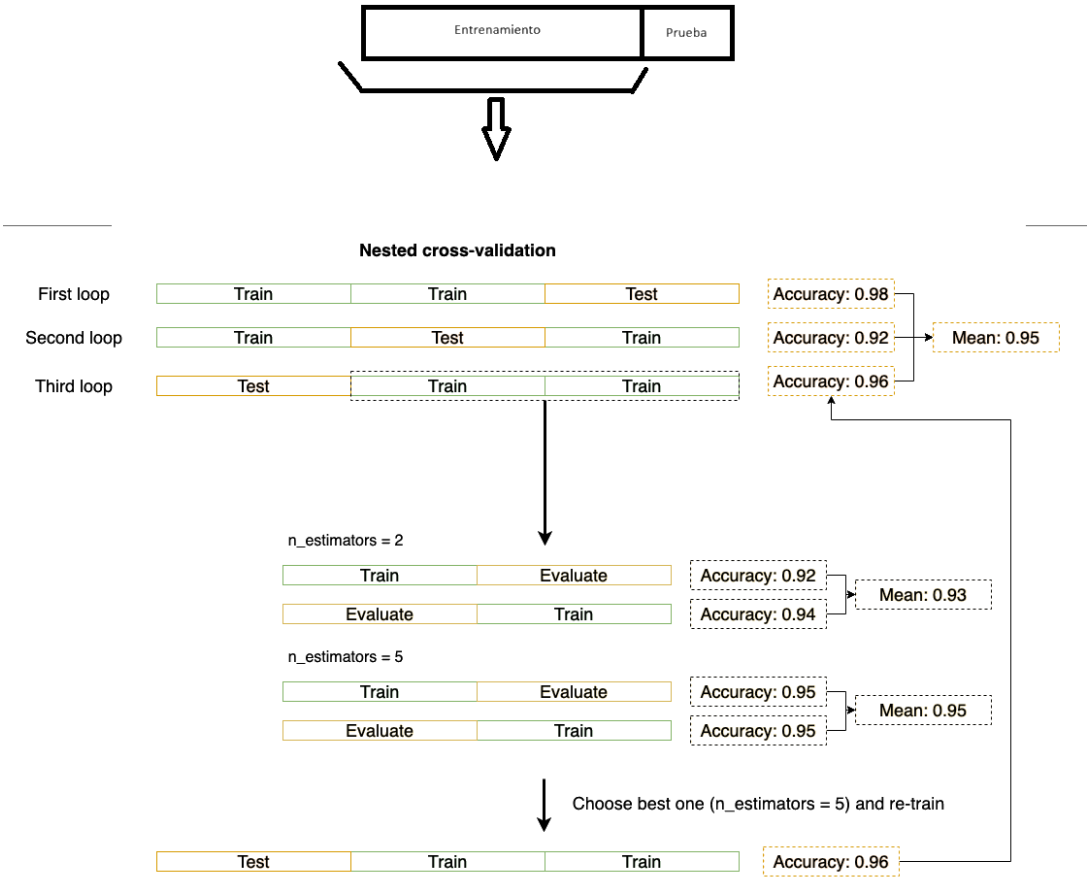
Run	Iteración	Costo/Fitness	TiempoIter
6	17	0.8804162372405642	51.81294822692871
6	18	0.8804162372405642	50.3688542842865
6	19	0.8804162372405642	44.28783345222473
6	20	0.8840311378013915	44.48114323616028
6	21	0.8840311378013915	45.666181802749634
6	22	0.8840311378013915	46.839799642562866
6	23	0.8840311378013915	46.277671337127686
6	24	0.8840311378013915	45.49969458580017
6	25	0.8840311378013915	42.54486846923828
6	26	0.8840311378013915	44.5674831867218
6	27	0.8840311378013915	51.536115646362305
6	28	0.8840311378013915	43.879324436187744
6	29	0.8840311378013915	52.81076455116272
6	30	0.8840311378013915	43.48630452156067
7	0	0.8304521860238283	43.48630452156067
7	1	0.8304521860238283	17.475841760635376
7	2	0.8312822102381494	23.05665111541748
7	3	0.8755241612870917	21.796114921569824
7	4	0.8755241612870917	23.02547836303711
7	5	0.8755241612870917	22.298980712890625
7	6	0.8755241612870917	24.541898727416992
7	7	0.8755241612870917	21.969618797302246
7	8	0.8758794076064385	22.437665700912476
7	9	0.8758794076064385	22.953636407852173
7	10	0.8766371697112773	24.534921884536743
7	11	0.8766371697112773	23.539386510849
7	12	0.8766371697112773	23.51658034324646
7	13	0.8766371697112773	19.81188178062439
7	14	0.8766371697112773	22.69292640686035
7	15	0.8766371697112773	26.380601167678833
7	16	0.8766371697112773	27.107287883758545
7	17	0.8769933153239572	26.328115463256836
7	18	0.8769933153239572	24.913848876953125
7	19	0.8769933153239572	23.128046989440918
7	20	0.8769933153239572	24.628626585006714
7	21	0.8769933153239572	24.037798404693604
7	22	0.8769933153239572	28.394699573516846
7	23	0.8769933153239572	24.71786403656006
7	24	0.8769933153239572	25.983225345611572
7	25	0.8769933153239572	24.485461235046387
7	26	0.8777039228904236	28.690797328948975
7	27	0.8777039228904236	26.335745334625244
7	28	0.8777039228904236	25.53127360343933
7	29	0.8777039228904236	24.80067777633667
7	30	0.8777039228904236	21.62926459312439



(a) Train-Test Split



(b) Train-Validation-Test Split



(c) Cross-Validation (No Shuffle)

Figura 6.1: Comparación de métodos de validación

Cuadro 6.8: Datos del archivo `w9_nomut_speedlimit.csv`

Run	Iteración	Costo/Fitness	TiempoIter
1	0	0.8279762685855298	5.9562395752395853
1	1	0.8279762685855298	2.2095484733581543
1	2	0.8279762685855298	2.9230051040649414
1	3	0.8279762685855298	9.401171207427979
1	4	0.8525014100922015	15.133528709411621
1	5	0.8744048818016849	22.40334963798523
1	6	0.8744048818016849	19.725903511047363
1	7	0.8744048818016849	15.82352614402771
1	8	0.8744048818016849	16.4849796295166
1	9	0.8744048818016849	20.908331155776978

Run	Iteración	Costo/Fitness	TiempoIter
1	10	0.874851429169943	21.057085037231445
1	11	0.8776058134281165	21.535558223724365
1	12	0.8776058134281165	14.668045043945312
1	13	0.8776058134281165	16.38143491744995
1	14	0.8776058134281165	15.264633655548096
1	15	0.8776058134281165	17.53962230682373
1	16	0.8776058134281165	19.37262201309204
1	17	0.8776058134281165	25.353495121002197
1	18	0.8776058134281165	21.13241147994995
1	19	0.8776058134281165	24.018659830093384
1	20	0.8776058134281165	18.16219401359558
1	21	0.8776058134281165	19.67044425010681
1	22	0.8776058134281165	24.78828525543213
1	23	0.8776058134281165	26.10194969177246
1	24	0.8776058134281165	25.947271585464478
1	25	0.8776058134281165	24.303781986236572
1	26	0.8776058134281165	27.714409589767456
1	27	0.8776058134281165	25.224951028823853
1	28	0.8776058134281165	23.532402992248535
1	29	0.8776058134281165	19.78283452987671
1	30	0.8776058134281165	22.464252948760986
2	0	0.8419007930597754	9.432523545456665
2	1	0.8419007930597754	8.039284944534302
2	2	0.8419007930597754	8.734791040420532
2	3	0.8574650958319696	9.08257532119751
2	4	0.8574650958319696	8.315770387649536
2	5	0.8574650958319696	8.881442785263062
2	6	0.8574650958319696	8.066135883331299
2	7	0.8574650958319696	8.077324151992798
2	8	0.8574650958319696	8.674825668334961
2	9	0.8574650958319696	8.841343402862549
2	10	0.8574650958319696	8.550532341003418
2	11	0.8574650958319696	8.982436180114746
2	12	0.8574650958319696	9.657135009765625
2	13	0.868249935998185	8.534435033798218
2	14	0.868249935998185	7.086875677108765
2	15	0.868249935998185	6.742965221405029
2	16	0.868249935998185	7.342882871627808
2	17	0.8703777493845614	8.916354656219482
2	18	0.87210525813887	9.984396934509277
2	19	0.87210525813887	10.968380689620972
2	20	0.87210525813887	13.637169122695923
2	21	0.87210525813887	15.435372829437256
2	22	0.8728307650788836	14.776111364364624
2	23	0.8728307650788836	13.268235445022583

Run	Iteración	Costo/Fitness	TiempoIter
2	24	0.8728307650788836	11.136013746261597
2	25	0.8728307650788836	11.226024150848389
2	26	0.8728307650788836	14.746661901473999
2	27	0.8728307650788836	33.564311504364014
2	28	0.8728307650788836	11.981794834136963
2	29	0.8746023353366098	13.332227230072021
2	30	0.8746023353366098	15.476420640945435
3	0	0.8539737997073482	7.243579149246216
3	1	0.8558850955423483	6.789034605026245
3	2	0.8717724226512494	11.439035892486572
3	3	0.8738737976188358	17.201905250549316
3	4	0.8738737976188358	19.242029905319214
3	5	0.8748372122283827	20.211219549179077
3	6	0.8748372122283827	21.73905611038208
3	7	0.8787817691198019	22.401259899139404
3	8	0.8787817691198019	22.30462908744812
3	9	0.8787953699668286	18.90191674232483
3	10	0.8787953699668286	16.947788953781128
3	11	0.8787953699668286	18.5948383808136
3	12	0.8787953699668286	25.48851490020752
3	13	0.8787953699668286	22.33894920349121
3	14	0.8787953699668286	20.33956027030945
3	15	0.8787953699668286	20.329079627990723
3	16	0.8787953699668286	22.720264196395874
3	17	0.8787953699668286	24.199524402618408
3	18	0.8787953699668286	25.53540873527527
3	19	0.8787953699668286	24.07249689102173
3	20	0.8790392539718688	27.188477277755737
3	21	0.8790392539718688	26.695369482040405
3	22	0.8790643109310385	33.0737988948822
3	23	0.8790643109310385	25.53951144218445
3	24	0.8790643109310385	22.267051458358765
3	25	0.8790643109310385	19.397595405578613
3	26	0.8790643109310385	20.187062978744507
3	27	0.8790643109310385	21.187257766723633
3	28	0.8790643109310385	25.258174419403076
3	29	0.8790643109310385	22.243795156478882
3	30	0.8790643109310385	22.489729404449463
4	0	0.8327031557159044	22.489729404449463
4	1	0.8327031557159044	10.701869010925293
4	2	0.8525147826564338	13.636813640594482
4	3	0.8714527693554045	21.049296855926514
4	4	0.8727676435408942	11.264357566833496
4	5	0.8727676435408942	6.600890398025513
4	6	0.8727676435408942	5.180272340774536

Run	Iteración	Costo/Fitness	TiempoIter
4	7	0.8727676435408942	7.706348896026611
4	8	0.8741146506046654	10.720192909240723
4	9	0.8755642477235615	11.255378723144531
4	10	0.8755642477235615	11.478447675704956
4	11	0.8755642477235615	9.854994058609009
4	12	0.8755642477235615	9.92008900642395
4	13	0.8755642477235615	8.353275537490845
4	14	0.8755642477235615	8.174203872680664
4	15	0.8755642477235615	9.760169267654419
4	16	0.8755642477235615	11.413787126541138
4	17	0.8755642477235615	12.827155113220215
4	18	0.8755642477235615	11.50020980834961
4	19	0.8755642477235615	9.183292627334595
4	20	0.8755642477235615	10.24988317489624
4	21	0.8755642477235615	11.42962646484375
4	22	0.8755642477235615	11.088425159454346
4	23	0.8755642477235615	11.325618505477905
4	24	0.8755642477235615	10.113517999649048
4	25	0.8755642477235615	10.695216178894043
4	26	0.8755642477235615	13.616304874420166
4	27	0.8755642477235615	14.196081638336182
4	28	0.8755642477235615	11.908468961715698
4	29	0.8755642477235615	11.118984699249268
4	30	0.8755642477235615	10.776690244674683
5	0	0.8310764644186058	10.776690244674683
5	1	0.8352394283232124	20.097368955612183
5	2	0.8528208387624565	11.615482568740845
5	3	0.8736767645010868	6.495919942855835
5	4	0.8820798139301738	7.595257520675659
5	5	0.8820798139301738	9.80849289894104
5	6	0.8820798139301738	10.706567287445068
5	7	0.8820798139301738	11.81576132774353
5	8	0.8820798139301738	11.379760503768921
5	9	0.8820798139301738	7.930609703063965
5	10	0.8820798139301738	9.810848474502563
5	11	0.8820798139301738	11.853264331817627
5	12	0.8820798139301738	12.348715543746948
5	13	0.8820798139301738	13.404436826705933
5	14	0.8820798139301738	9.494974851608276
5	15	0.8820798139301738	9.367644786834717
5	16	0.8820798139301738	11.011862754821777
5	17	0.8820798139301738	9.090226173400879
5	18	0.8820798139301738	9.518994331359863
5	19	0.8820798139301738	10.216181516647339
5	20	0.8820798139301738	11.452204942703247

Run	Iteración	Costo/Fitness	TiempoIter
5	21	0.8820798139301738	13.200129508972168
5	22	0.8825300472327035	14.558380603790283
5	23	0.8825300472327035	12.497781753540039
5	24	0.8825300472327035	10.587860107421875
5	25	0.8825300472327035	11.669413566589355
5	26	0.8825300472327035	11.224555492401123
5	27	0.8825300472327035	13.164191007614136
5	28	0.8825300472327035	13.651304006576538
5	29	0.8825300472327035	11.539177417755127
5	30	0.8825300472327035	12.16887617111206
6	0	0.8788855077524301	12.16887617111206
6	1	0.8788855077524301	32.49171495437622
6	2	0.8788855077524301	48.9544894695282
6	3	0.8799846415874022	77.92001581192017
6	4	0.8799846415874022	81.08318901062012
6	5	0.8802591532200914	78.88695359230042
6	6	0.8802591532200914	71.16994976997375
6	7	0.8802591532200914	66.62675070762634
6	8	0.8802591532200914	87.42048048973083
6	9	0.8802591532200914	60.3619441986084
6	10	0.8802591532200914	68.89302825927734
6	11	0.8802591532200914	70.24100637435913
6	12	0.8802591532200914	78.40693187713623
6	13	0.8802591532200914	100.32321262359619
6	14	0.8802591532200914	69.31803035736084
6	15	0.8802591532200914	69.64151620864868
6	16	0.8802591532200914	78.52819681167603
6	17	0.8802591532200914	89.80631303787231
6	18	0.8802591532200914	89.31334900856018
6	19	0.8802591532200914	65.58827567100525
6	20	0.8802591532200914	129.59538006782532
6	21	0.8803085831099103	63.29284477233887
6	22	0.8803085831099103	57.07614755630493
6	23	0.8803085831099103	57.23311424255371
6	24	0.8803085831099103	58.76535224914551
6	25	0.8803085831099103	343.39771485328674
6	26	0.8803085831099103	50.70886588096619
6	27	0.8803085831099103	62.512399196624756
6	28	0.8803085831099103	73.56450462341309
6	29	0.8803085831099103	67.8004789352417
6	30	0.8803085831099103	67.07644438743591
7	0	0.8347099734491354	28.472529888153076
7	1	0.83487692045849	28.67145800590515
7	2	0.8372514124785202	23.39138388633728
7	3	0.8662125234050627	22.22302532196045

Run	Iteración	Costo/Fitness	TiempoIter
7	4	0.8662125234050627	19.74673104286194
7	5	0.8707008120260153	19.55985164642334
7	6	0.8707008120260153	27.37162494659424
7	7	0.8773290733051565	30.19308853149414
7	8	0.8773290733051565	49.0477728843689
7	9	0.8780742893139362	53.19933724403381
7	10	0.8780742893139362	55.34209990501404
7	11	0.8783988137947464	43.34281373023987
7	12	0.8783988137947464	41.51761293411255
7	13	0.8783988137947464	36.78053545951843
7	14	0.8783988137947464	35.04979991912842
7	15	0.8783988137947464	34.541704416275024
7	16	0.8783988137947464	48.745177030563354
7	17	0.8783988137947464	51.85898399353027
7	18	0.8783988137947464	46.84814953804016
7	19	0.8783988137947464	51.066609382629395
7	20	0.8783988137947464	57.05413365364075
7	21	0.8783988137947464	45.22613883018494
7	22	0.8783988137947464	41.592939138412476
7	23	0.8783988137947464	37.942304372787476
7	24	0.8783988137947464	58.123169898986816
7	25	0.8783988137947464	44.99326515197754
7	26	0.8783988137947464	40.23967123031616
7	27	0.8783988137947464	52.044575214385986
7	28	0.8783988137947464	63.68763589859009
7	29	0.8783988137947464	51.131473541259766
7	30	0.8783988137947464	42.22018575668335

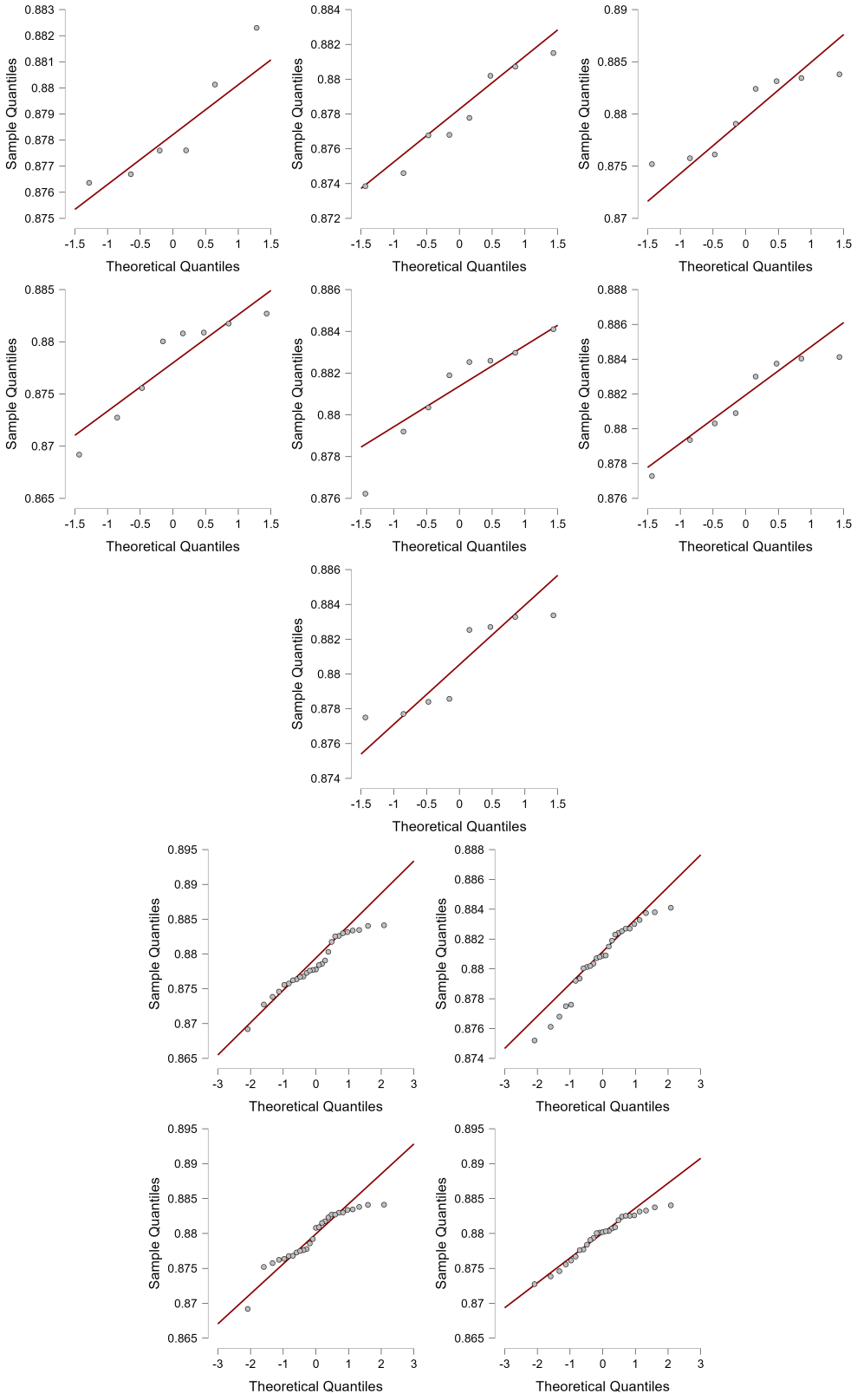


Figura 6.3: Normalidad de los residuos por variable

Bibliografía

[1] A Galindo Durán. Integración de la inteligencia artificial en la enseñanza de las artes plásticas. *Revista de ciencias sociales*, 29(4):17–29, 2023.

[2] Eduardo Biancas. Model selection done right: A gentle introduction to nested cross-validation. *Ploomber*, 2022.

[3] Gavin Cawley and Nicola Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 07 2010.

[4] J. Nocedal and S. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer New York, 2006.

[5] J. Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley series in artificial intelligence. Addison-Wesley Publishing Company, 1984.

[6] Tanay Agrawal. *Hyperparameter Optimization in Machine Learning: Make Your Machine Learning and Deep Learning Models More Efficient*. 01 2021.

[7] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013.

[8] Ágora Dixital. Árbol de decisiones. *Ágora Dixital*, unknown.

[9] Jacob Murel. What is ensemble learning? Technical report, IBM, 2024.

[10] Nimisha Singh. Soft margin svm: Exploring slack variables, the 'c' parameter, and flexibility. *Medium*, 2023.

[11] Ligdi Gonzalez. Regresión logística – teoría. *aprendeia*, 2019.

[12] J.P. Mueller and L. Massaron. *Machine Learning For Dummies*. For dummies. Wiley, 2016.

[13] Federico Marini and Beata Walczak. Particle swarm optimization (pso). a tutorial. *Chemometrics and Intelligent Laboratory Systems*, 149:153–165, 2015.

[14] Fernando Sancho Caparrini. Pso: Optimización por enjambres de partículas. *Models for the course Artificial Intelligence from the Bachelor's Degree in Çomputers Engineering.ºf ETSII-University of Seville*, 2020.

[15] Russell C Eberhart, Yuhui Shi, and James Kennedy. *Swarm intelligence*. Elsevier, 2001.

[16] Singiresu S Rao. *Engineering optimization: theory and practice*. John Wiley & Sons, 2019.

[17] Maxim Gurasov. Do i need to tune logistic regression hyperparameters? *Medium*, 2022.

[18] Abhishek Jain. Svm kernels and its type. *Medium*, 2024.

[19] Guillem Servera. Huge tennis database, 2024.

[20] Eric Bonabeau, Marco Dorigo, and Guy Theraulaz. Swarm intelligence : From natural to artificial systems / e. bonabeau, m. dorigo, g. theraulaz. 01 2001.

[21] Ahmed Gad. Particle swarm optimization algorithm and its applications: A systematic review. *Archives of Computational Methods in Engineering*, 29:2531–2561, 04 2022.

[22] Prokop Moravec and Pavel Rudolf. Application of a particle swarm optimization for shape optimization in hydraulic machinery. *EPJ Web of Conferences*, 143:02076, 01 2017.

[23] Mauro S. Innocente and Johann Sienz. Coefficients' settings in particle swarm optimization: Insight and guidelines, 2021.

[24] James McCaffrey. Inteligencia artificial optimización por enjambre de partículas. *MSDN Magazine*, 2011.

[25] Till Buchacher. Hyperparameter optimisation utilising a particle swarm approach. *Medium*, 2018.